

MACHINE LEARNING-2 PROJECT

DSBA

tasmiyasana2@gmail.com

DATE: 30/07/2025

Table of Content

1	PROBLEM STATEMENT	1
1.1	CONTEXT	1
1.1	OBJECTIVE	1
1.2	DATA DESCRIPTION.....	1
2	DATA OVERVIEW.....	2
2.1	DATASET ROWS & COLUMNS OVERVIEW	2
2.2	STATISTICAL SUMMARY OF DATASET	4
2.3	UNIQUE VALUE & COUNTS	5
2.4	MISSING & DUPLICATE VALUES.....	7
2.5	DATA CLEANING.....	8
3	EXPLORATORY DATA ANALYSIS (EDA)	9
3.1	UNIVARIATE ANALYSIS.....	9
3.2	BIVARIATE ANALYSIS	19
3.3	MULTIVARIATE ANALYSIS	33
3.4	OVERALL EDA INSIGHTS	35
4	DATA PROCESSING	36
4.1	DATA PREPARATION	36
4.2	FEATURE ENGINEERING	37
4.3	MISSING & DUPLICATE VALUES	37
4.4	OUTLIER DETECTION & TREATMENT.....	37
4.5	DATA PREPARATION FOR MODELING	39
5	MODEL BUILDING.....	40
5.1	MODEL EVALUATION CRITERIA.....	40
5.2	INITIAL MODEL BUILDING	41
5.2.1	<i>Decision Tree (DT) Initial Model Building.....</i>	<i>41</i>
5.2.2	<i>Bagging Classifier Initial Model</i>	<i>43</i>
5.2.3	<i>Boosting Initial Model Building.....</i>	<i>46</i>
5.2.4	<i>Gradient Boost Initial Model Building.....</i>	<i>48</i>
5.2.5	<i>XGBoost Initial Model Building</i>	<i>51</i>
6	MODEL BUILDING - OVERSAMPLED DATA	54
	OVERSAMPLING IS USED IN MODEL BUILDING TO ADDRESS CLASS IMBALANCE BY INCREASING THE REPRESENTATION OF MINORITY CLASS. WE WILL BE USING SMOTE (SYNTHETIC MINORITY OVERSAMPLING TECHNIQUE) TO PERFORM OVERSAMPLING.....	54
6.1	DECISION TREE (DT) - OVERSAMPLED DATA	55
6.2	BAGGING- RANDOM FOREST (RF) MODEL ON OVERSAMPLED	57
6.3	BOOSTING MODEL – OVERSAMPLED DATA	60
6.3.1	<i>AdaBoost Model – Oversampled Data.....</i>	<i>60</i>
6.3.2	<i>Gradient Boosting Model - Oversampled Data.....</i>	<i>63</i>
6.3.3	<i>XGBoost Model – Oversampled Data</i>	<i>66</i>
7	MODEL BUILDING – UNDERSAMPLED DATA	69
7.1	DECISION TREE (DT) - UNDERSAMPLED DATA	70
7.2	RANDOM FOREST (RF) USING BAGGING – UNDERSAMPLED DATA	72
7.3	BOOSTING MODELS - UNDERSAMPLED DATA.....	74
7.3.1	<i>AdaBoost Model – UnderSampled Data</i>	<i>74</i>
7.3.2	<i>Gradient Boosting (GB) - UnderSampled Data.....</i>	<i>77</i>
7.3.3	<i>XGBoost Model - UnderSampled Data.....</i>	<i>79</i>
8	HYPERPARAMETER TUNING	82
8.1	GRADIENT BOOSTING MODEL TUNING	83
8.2	XGBOOST (XGB) MODEL TUNING	86

8.3	RANDOM FOREST (RF) MODEL TUNING	88
8.4	ADABOOST MODEL TUNING.....	91
9	MODEL COMPARISON & FINAL MODEL SELECTION	94
10	FINAL MODEL - TUNED RANDOM FOREST.....	95
10.1	PERFORMANCE EVALUATION ON TEST	95
10.2	CONFUSION MATRIX.....	96
10.3	PREDICTIONS ON TEST & CLASSIFICATION REPORT	96
10.4	FEATURE IMPORTANCE ANALYSIS.....	97
11	ACTIONABLE INSIGHTS.....	99
12	BUSINESS RECOMMENDATIONS.....	100

List of Figures

FIGURE 1. DUPLICATE VALUES.....	8
FIGURE 2. TOTAL NEGATIVE VALUES AFTER TREATMENT	9
FIGURE 3. HISTOGRAM & BOX PLOT OF NO_OF_EMPLOYEES.....	10
FIGURE 4. HISTOGRAM & BOX PLOT OF YR_OF_ESTAB	11
FIGURE 5. HISTOGRAM & BOX PLOT OF PREVAILING_WAGE	11
FIGURE 6. DISTRIBUTION OF CONTINENT	12
FIGURE 7. DISTRIBUTION OF EDUCATION_OF_EMPLOYEE	13
FIGURE 8. DISTRIBUTION OF REGION_OF_EMPLOYMENT	14
FIGURE 9. DISTRIBUTION OF UNIT_OF_WAGE	15
FIGURE 10. DISTRIBUTION OF HAS_JOB_EXPERIENCE.....	16
FIGURE 11. DISTRIBUTION OF REQUIRES_JOB_TRAINING.....	17
FIGURE 12. DISTRIBUTION OF FULL_TIME_POSITION	17
FIGURE 13. DISTRIBUTION OF CASE_STATUS.....	18
FIGURE 14. LABEL ENCODING	19
FIGURE 15. HEATMAP ANALYSIS	19
FIGURE 16. EDUCATION_OF_EMPLOYEE VS CASE_STATUS	21
FIGURE 17. UNIT_OF_WAGE VS CASE_STATUS	22
FIGURE 18. CONTINENT VS CASE_STATUS	23
FIGURE 19. HAS_JOB_EXPERIENCE VS CASE_STATUS	24
FIGURE 20. REQUIRES_JOB_TRAINING VS CASE_STATUS.....	25
FIGURE 21. PREVAILING_WAGE VS CASE_STATUS.....	26
FIGURE 22. NO_OF_EMPLOYEES VS CASE_STATUS	27
FIGURE 23. YR_OF_ESTAB VS CASE_STATUS	28
FIGURE 24. REGION_OF_EMPLOYMENT VS CASE_STATUS	29
FIGURE 25. FULL_TIME_POSITION VS CASE_STATUS	30
FIGURE 26. EDUCATION_OF_EMPLOYEE VS CONTINENT	31
FIGURE 27. HAS_JOB_EXPERIENCE VS CONTINENT	32
FIGURE 28. EDUCATION_OF_EMPLOYEE AND HAS_JOB_EXPERIENCE VS CASE_STATUS	34
FIGURE 29. OUTLIER DETECTION USING BOX PLOT.....	37
FIGURE 30. OUTLIERS DETECTION USING IQR METHOD	38
FIGURE 31. DECISION TREE INITIAL MODEL BUILDING	41
FIGURE 32. CONFUSION MATRIX-INITIAL DT (TRAIN)	42
FIGURE 33. CONFUSION MATRIX-INITIAL DT(VALIDATION)	43
FIGURE 34. RANDOM FOREST (BAGGING) INITIAL MODEL.....	43
FIGURE 35. CONFUSION MATRIX-INITIAL RF (TRAIN).....	45
FIGURE 36. CONFUSION MATRIX-INITIAL RF (VALIDATION).....	45
FIGURE 37. INITIAL ADABOOST MODEL BUILDING	46
FIGURE 38. CONFUSION MATRIX-INITIAL ADABOOST (TRAIN)	47
FIGURE 39. CONFUSION MATRIX-INITIAL ADABOOST (VALIDATION)	48
FIGURE 40. GRADIENT BOOST INITIAL MODEL BUILDING	48

FIGURE 41. CONFUSION MATRIX-GRADIENT BOOSTING(TRAIN).....	50
FIGURE 42. CONFUSION MATRIX-GRADIENT BOOSTING(VALIDATION).....	50
FIGURE 43. XGBOOST INITIAL MODEL BUILDING	51
FIGURE 44. CONFUSION MATRIX-XGBOOST INITIAL MODEL (TRAIN).....	52
FIGURE 45. CONFUSION MATRIX-XGBOOST INITIAL MODEL (VALIDATION).....	53
FIGURE 46. DECISION TREE OVERSAMPLED MODEL	55
FIGURE 47. CONFUSION MATRIX-DT (TRAIN-OVERSAMPLED)	56
FIGURE 48. CONFUSION MATRIX-DT (VALIDATION-OVERSAMPLED)	57
FIGURE 49. RANDOM FOREST OVERSAMPLED MODEL	57
FIGURE 50. CONFUSION MATRIX-RF TRAIN (OVERSAMPLED)	58
FIGURE 51. CONFUSION MATRIX-RF VALIDATION (OVERSAMPLED).....	59
FIGURE 52. ADABOOST OVERSAMPLED MODEL	60
FIGURE 53. CONFUSION MATRIX-ADABOOST OVERSAMPLED (TRAIN).....	61
FIGURE 54. CONFUSION MATRIX-ADABOOST OVERSAMPLED (VALIDATION)	62
FIGURE 55. GRADIENT BOOSTING OVERSAMPLED MODEL.....	63
FIGURE 56. CONFUSION MATRIX - GB OVERSAMPLED (TRAIN)	64
FIGURE 57. CONFUSION MATRIX - GB OVERSAMPLED (VALIDATION).....	65
FIGURE 58. XGBOOST OVERSAMPLED MODEL	66
FIGURE 59. XGBOOST OVERSAMPLED TRAIN-CONFUSION MATRIX.....	67
FIGURE 60. XGBOOST OVERSAMPLED VALIDATION-CONFUSION MATRIX.....	68
FIGURE 61. CLASS DISTRIBUTION BEFORE UNDERSAMPLING	69
FIGURE 62. CLASS DISTRIBUTION AFTER UNDERSAMPLING	69
FIGURE 63. CONFUSION MATRIX (TRAIN) - DT UNDERSAMPLED	71
FIGURE 64. CONFUSION MATRIX (VALIDATION) - DT UNDERSAMPLED	71
FIGURE 65. CONFUSION MATRIX RF UNDERSAMPLED (TRAIN)	73
FIGURE 66. CONFUSION MATRIX RF UNDERSAMPLED (VALIDATION)	74
FIGURE 67. CONFUSION MATRIX AdaBOOST UNDERSAMPLED (TRAIN).....	76
FIGURE 68. CONFUSION MATRIX AdaBOOST UNDERSAMPLED (VALIDATION).....	76
FIGURE 69. CONFUSION MATRIX GB UNDERSAMPLED (TRAIN)	78
FIGURE 70. CONFUSION MATRIX GB UNDERSAMPLED (VALIDATION)	79
FIGURE 71. CONFUSION MATRIX XGB UNDERSAMPLED (TRAIN)	81
FIGURE 72. CONFUSION MATRIX XGB UNDERSAMPLED (VALIDATION).....	81
FIGURE 73. CLASS WEIGHT & SCALE_POS_WEIGHT	83
FIGURE 74. GB HYPERPARAMETER TUNING.....	83
FIGURE 75. CONFUSION MATRIX TUNED GB (TRAIN)	85
FIGURE 76. CONFUSION MATRIX TUNED GB (VALIDATION)	85
FIGURE 77. XGBOOST MODEL TUNING	86
FIGURE 78. CONFUSION MATRIX TUNED XGB (TRAIN)	87
FIGURE 79. CONFUSION MATRIX TUNED XGB (VALIDATION)	88
FIGURE 80. RANDOM FOREST MODEL TUNING	89
FIGURE 81. CONFUSION MATRIX TUNED RF (TRAIN)	90
FIGURE 82. CONFUSION MATRIX TUNED RF (VALIDATION)	91
FIGURE 83. ADABOOST MODEL TUNING	91
FIGURE 84. TUNED ADABOOST - CONFUSION MATRIX (TRAIN).....	93
FIGURE 85. TUNED ADABOOST - CONFUSION MATRIX (VALIDATION).....	93
FIGURE 86. TUNED RF CONFUSION MATRIX (TEST)	96
FIGURE 87. FEATURE IMPORTANCE PLOT	98

List of Tables

TABLE 1. TOP 5 ROWS.....	2
TABLE 2. BOTTOM 5 ROWS	3
TABLE 3. DATA TYPES.....	3
TABLE 4. STATISTICAL SUMMARY.....	4
TABLE 5. UNIQUE VALUE & COUNTS	5
TABLE 6. CATEGORICAL COLUMNS UNIQUE VALUE & COUNTS.....	6
TABLE 7. MISSING VALUES	7
TABLE 8. TOP 5 ROWS AFTER DROPPING CASE_ID	8
TABLE 9. NEGATIVE VALUES IN NO_OF_EMPLOYEES	9
TABLE 10. CONVERTED DTYPE OBJECT INTO CATEGORY.....	36
TABLE 11. ONE- HOT ENCODING	39
TABLE 12. TRAIN- VALIDATION- TEST SPLIT DISTRIBUTION	40
TABLE 13. DECISION TREE BASE MODEL TRAINING PERFORMANCE.....	41
TABLE 14. DECISION TREE BASE MODEL VALIDATION PERFORMANCE	42
TABLE 15. RANDOM FOREST INITIAL MODEL TRAIN PERFORMANCE	43
TABLE 16. RANDOM FOREST INITIAL MODEL VALIDATION PERFORMANCE	44
TABLE 17. ADABOOST INITIAL MODEL TRAIN PERFORMANCE	46
TABLE 18. ADABOOST INITIAL MODEL VALIDATION PERFORMANCE	47
TABLE 19. TRAIN PERFORMANCE-GRADIENT BOOST INITIAL MODEL	49
TABLE 20. VALIDATION PERFORMANCE-GRADIENT INITIAL BOOSTING MODEL	49
TABLE 21. TRAIN PERFORMANCE-XGBOOST INITIAL MODEL	51
TABLE 22. VALIDATION PERFORMANCE-XGBOOST INITIAL MODEL	52
TABLE 23. CLASS DISTRIBUTION BEFORE OVERSAMPLING	54
TABLE 24. CLASS DISTRIBUTION AFTER OVERSAMPLING.....	54
TABLE 25. DT - TRAINING PERFORMANCE (OVERSAMPLED).....	55
TABLE 26. DT - VALIDATION PERFORMANCE (OVERSAMPLED).....	56
TABLE 27. TRAIN PERFORMANCE – RF OVERSAMPLED	57
TABLE 28. VALIDATION PERFORMANCE – RF OVERSAMPLED	58
TABLE 29. TRAIN PERFORMANCE - ADABOOST OVERSAMPLED	60
TABLE 30. VALIDATION PERFORMANCE - ADABOOST OVERSAMPLED	60
TABLE 31. GB OVERSAMPLED - TRAIN PERFORMANCE	63
TABLE 32. GB OVERSAMPLED - VALIDATION PERFORMANCE	63
TABLE 33. XGBOOST OVERSAMPLED - TRAIN PERFORMANCE	66
TABLE 34. XGBOOST OVERSAMPLED - VALIDATION PERFORMANCE	67
TABLE 35. DECISION TREE (DT) UNDERSAMPLED MODEL.....	70
TABLE 36. DECISION TREE (UNDERSAMPLED) TRAIN PERFORMANCE	70
TABLE 37. DECISION TREE (UNDERSAMPLED) VALIDATION PERFORMANCE.....	70
TABLE 38. RANDOM FOREST-UNDERSAMPLED MODEL.....	72
TABLE 39. RANDOM FOREST (UNDERSAMPLED) TRAIN PERFORMANCE.....	72
TABLE 40. RANDOM FOREST (UNDERSAMPLED) VALIDATION PERFORMANCE.....	73
TABLE 41. ADABOOST UNDERSAMPLED MODEL.....	75
TABLE 42. ADABOOST TRAIN PERFORMANCE (UNDERSAMPLED)	75
TABLE 43. ADABOOST VALIDATION PERFORMANCE (UNDERSAMPLED).....	75
TABLE 44. GB UNDERSAMPLED MODEL	77
TABLE 45. GB TRAIN PERFORMANCE (UNDERSAMPLED).....	77
TABLE 46. GB VALIDATION PERFORMANCE (UNDERSAMPLED)	78
TABLE 47. XGB UNDERSAMPLED MODEL	79
TABLE 48. XGB TRAIN PERFORMANCE (UNDERSAMPLED).....	80
TABLE 49. XGB VALIDATION PERFORMANCE (UNDERSAMPLED)	80
TABLE 50. TUNED GB TRAIN PERFORMANCE	83
TABLE 51. TUNED GB VALIDATION PERFORMANCE.....	84
TABLE 52. TUNED XGB TRAIN PERFORMANCE.....	86
TABLE 53. TUNED XGB VALIDATION PERFORMANCE.....	87
TABLE 54. TUNED RF TRAIN PERFORMANCE	89
TABLE 55. TUNED RF VALIDATION PERFORMANCE	89
TABLE 56. TUNED ADABOOST TRAIN PERFORMANCE	92
TABLE 57. TUNED ADABOOST VALIDATION PERFORMANCE	92

TABLE 58. TUNED MODELS TRAINING PERFORMANCE COMPARISON.....	94
TABLE 59. TUNED MODELS VALIDATION PERFORMANCE COMPARISON	94
TABLE 60. TUNED RF TEST PERFORMANCE	95
TABLE 61. PREDICTIONS ON TEST & CLASSIFICATION REPORT	96
TABLE 62. FEATURE RANKING.....	97

List of Abbreviations

1. Decision Tree - DT
2. Random Forest - RF
3. Gradient Boosting - GB
4. Extreme Gradient Boosting - XGB

1 PROBLEM STATEMENT

1.1 Context

Business communities in the United States face a strong demand for human resources, but one of the most difficult issues is finding and attracting the proper talent, which is likely the most crucial factor in staying competitive. Companies in the United States strive for hard-working, skilled, and qualified professionals both locally and internationally.

The US Immigration and Nationality Act (INA) allows foreign workers to work in the United States on a temporary or permanent basis. The act also safeguards US workers from negative effects on their salaries or working conditions by requiring US firms to comply with statutory requirements when hiring foreign workers to fill labour shortages. The Office of Foreign Labour Certification (OFLC) administers the immigration programs.

OFLC processes job certification applications for employers seeking to bring foreign workers into the United States and grants certifications in cases where employers can demonstrate that there are insufficient US workers available to perform the work at wages equal to or greater than the wage paid for the occupation in the area of intended employment.

1.1 Objective

In Fiscal Year 2016, the OFLC handled 775,979 employer applications for 1,699,957 temporary and permanent labour certificates. The overall number of processed applications increased by nine percent compared to the previous year. As the number of applications grows year after year, the procedure of examining each case becomes increasingly time-consuming.

The increasing number of applicants each year necessitates a Machine Learning-based solution that can assist in shortlisting prospects with a better possibility of VISA approval. OFLC has recruited the firm EasyVisa to provide data-driven solutions. As a data scientist at EasyVisa, we have to analyse the provided data and, using a classification model to:

- Make the visa approval procedure easier.
- Recommend a suitable profile for applicants whose visas should be confirmed or rejected depending on the factors that significantly influence the case status.

1.2 Data Description

The data covers numerous attributes of both the employee and the employer. Below is a detailed data dictionary.

- case_id: ID of each visa application
- continent: Information of continent the employee
- education_of_employee: Information of education of the employee
- has_job_experience: Does the employee have any job experience? Y= Yes; N = No
- requires_job_training: Does the employee require any job training? Y = Yes; N = No
- no_of_employees: Number of employees in the employer's company

- yr_of_estab: Year in which the employer's company was established
- region_of_employment: Information of foreign worker's intended region of employment in the US.
- prevailing_wage: Average wage paid to similarly employed workers in a specific occupation in the area of intended employment. The purpose of the prevailing wage is to ensure that the foreign worker is not underpaid compared to other workers offering the same or similar service in the same area of employment.
- unit_of_wage: Unit of prevailing wage. Values include Hourly, Weekly, Monthly, and Yearly.
- full_time_position: Is the position of work full-time? Y = Full-Time Position; N = Part-Time Position
- case_status: Flag indicating if the Visa was certified or denied

2 DATA OVERVIEW



- Finding out the data types of the columns to ensure that data is stored in the preferred format and the value of each property is as expected.
- Checking the statistical summary of the dataset to get an overview of the numerical columns of the dataset.

Displaying top 5 rows of dataset:

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab
0	EZYV01	Asia	High School	N	N	14513	2007
1	EZYV02	Asia	Master's	Y	N	2412	2002
2	EZYV03	Asia	Bachelor's	N	Y	44444	2008
3	EZYV04	Asia	Bachelor's	N	N	98	1897
4	EZYV05	Africa	Master's	Y	N	1082	2005
	region_of_employment prevailing_wage unit_of_wage full_time_position case_status						
	West	592.2029	Hour	Y	Denied		
	Northeast	83425.6500	Year	Y	Certified		
	West	122996.8600	Year	Y	Denied		
	West	83434.0300	Year	Y	Denied		
	South	149907.3900	Year	Y	Certified		

Displaying bottom 5 rows of dataset:

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab
25475	EZYV25476	Asia	Bachelor's	Y	Y	2601	2008
25476	EZYV25477	Asia	High School	Y	N	3274	2006
25477	EZYV25478	Asia	Master's	Y	N	1121	1910
25478	EZYV25479	Asia	Master's	Y	Y	1918	1887
25479	EZYV25480	Asia	Bachelor's	Y	N	3195	1960

The screenshot shows a Google Colab notebook interface. The top part displays the shape of the 'visa' dataset as (25480, 12). Below this, the 'Observations' section states: 'Dataset has 25,480 rows and 12 columns.' The next section, 'Checking Data Types', shows the output of the 'visa.info()' command. This output provides a detailed summary of the dataset, including the number of entries, the total number of columns, and a table of column names, non-null counts, and data types. The data types are listed as float64(1), int64(2), and object(9). The memory usage is noted as 2.3+ MB.

```
[6] visa.shape
(25480, 12)

Observations:
• Dataset has 25,480 rows and 12 columns.

Checking Data Types

[7] visa.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   case_id                               25480 non-null  object
1   continent                             25480 non-null  object
2   education_of_employee                 25480 non-null  object
3   has_job_experience                    25480 non-null  object
4   requires_job_training                 25480 non-null  object
5   no_of_employees                      25480 non-null  int64
6   yr_of_estab                          25480 non-null  int64
7   region_of_employment                 25480 non-null  object
8   prevailing_wage                      25480 non-null  float64
9   unit_of_wage                         25480 non-null  object
10  full_time_position                   25480 non-null  object
11  case_status                          25480 non-null  object
dtypes: float64(1), int64(2), object(9)
memory usage: 2.3+ MB
```

Checking the d:

Observations:

- There are 9 categorical variables: "case_id", "continent", "education_of_employee", "has_job_experience", "requires_job_training", "region_of_employment", "unit_of_wage", "full_time_position", "case_status" (target variable).
- There are only three numerical variables: "no_of_employees", "yr_of_estab", "prevailing_wage".

File Edit View Insert Runtime Tools Help

Q Commands

+ Code + Text

▶ Run all ▼

2.2 Statistical Summary

Checking statistical summary

Observations

Number of Employees

- The average company size is 5,667 employees, with a very high standard deviation of 22,878, indicating a wide range of company sizes.
- The minimum value is -26, which is not valid and suggests a data entry error or quality issue as number of employees cannot be negative.
- The maximum value is 602,069, indicating that a few very large organizations act as outliers.
- Most companies, 75% of the data, have fewer than 3,504 employees.

Year of Establishment

- The average year of establishment is around 1979.
- The minimum value is 1800, which is realistic suggesting old establishment, and could be an outlier.
- The maximum value is 2016, which matches the latest application year in the dataset.
- The median year is 1997, indicating that most sponsoring companies are relatively modern, although some very old institutions do exist.

Prevailing Wage

- The average prevailing wage is about \$74,456, with significant variation, roughly 52,816.
- The minimum value is \$2.13, which is below the legal minimum for wages in the US and suggests possible data entry issues or incorrect unit reporting. This needs further investigation.
- The maximum value is \$319,210, reflecting high-skilled or executive-level jobs.
- The median wage is 70,308, and 75th percentile is 107,735, indicating a positively skewed distribution.

Conclusions:

- All three numeric variables show extreme values and outliers, with impossible minimum values like negative employees, a wage of \$2.13, and an establishment year of 1800 which could be an outlier.

Checking Statistical Summary

visa.describe()

	no_of_employees	yr_of_estab	prevailing_wage
count	25480.000000	25480.000000	25480.000000
mean	5667.043210	1979.409929	74455.814592
std	22877.928848	42.366929	52815.942327
min	-26.000000	1800.000000	2.136700
25%	1022.000000	1976.000000	34015.480000
50%	2109.000000	1997.000000	70308.210000
75%	3504.000000	2005.000000	107735.512500
max	602069.000000	2016.000000	319210.270000

Observations:

Number of Employees (no_of_employees)

- The average company size is 5,667 employees, with a very high standard deviation of 22,878, indicating a wide range of company sizes.
- The minimum value is -26, which is not valid and suggests a data entry error or quality issue as number of employees cannot be negative.
- The maximum value is 602,069, indicating that a few very large organizations act as outliers.
- Most companies, 75% of the data, have fewer than 3,504 employees.

Year of Establishment (yr_of_estab)

- The average year of establishment is around 1979.
- The minimum value is 1800, which is realistic suggesting old establishment, and could be an outlier.
- The maximum value is 2016, which matches the latest application year in the dataset.
- The median year is 1997, indicating that most sponsoring companies are relatively modern, although some very old institutions do exist.

Prevailing Wage (prevailing_wage)

- The average prevailing wage is about \$74,456, with significant variation, roughly 52,816.
- The minimum value is \$2.13, which is below the legal minimum for wages in the US and suggests possible data entry issues or incorrect unit reporting. This needs further investigation.
- The maximum value is \$319,210, reflecting high-skilled or executive-level jobs.
- The median wage is 70,308, and 75th percentile is 107,735, indicating a positively skewed distribution.

Conclusions:

- All three numeric variables show extreme values and outliers, with impossible minimum values like negative employees, a wage of \$2.13, and an establishment year of 1800 which could be an outlier.

4

2.3 Unique Value & Counts

Checking unique value & counts

Unique values are:		Column:	
	0	case_id	['EZYV01' 'EZYV02' 'EZYV03' ... 'EZYV25478' 'EZYV25479' 'EZYV25480']
case_id	25480	continent	['Asia' 'Africa' 'North America' 'Europe' 'South America' 'Oceania']
continent	6	education_of_employee	['High School' 'Master's' 'Bachelor's' 'Doctorate']
education_of_employee	4	has_job_experience	['N' 'Y']
has_job_experience	2	requires_job_training	['N' 'Y']
requires_job_training	2	no_of_employees	7105
no_of_employees	7105	yr_of_estab	199
yr_of_estab	199	region_of_employment	['West' 'Northeast' 'South' 'Midwest' 'Island']
region_of_employment	5	prevailing_wage	25454
prevailing_wage	25454	unit_of_wage	['Hour' 'Year' 'Week' 'Month']
unit_of_wage	4	full_time_position	['Y' 'N']
full_time_position	2	case_status	['Denied' 'Certified']
case_status	2		
dtype: int64			

- **prevailing_wage** also has **25,454** unique values, almost as many as rows. This shows it is a continuous variable with very high distinctness.
- **no_of_employees** has **7,105** unique values, which confirms a wide range in company sizes.
- **yr_of_estab** has **199** unique values, which is reasonable since it covers more than two centuries of company founding years.

2. Categorical Variables

- **continent** has **6** unique values, which matches the number of inhabited continents.
- **education_of_employee** has **4** categories, which seems manageable. These are likely High School, Bachelor's, Master's, and Doctorate.
- **region_of_employment** has **5** categories that correspond to US regions.
- **unit_of_wage** has **4** categories: Hourly, Weekly, Monthly, and Yearly.
- **has_job_experience**, **requires_job_training**, **full_time_position**, and **case_status** all have **2** unique values each, which confirms they are binary categorical variables.

3.Target Variable

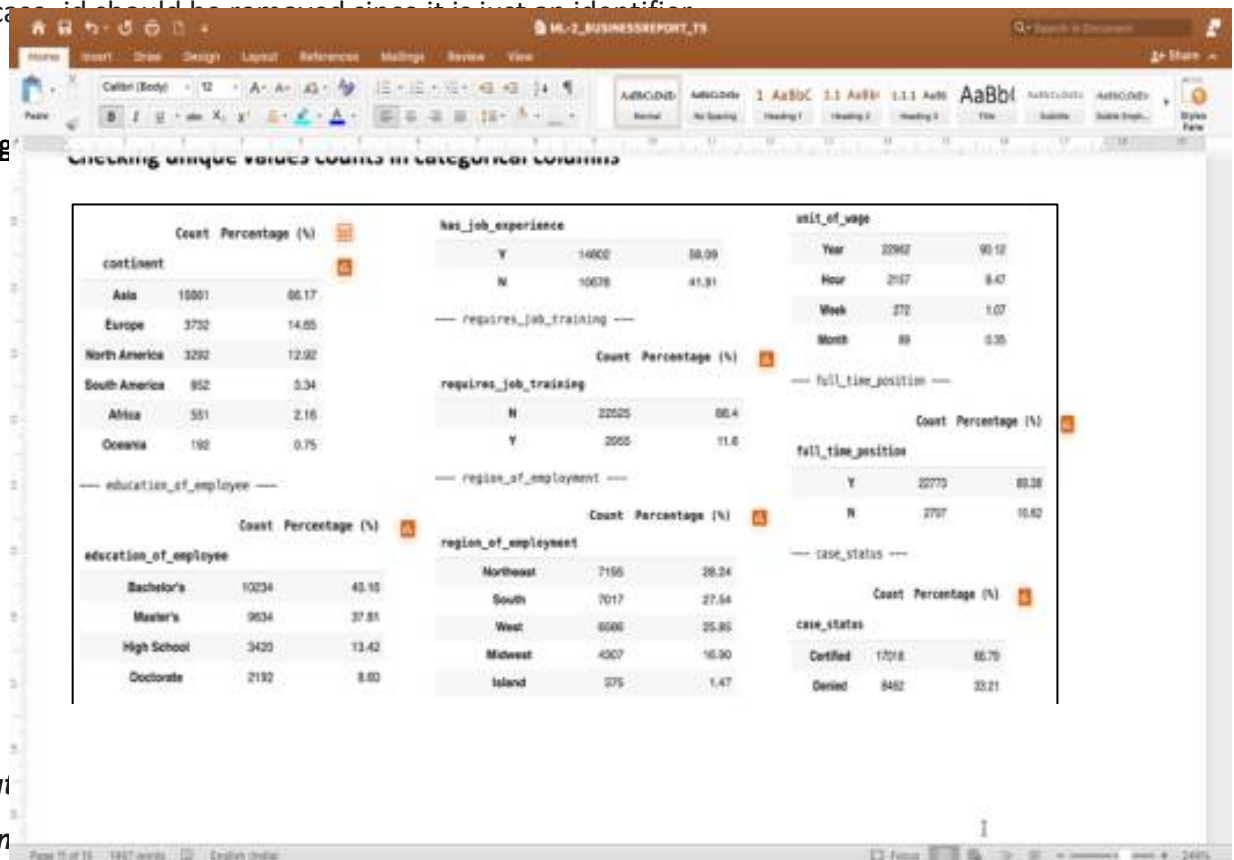
- **case_status** (Certified/Denied) has 2 unique values, making it a binary classification problem.

Conclusions

- The dataset includes a mix of high-uniqueness continuous variables, such as wages, employees, and establishment year, along with low-cardinality categorical variables, such as continent, education, and job experience.
- Most categorical variables have relatively few unique values, which makes them easy to binary encode.
- Some variables, like prevailing_wage and no_of_employees, may need scaling and outlier handling due to their wide ranges.

- ~~case_status should be removed since it is just an identifier~~

Checking



Observa

Continen

- Strong Asian Majority: **Most applications come from Asia at 66.17%**, far exceeding other regions.
- European & North American Contribution: Europe provides **14.65%** and North America accounts for **12.92%**.
- Minority Regions: Applications from South America (**3.34%**), Africa (**2.16%**), and Oceania (**0.75%**) are relatively rare.

education_of_employee

- Highly Educated Pool: **More than three-quarters (77.97%) of applicants hold an advanced degree.** Of these, **40.16%** have a Bachelor's and **37.81%** have a Master's.
- Significant Doctoral Presence: Those with a Doctorate make up a small minority at **8.6%**.
- Low High School Representation: Only **13.42%** of applicants have a High School diploma as their highest degree.

has_job_experience

- Experienced Majority: A clear majority (**58.09%**) of applicants have prior job experience.
- Inexperienced Minority: A notable portion (**41.91%**) apply without any job experience.

requires_job_training

- Low Training Need: Most positions, **88.4%**, do not require specific job training.
- Niche Training Roles: Only **11.6%** of visa applications are for positions that require training.

region_of_employment

- Even Coastal Distribution: Job opportunities mainly focus on the Northeast (**28.24%**) and West (**25.85%**), with the South (**27.54%**) also being a key area.
- Lower Midwest & Island Share: The Midwest (**16.9%**) has fewer opportunities, while positions in Island territories (**1.47%**) are very uncommon.

unit_of_wage

- Annual Salary Standard: The vast majority of positions (**90.12%**) offer a yearly salary.
- Hourly Roles Exist: A small but significant portion (**8.47%**) pay on an hourly basis.
- Weekly/Monthly Rarity: Contracts specifying Weekly (**1.07%**) or Monthly (**0.35%**) pay are quite rare.

full_time_position

- Overwhelmingly Full-Time: The visa program is mainly used for full-time positions (**89.38%**).
- Limited Part-Time Scope: Part-time roles make up a small minority (**10.62%**) of applications.

case_status

- High Approval Rate: Nearly two-thirds (**66.79%**) of all applications are Certified.
- Significant Denial Rate: A notable portion (**33.21%**) of applications are Denied, showing a competitive or strict process.

2.4 Missing & Duplicate Values

Checking missing values

Missing values:	
case_id	0
continent	0
education_of_employee	0
has_job_experience	0
requires_job_training	0
no_of_employees	0
yr_of_estab	0
region_of_employment	0
prevailing_wage	0
unit_of_wage	0
full_time_position	0
case_status	0
dtype: int64	

Observations:

- There are no missing values in our dataset. This suggest our dataset do not require any imputin data integrity.

Dropping Unnecessary Variables

Observations:

- There are **no** missing techniques. There are

Checking duplicate value:

- Most categorical variables have relatively few unique values, which makes them
- Some variables, like prevailing_wage and no_of_employees, may need scaling
- case_id should be removed since it is just an identifier.

Checking for Duplicates

```
[11] print("Number of duplicate:")  
visa.duplicated().sum()
```

```
Number of duplicate:  
np.int64(0)
```

Great Learning ML-2_PROJECT_TS.ipynb

colab.research.google.com/drive/1hP-nNjS-gnterHArHbK6x-KdVN7LA1#scrollTo=dHWU3L08Ea

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[14] df.shape

(25480, 11)

Checking the Top 5 Rows

[15] df.head()

	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab	region_of_employment	prevailing_wage	unit_of_w
0	Asia	High School	N	N	14513	2007	West	592.2029	H
1	Asia	Master's	Y	N	2412	2002	Northeast	83425.6500	Y
2	Asia	Bachelor's	N	Y	44444	2008	West	122996.8600	Y
3	Asia	Bachelor's	N	N	98	1897	West	83434.0300	Y
4	Africa	Master's	Y	N	1082	2005	South	148907.3900	Y

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

Observations:

- We can see that we have 11 columns and this confirms case_id is dropped successfully.

Treating Negative Values

- We know there are negative values in no_of_employees where they don't make logical sense, as it indicates data entry errors or anomalies that could skew our analysis.

Variables Terminal 22:20 Python 3

indicates data entry errors or anomalies that could skew our analysis. Therefore, we will address these negative values beforehand, to ensure that the trends, patterns, and descriptive statistics we observe in EDA are reflective of the true data characteristics.

Number of rows with negative 'no_of_employees': 33

	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab	region_of_employment	prevailing_wage
245	Europe	Master's	N	N	-25	1908	Northwest	39452.0000
378	Asia	Bachelor's	N	Y	-11	2011	Northwest	32586.1400
832	South America	Master's	Y	N	-17	2002	South	125782.0400
2928	Asia	Master's	Y	N	-26	2005	Midwest	112799.4000
6439	Asia	Bachelor's	N	N	-14	2013	South	583.5700
6634	Asia	Bachelor's	Y	N	-26	1923	West	1247.3200
7224	Europe	Doctorate	N	N	-21	1998	Midwest	141425.0500
7281	Asia	High School	N	N	-14	2000	Midwest	59488.5000
7228	Asia	Bachelor's	Y	Y	-20	2000	South	110409.6200
7781	Asia	Master's	N	N	-11	2009	Midwest	38457.3200
9872	Europe	Master's	Y	N	-26	1996	South	37397.8500
11493	Asia	High School	Y	N	-14	1999	South	27599.3500
13471	North America	Master's	N	N	-17	2003	Northwest	367.3400
14822	Asia							
14546	Asia							
14726	Asia							
15600	Asia							
15850	Asia							
16257	Asia							
16883	North America							
17005	Asia							
17655	North America							
17944	Asia							
17983	Asia							
20015	Asia							
20984	Europe							
21255	North America							
21768	Asia							
21944	Africa							
22084	North America							
22388	Asia							
22388	Asia							
23470	Europe							

Observations:

- There are **33 rows** with negative values
- The negative values have a data entry error
- We **imputed** these 33 values by absolute value transformation

```

[18] # fixing -ve values using .abs() directly on the series
df['no_of_employees'] = df['no_of_employees'].apply(lambda x: abs(x) if x < 0 else x)

# Verifying the fix
negative_count_after = (df['no_of_employees'] < 0).sum()
print(f"Number of rows with negative values after fix: {negative_count_after}")

Number of rows with negative values after fix: 0

[19] #shape of negative values after abs value transformation
df.loc[df['no_of_employees'] < 0].shape

(0, 11)

Observations:
• After transforming negative values in no_of_employees' there are no negative values left. Hence, our transformation was successful.

Checking unique values in Categorical columns

[20] def value_counts_with_percent(df, col):
    counts = df[col].value_counts()
    percents = df[col].value_counts(normalize=True) * 100
    result = pd.DataFrame({'Count': counts, 'Percentage (%)': percents.round(2)})
    return result

cat_cols = df.select_dtypes(['object'])

```

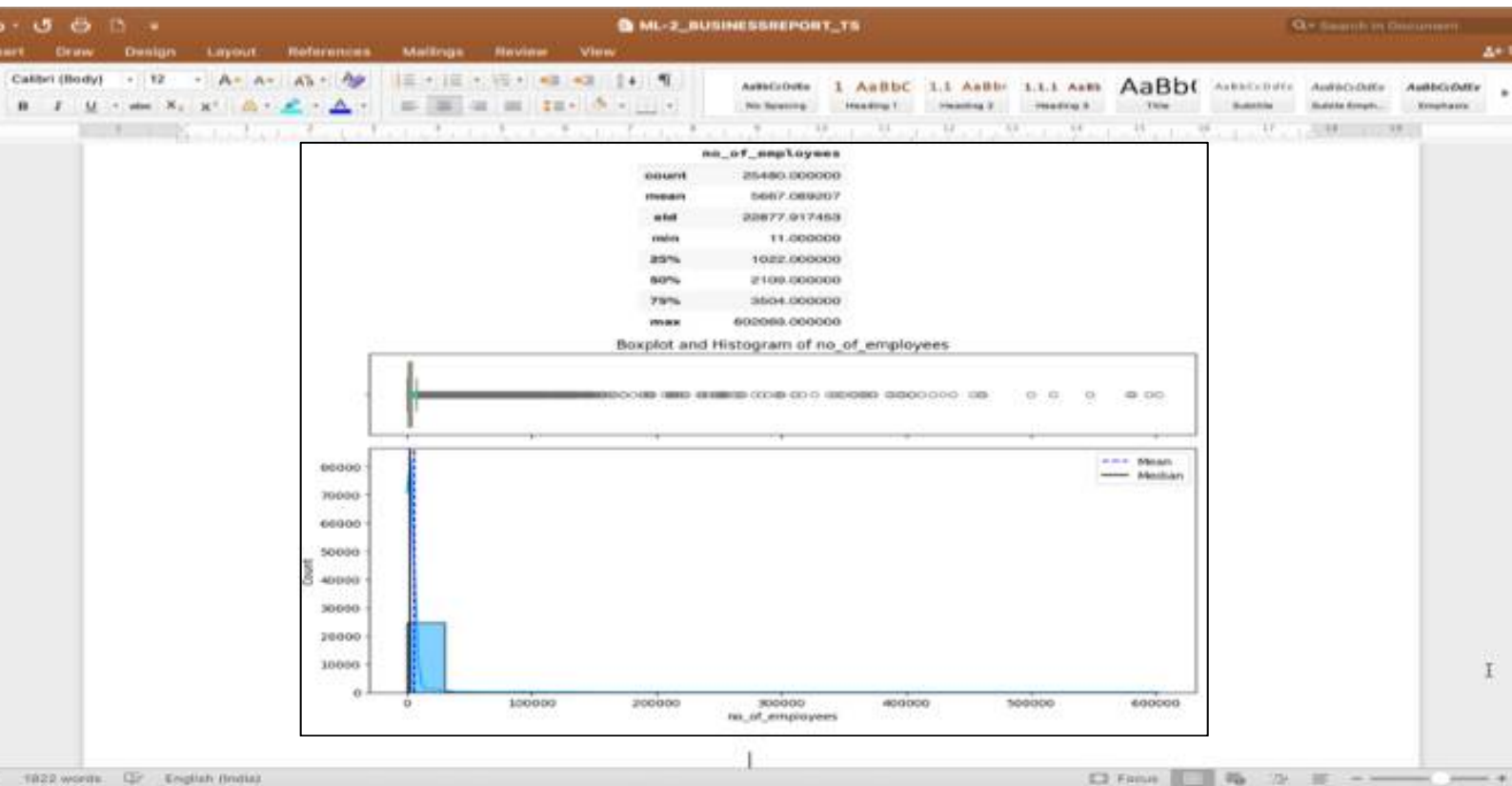
3 EXPLORATORY DATA ANALYSIS

We defined functions to carry out box plot, stacked bar plot, etc.

We will do EDA analysis on both categorical and numerical columns.

3.1 Univariate Analysis

In univariate analysis, we will do analysis on one feature column at a time.



Observations:

- Histogram & bar plot depicts that distribution is **highly right skewed**.
- The vast majority of companies in the dataset has **fewer than 10,000 employees**, as shown by the bars on the left side of histogram plot with **median of 3504 employees**.
- Maximum number of employees is around **60,000**.
- Minimum number of employees is **11**.
- There is presence of multiple extreme **outliers extending up to 60,206** employees as shown by box plot. These points are far beyond the upper whisker of the boxplot.

Observations on yr_of_estab

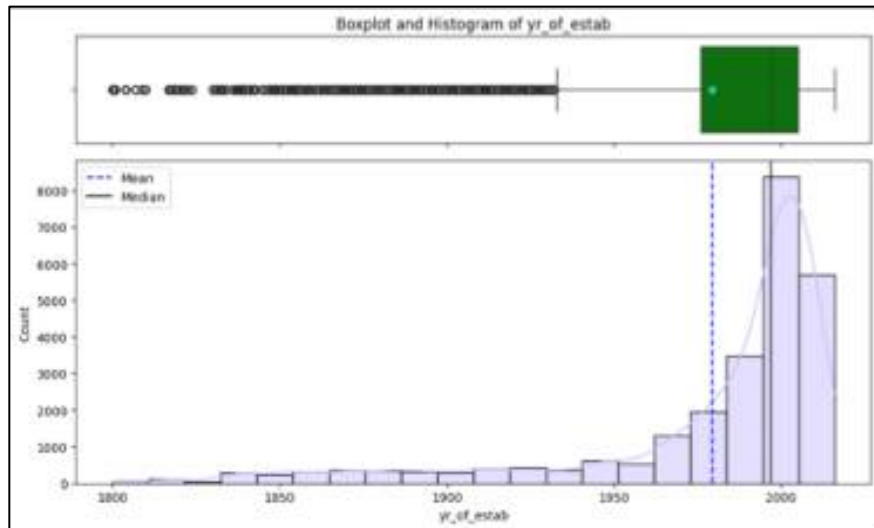


Figure 4. Histogram & box plot of yr_of_estab

Observations:

- Histogram plot reveals distributions is **highly left skewed** with distinct peaks. This suggests that data comes from different years groups.
- The data spans over two centuries, from the **oldest company founded in 1800** to the **newest in 2016**.
- There are **several extreme low value outliers** present beyond upper whisker i.e. companies established in 1800s and early 1900s are acting as historical outlier. They are not error but rare observation and actual data.
- **The mean year (1979)** is significantly lower than the **median year (1997)**, points to the left skewness.
- The **IQR is from 1976 to 2005**, this means 50% of all companies in the dataset were established within this recent 29year window and this recent cluster forms the taller peak on the right side of histogram.

Observations on prevailing_wage

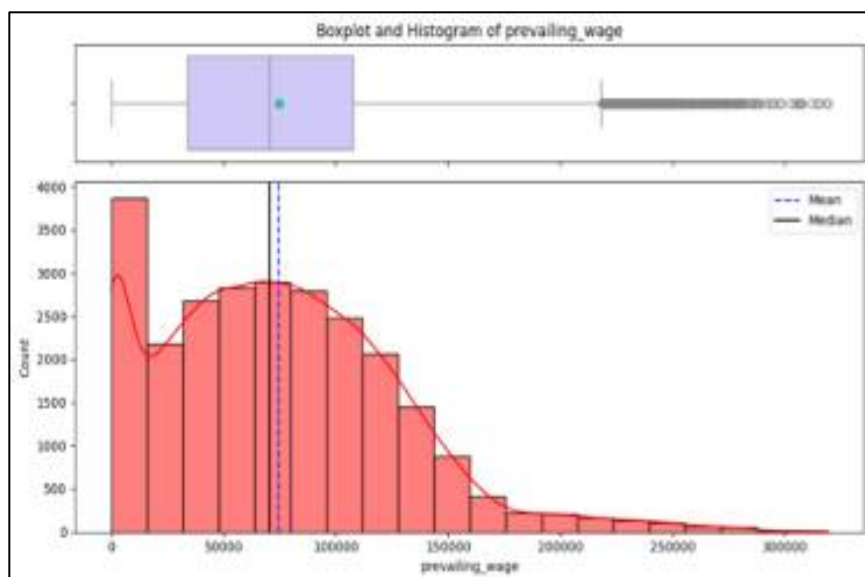


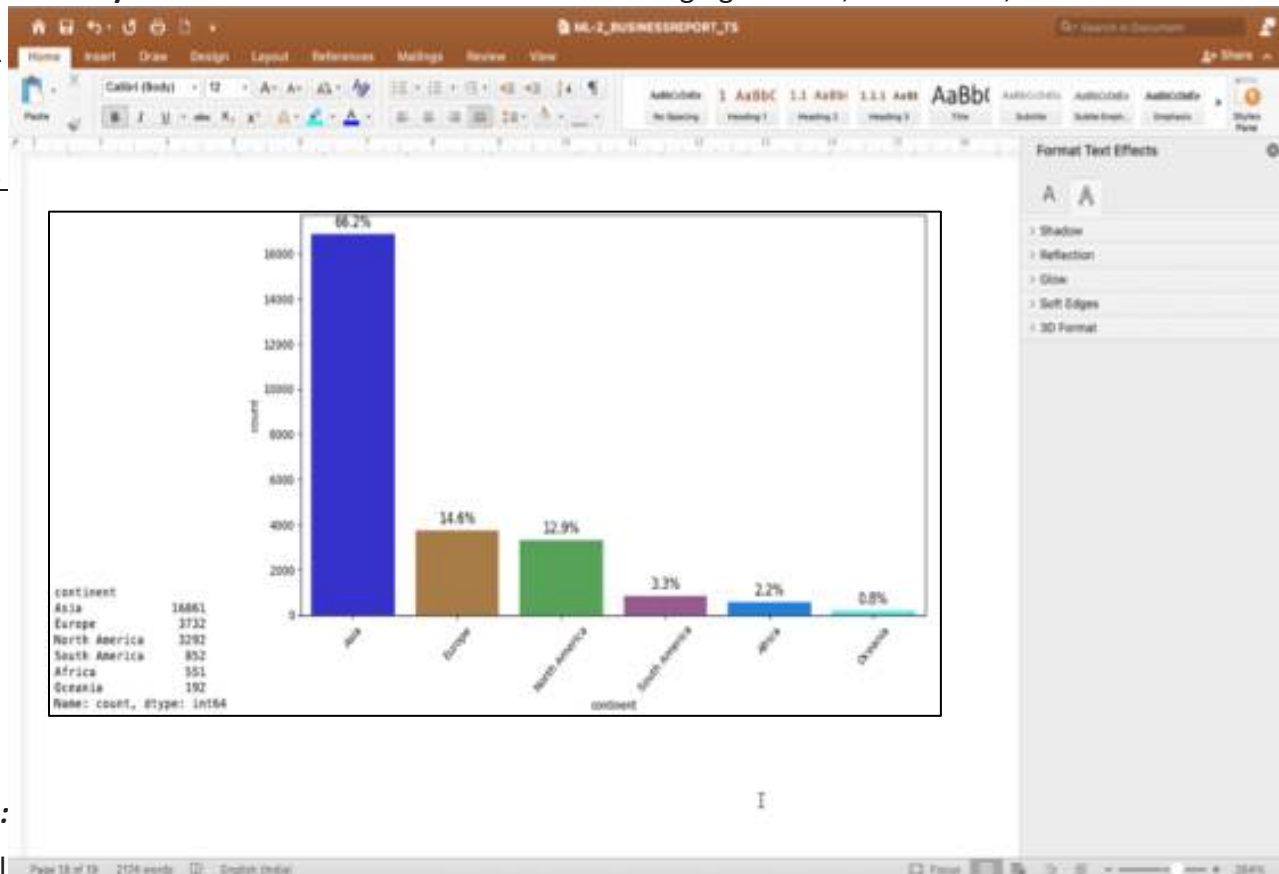
Figure 5. Histogram & box plot of prevailing_wage

Observations:

- The histogram shows a **highly right skewed** distribution. Majority of data is concentrated around on the left side of the plot, reflecting most job offers around standard market rates.
- The median wage of **\$70,308** serves as a solid reference for the average job offer since it is less affected by extreme values than the mean.
- The wide interquartile range **~(\$34k - \$108k)** shows how much variations in standard wages offered.
- The minimum wage offered is **~\$2.13**, while maximum salary offered is **\$31,9210**.
- There are **many extreme outliers** on the lower whisker ranging from 21,000 to 319,210 dollars.

These
senior

Observations

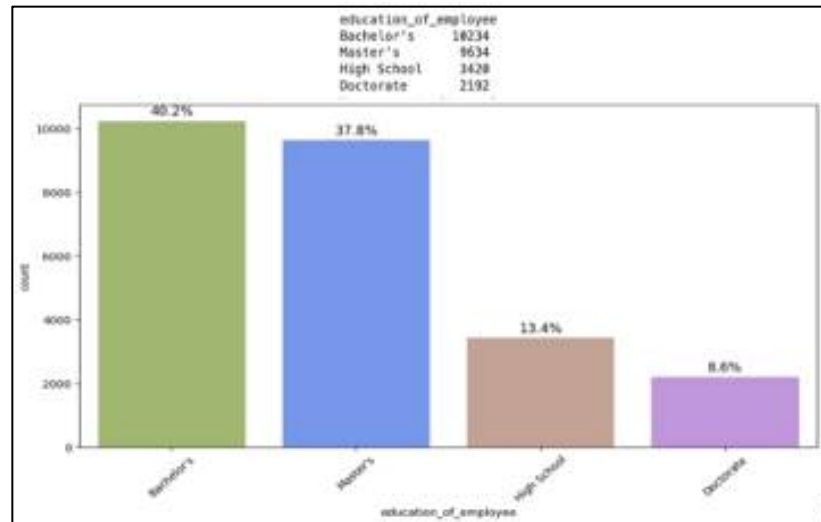


Observations:

- Overall
- **Asia dominates:** Asia has the highest number of observations (**16,861, or 66.2%** of the overall dataset). This suggests a significant over-representation of Asia in the data.
- The **dataset is substantially skewed** between continents as shown in the bar plot, with Asia accounting for more than half of the total observations. This may bias any model trained on this data towards Asia.
- **Europe and North America** have a moderate representation, with **3,732 (14.6%)** and **3,292 (12.9%)**, respectively.
- The geographic regions of **South America, Africa, and Oceania** have little representation in the dataset. Their totals are **852 (3.3%), 551 (2.2%), and 192 (0.8%)**, respectively.
- The low numbers for Africa, Oceania, and South America indicate that insights for **these regions may be less reliable or statistically weaker** than for Asia, Europe, and North America.

Observations on education_of_employee

Observations on education_of_employee



Observations:

Bachelor's Degree: The largest applicant pool

- Individuals with a Bachelor's degree represent the single largest educational demographic, accounting for **10,234 people (40.2%)**.
- This suggests that a four-year university degree is the most common prerequisite for the positions in this dataset.

Master's Degree: Strong Second Majority

- Individuals with a Master's degree are nearly as common, at **9,634 people (37.8%)**.
- The large number of Master's degree holders demonstrates a significant demand for positions requiring specialised, advanced expertise.

High School Diploma: A Distinct Minority

- Only **3,420 people (13.4%)** identify a High School certificate as their highest educational achievement.
- This modest share indicates that the opportunities in this dataset are mostly for professions requiring a higher degree background.

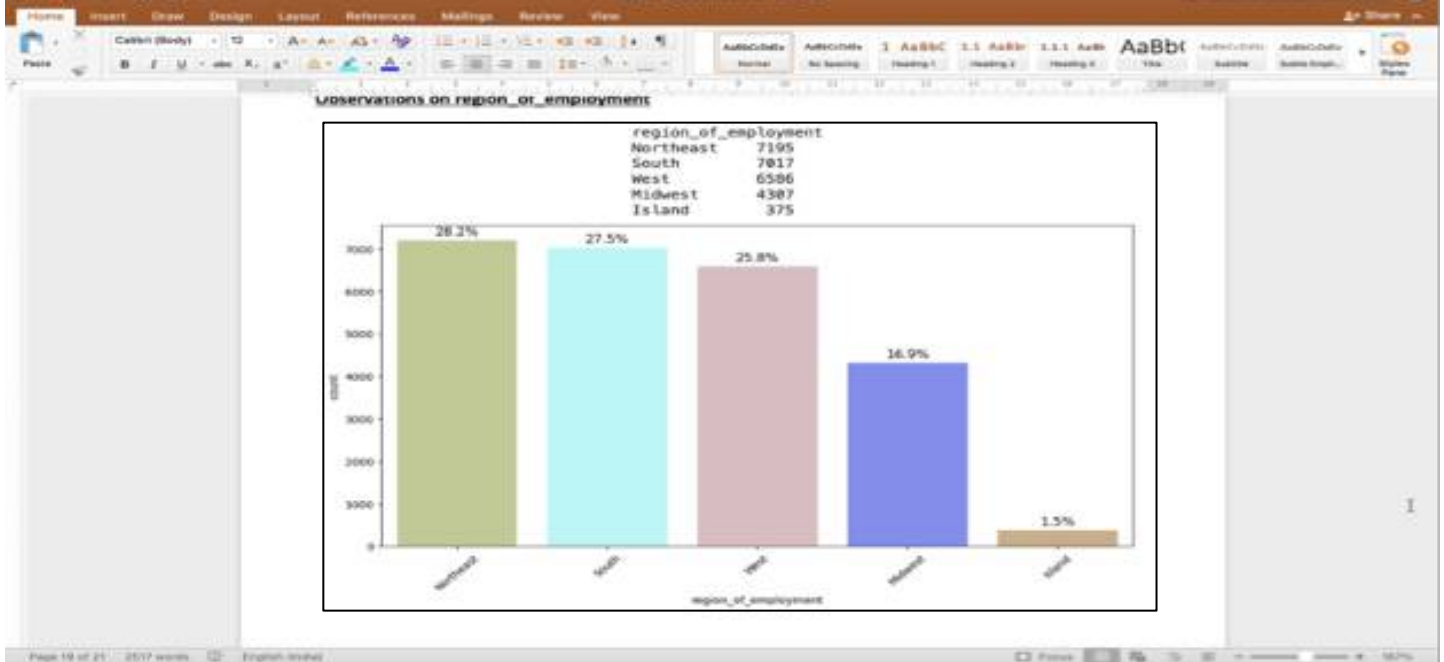
Doctorate Degree: A Highly Specialised Niche

- Individuals with Doctorates are the lowest educational category, with **2,192 people (8.6%)**.
- This category includes a highly specialised group of top-tier experts and senior professionals.

Conclusion:

- **Overall, 86.6% of persons in this dataset have a Bachelor's degree or higher.**
- This distribution is slightly right skewed & demonstrates that the majority of the jobs listed here are for roles that need a highly educated and specialised skilled workers.

Observations on region_of_employment



Observations:

Northeast: The primary hub

- The Northeast area is the leading job hotspot, with **7,195 roles** available, accounting for **28.2%** of all opportunities.
- This suggests a significant number of job opportunities in the region's key economic centres.

South: A Strong Second

- The South area is a close second, with **7,017 spots (27.5%)**.
- This nearly comparable percentage to the Northeast demonstrates that it is an equally important and dominant market for employment.

West is a major contributor

- The West area also contributes significantly, with **6,586 slots (25.9%)**.
- This reinforces the trend of coastal regions being the dominant provider of job opportunities.

Midwest: A Notable Minority

- The Midwest accounts for a lower share, with **** 4,307 positions (16.9%)***.
- While still large, its share is far lower than the top three regions.

Island's Almost Negligible Presence

- There are just **375 jobs (1.5% of total employment)** on island territories.
- This reflects a very specific and constrained job market within the dataset.

Conclusion:

- Overall distribution is skewed towards coastal regions.
- The distribution of work opportunities is extremely uneven.
- It is biased towards the three **coastal regions: northeast, south, and west**.
- These **three regions make up 81.6% of all job openings**, demonstrating a considerable coastal bias in employment location.

Observations

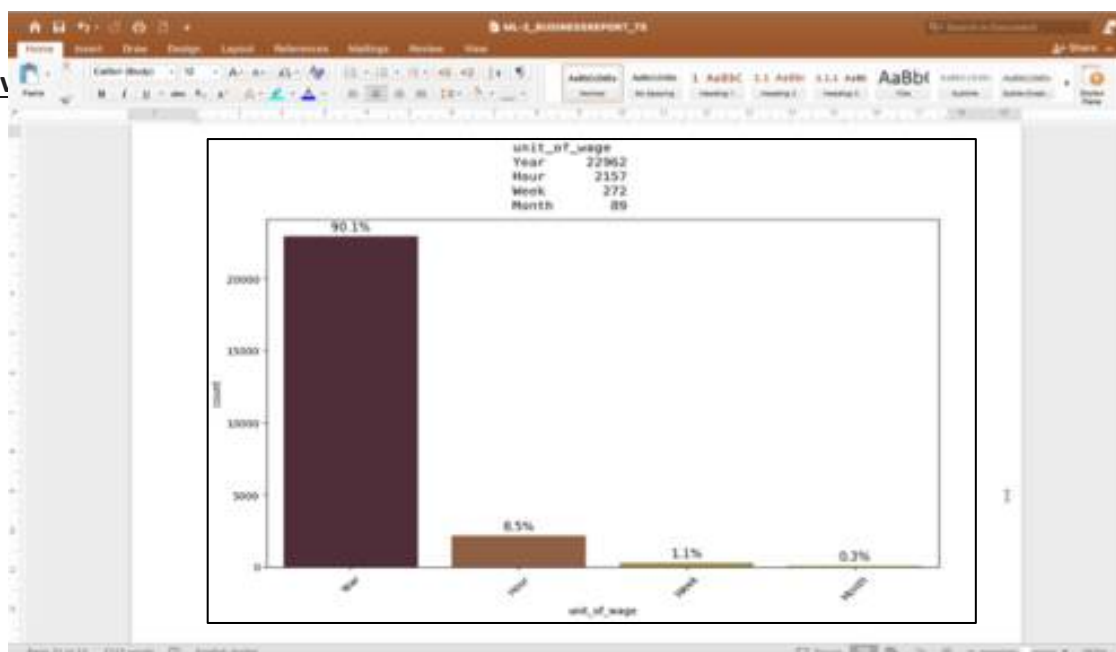


Figure 9. Distribution of unit_of_wage

Observations:

Strong Dominance of Yearly Wages

- The great majority of positions, **22,962 (90.1%)**, pay an annual salary.
- This suggests that the conventional and expected practice for these occupations is to present salary as an annual sum, as is customary for full-time, professional work.

The Low Presence of Hourly Wages

- A tiny but significant percentage of positions, **2,157 (8.5%)**, are paid hourly.
- This implies that some employment, such as specialised contractors, part-time specialists, or certain hourly-based professional services, fall into this category.

Narrow Use of Weekly and Monthly Units

- The Weekly and Monthly payment units are uncommon, with only **272 (1.1%)** and **89 (0.3%)** instances, respectively.
- These units in the dataset indicate employment contracts that are particularly niche or non-standard.

Conclusions:

- The distribution of salary units is **severely right skewed**, with a yearly wage being the nearly universal standard.
- This large bias towards yearly compensation (90.1%) is consistent with the fact that the majority of occupations are full-time, as evidenced by the full_time_position variable.

Obse

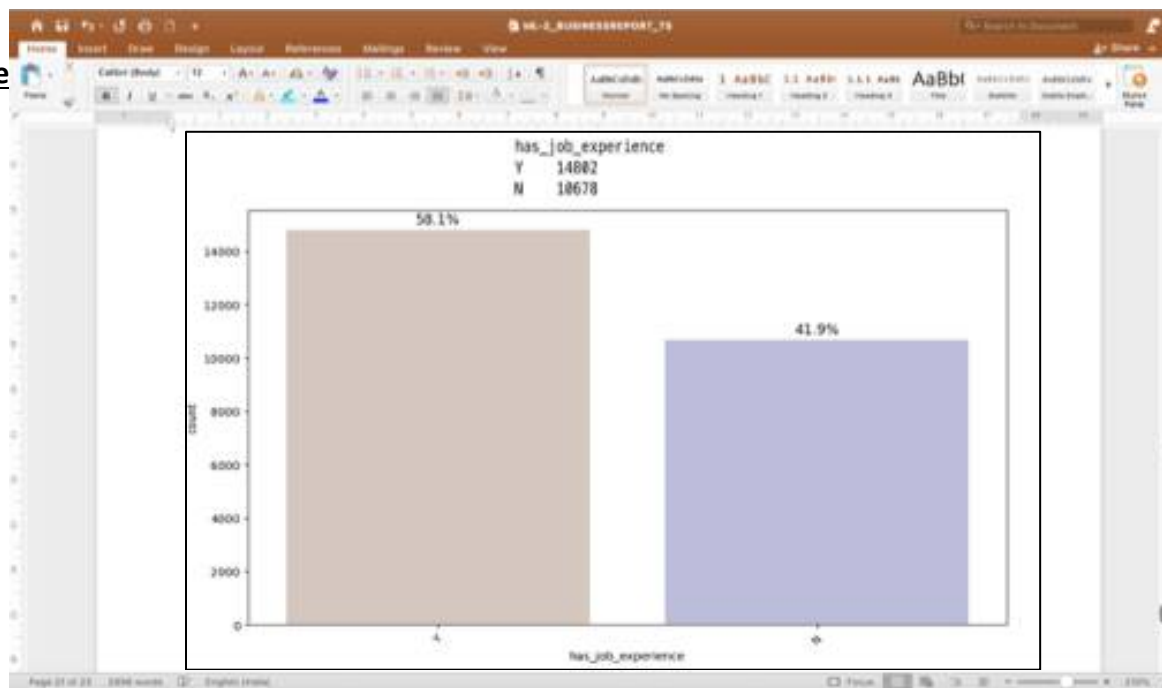


Figure 20: Distribution of has_job_experience

Observations:

Experienced applicants make up the majority

- The vast majority of applicants, **14,802 persons (58.1%)**, have prior employment experience (**Y**).
- This suggests that the majority of the occupations in the dataset require or prefer candidates who have prior professional experience.

A significant number of inexperienced applicants

- A significant proportion of candidates, **10,678 (41.9%)**, have no prior work experience (**N**).
- This significant share means that there are numerous chances for entry-level workers, recent graduates, and those transitioning careers.

Conclusion:

- The dataset includes a fair mix of chances for both **experienced professionals (58.1%)** and **freshers (41.9%)** to the employment market.
- The distribution of experienced and inexperienced applications is fairly even, with a **minor skewness** towards experienced individuals.

Observations on requires_job_training

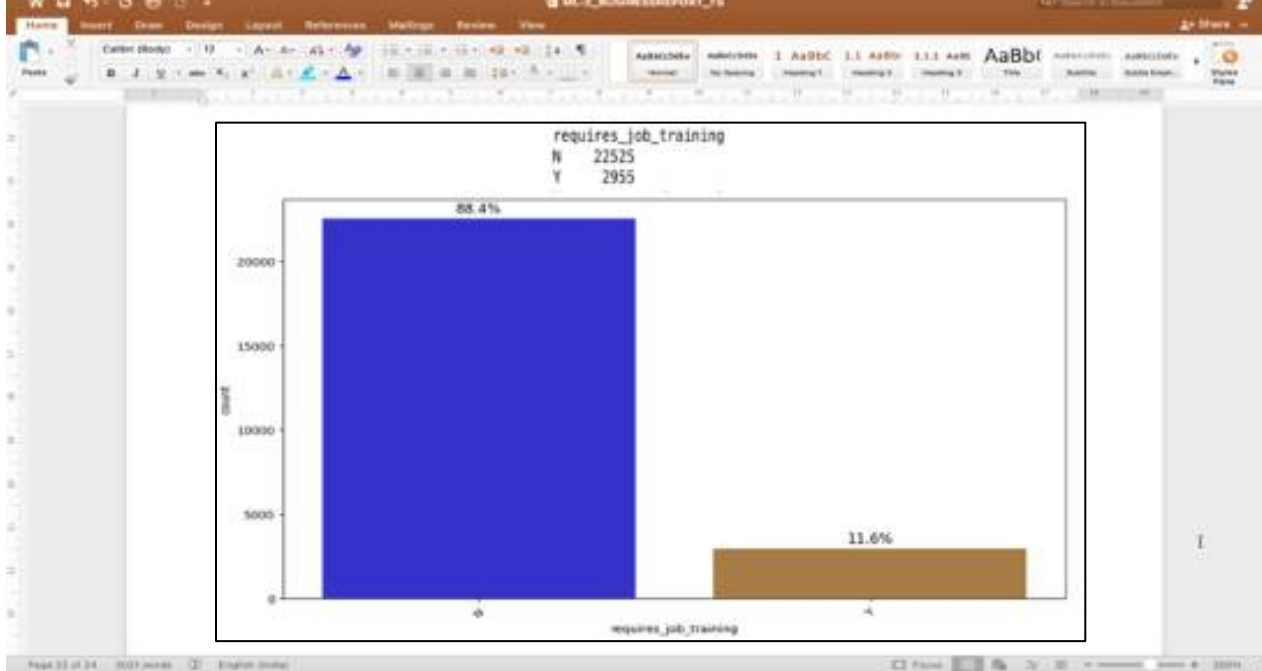


Figure 11. Distribution of requires_job_training

Observations:

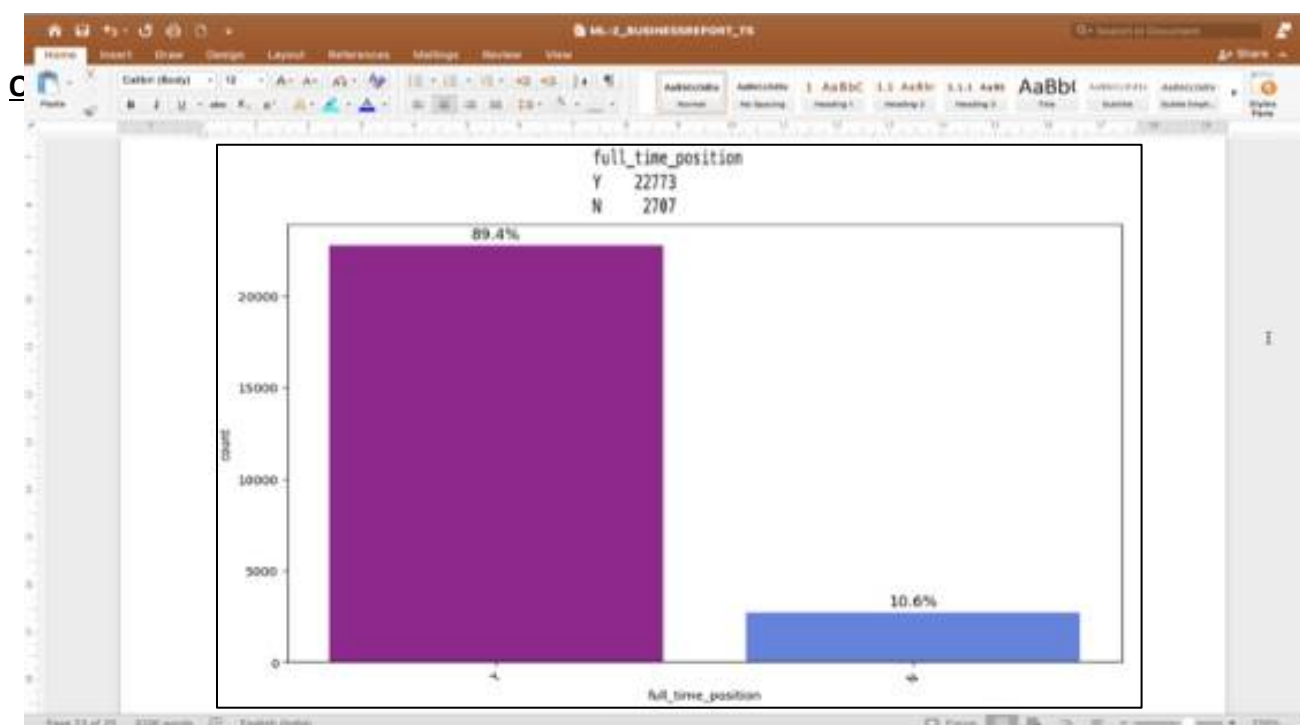
- The distribution is highly right skewed, with a dominant focus on roles that need no training.

An overwhelming majority requires no training (N)

- The great majority of employment, **22,525 (88.4%)**, require no job training for the employee (N).
- This suggests that employers are generally looking for applicants who already have the essential skills and can contribute quickly.

A small number of roles require training (Y)

- Only **2,955 (11.6%)** of positions indicate the need for job training (Y).
- This shows that firms are willing to invest in training for the suitable individual in a certain range of tasks, most likely for highly specialised talents or company-specific procedures.



Observations:

Dominance of full-time positions (Y)

- The vast majority of job jobs, **22,773 (89.4%)**, are full-time (Y).
- This suggests that the opportunities in this dataset are predominantly for secure, permanent employment, as opposed to temporary or part-time labour.

Limited part-time opportunities (N)

- Part-time employment accounts for only **2,707 (10.6%)** of total positions (N).
- This shows that part-time jobs are uncommon and most likely represent specialised, flexible, or niche job arrangements.

Conclusions:

- Overwhelmingly full-time positions The distribution of position kinds is **extremely right skewed**, with full-time opportunities being the nearly universal norm.
- This strong bias towards full-time employment (89.4%) indicates that the dataset represents a job market dominated by full-time, committed employment arrangements. Part-time positions are the exception rather than the rule.

Observations on case_status

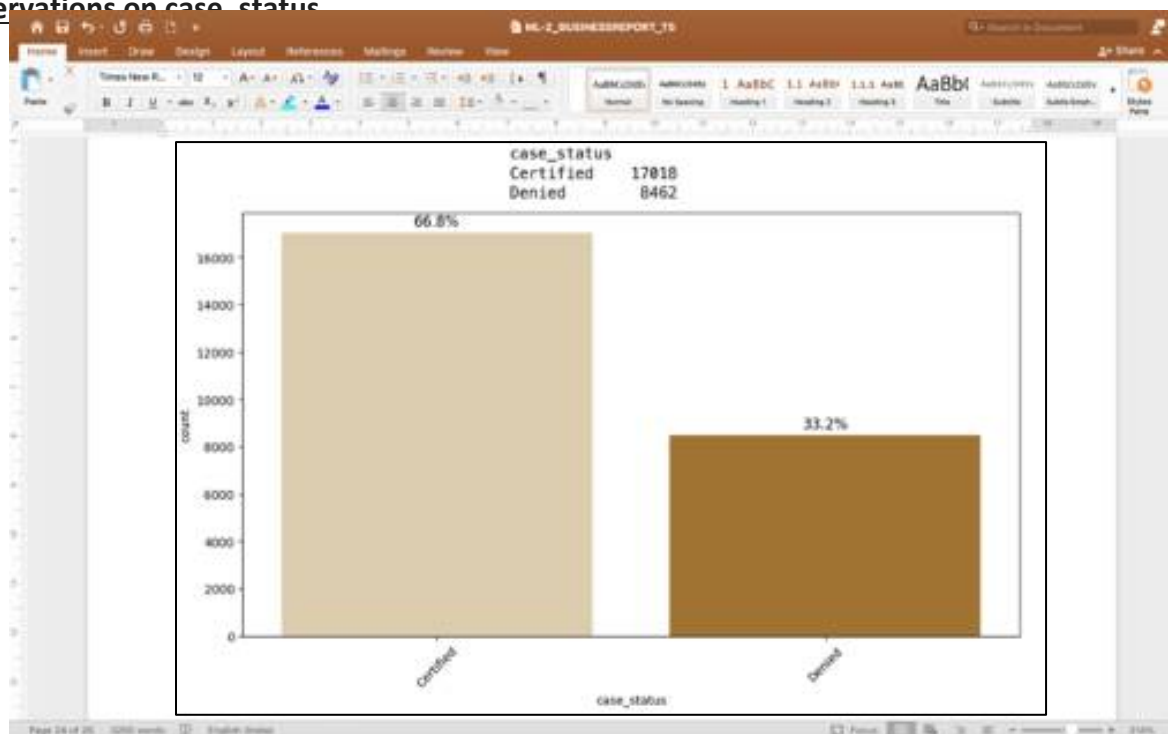


Figure 13. Distribution of case_status

Observations:

The majority of visa applications are certified

- The vast majority of visa applications, **17,018 (66.8%)**, were certified (approved).
- This suggests that the majority of applicants in the sample successfully meet the requirements for visa approval.

A significant number of visa applications are denied

- A significant percentage of applications, **8,462 (33.2%)**, were denied.
- This high denial rate demonstrates how selective the procedure is. with a large number of applications

Conclusions:

- The distribution is two-thirds
- However, it is hard to

3.2 Bivariate

In bivariate analysis, starting with bivariate analysis of 'has_job_experience' with 'Yes' with 1 and

```
ML_2_PROJECT_TS.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

# first we encode our dependent variable
df["case_status"] = df["case_status"].replace({'Certified':1, 'Denied':0})

# now we encode the independent variables with Y: 1 & N :0
df["has_job_experience"] = df["has_job_experience"].replace({'Y':1, 'N':0})
df["requires_job_training"] = df["requires_job_training"].replace({'Y':1, 'N':0})
df["full_time_position"] = df["full_time_position"].replace({'Y':1, 'N':0})

# List the columns we just label encoded
encoded_columns = ["case_status", "has_job_experience", "requires_job_training", "full_time_position"]

# verify the unique values for each of these columns
for col in encoded_columns:
    unique_vals = df[col].unique()
    print(f"Unique values in '{col}': {sorted(unique_vals)}")

Unique values in 'case_status': [np.int64(0), np.int64(1)]
Unique values in 'has_job_experience': [np.int64(0), np.int64(1)]
Unique values in 'requires_job_training': [np.int64(0), np.int64(1)]
Unique values in 'full_time_position': [np.int64(0), np.int64(1)]
```

Observations:

- We have successfully label encoded 'case_status', 'has_job_experience', 'requires_job_training', 'full_time_position'

Heatmap Analysis

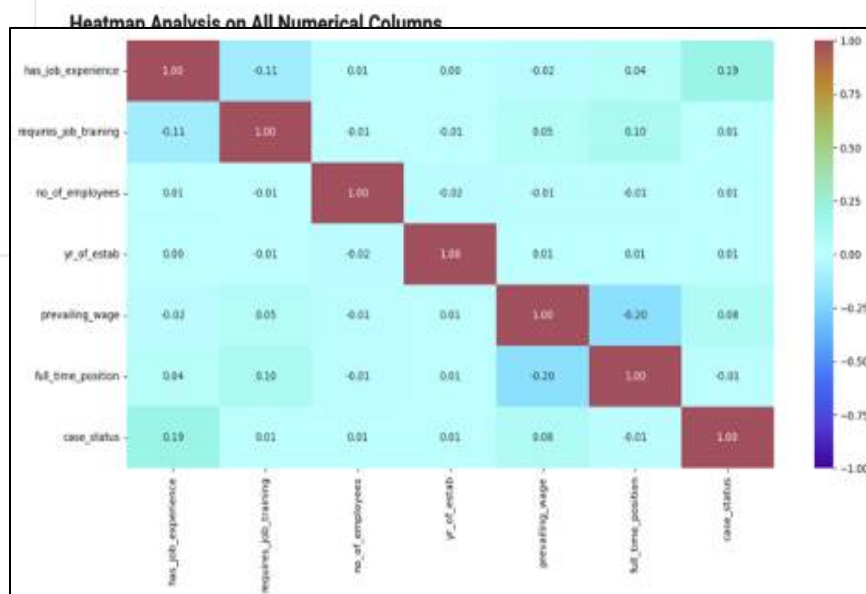


Figure 15. Heatmap Analysis

Observations:

Job experience is the strongest predictor of visa approval

- The highest positive correlation ($r = 0.19$) exists between visa certification (case_status) and employment experience (has_job_experience).
- This positive link suggests that applicants with prior job experience are more likely to get their visas approved.

Full-time roles are linked with lower wages

- A significant negative correlation ($r = -0.20$) appears between wage (prevailing_wage) and position type (full_time_position).
- This shows that full-time occupations in this dataset pay lower earnings than part-time or contract work, which is counterintuitive and warrants additional examination.

Experienced applicants require less training

- There is a logical negative association ($r = -0.11$) between experience (has_job_experience) and training requirements.
- This demonstrates that applicants with prior experience are less likely to require further job training.

There are no redundant variables detected

- All correlations between the predictor variables are quite weak (**less than $|r| = 0.20$**).
- This means that there is no multicollinearity - each variable contributes unique information and can be combined in a predictive model.

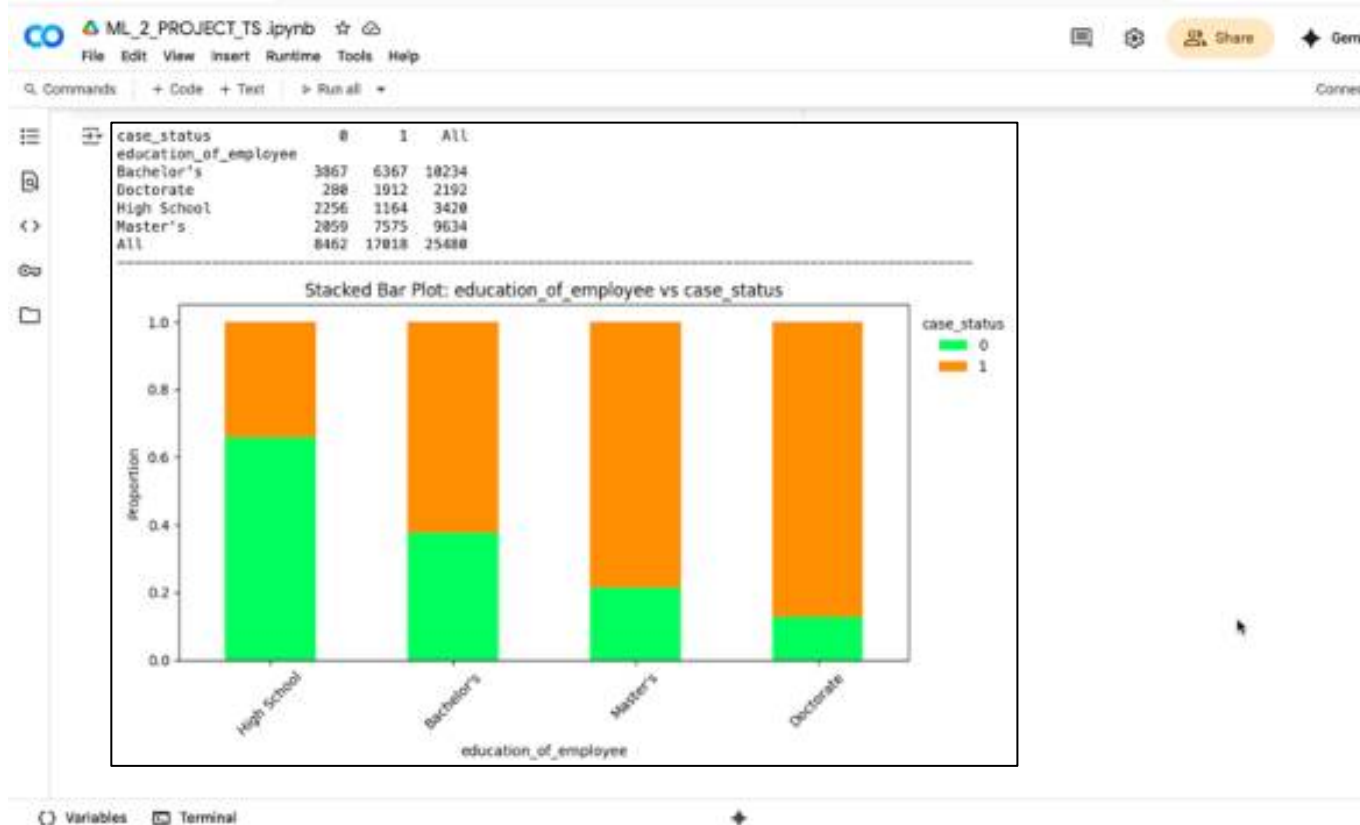
The Company Profile Has Little Direct Influence

- Both firm size (no_of_employees) and company age (yr_of_estab) have a near-zero correlation (**roughly $r = 0.01$**) with visa clearance.
- This suggests that the size or age of an organisation has almost no direct linear association with the likelihood of visa approval.

Most relationships are weak

- The vast majority of correlations are close to zero, indicating that the dataset lacks straightforward linear relationships.
- This implies that more advanced modelling techniques may be required to identify complicated trends driving visa choices as simple model will not perform well.

Observations on education_of_employee vs case_status



Observations:

Doctorate holders have the highest certification rate

- Applicants with a doctorate degree have a very high certification rate. **1,912 out of 2,192** applications were approved, for an impressive **87.2% approval rate**.
- This suggests that sophisticated, specialised knowledge is in high demand and significantly favoured during the visa acceptance procedure.

Very High Certification Rate: Master's Degree Holders

- Applicants holding a Master's degree have an excellent likelihood of approval. **7,575 of 9,634** applications were certified, yielding an acceptance percentage of **78.6%**.
- This confirms the high preference for applicants with more advanced education than a bachelor's degree.

Moderate Certification Rate for Bachelor's Degree Holders

- Applicants holding a Bachelor's degree have a moderate certification rate. **6,367 of 10,234** applications were approved, representing a **62.2%** visa acceptance rate.
- While, a bachelor's degree alone has a much lower chance of clearance than advanced degrees.

Lowest Certification Rate: High School Graduates

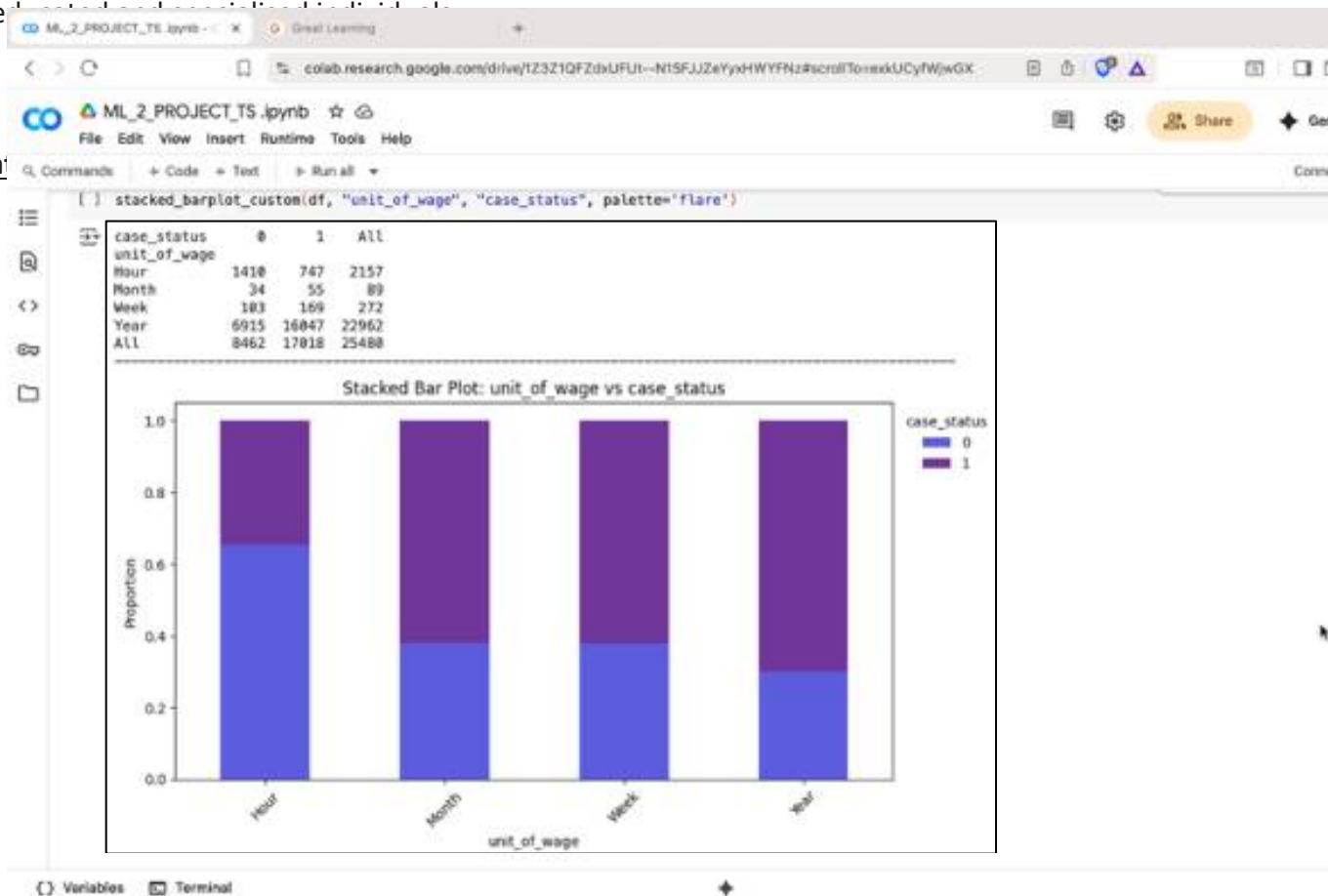
- Applicants with only a high school diploma face the most difficult odds. **Only 1,164 of 3,420** applications were approved, for a poor acceptance rate of **34.0%**.
- This shows that visas are largely certified for occupations requiring a higher education background, while applications based only on experience or abilities (without a degree) are rejected at a far higher rate.

Conclusions:

- There is a clear and substantial positive correlation between an applicant's educational level and their chances of visa approval.

- The certification rate rises considerably with each higher level of education. Higher education levels range from high school (34%), bachelor's (62%), master's (79%), to doctoral (87%).
- This tendency strongly suggests that the visa procedure is intended to prioritise and certify highly educated and specialised individuals.

Observations:



Observations:

Highest Certification Rate: Yearly-Paid Roles

- Positions with annual salaries have the greatest certification rate. **16,047 out of 22,962** applications were approved, yielding a solid **69.9%** visa acceptance rate.
- This suggests that secure, full-time, salaried jobs are evaluated favourably and have the best chance of visa acceptance.

High Certification Rate for Weekly and Monthly Paid Roles

- Roles with **weekly (62.1%)** and **monthly (61.8%)** pay have much higher approval percentages.
- This shows that reviewers believe certain non-hourly contract forms are legitimate and stable.

Lowest Certification Rate: Hourly-Paid Positions

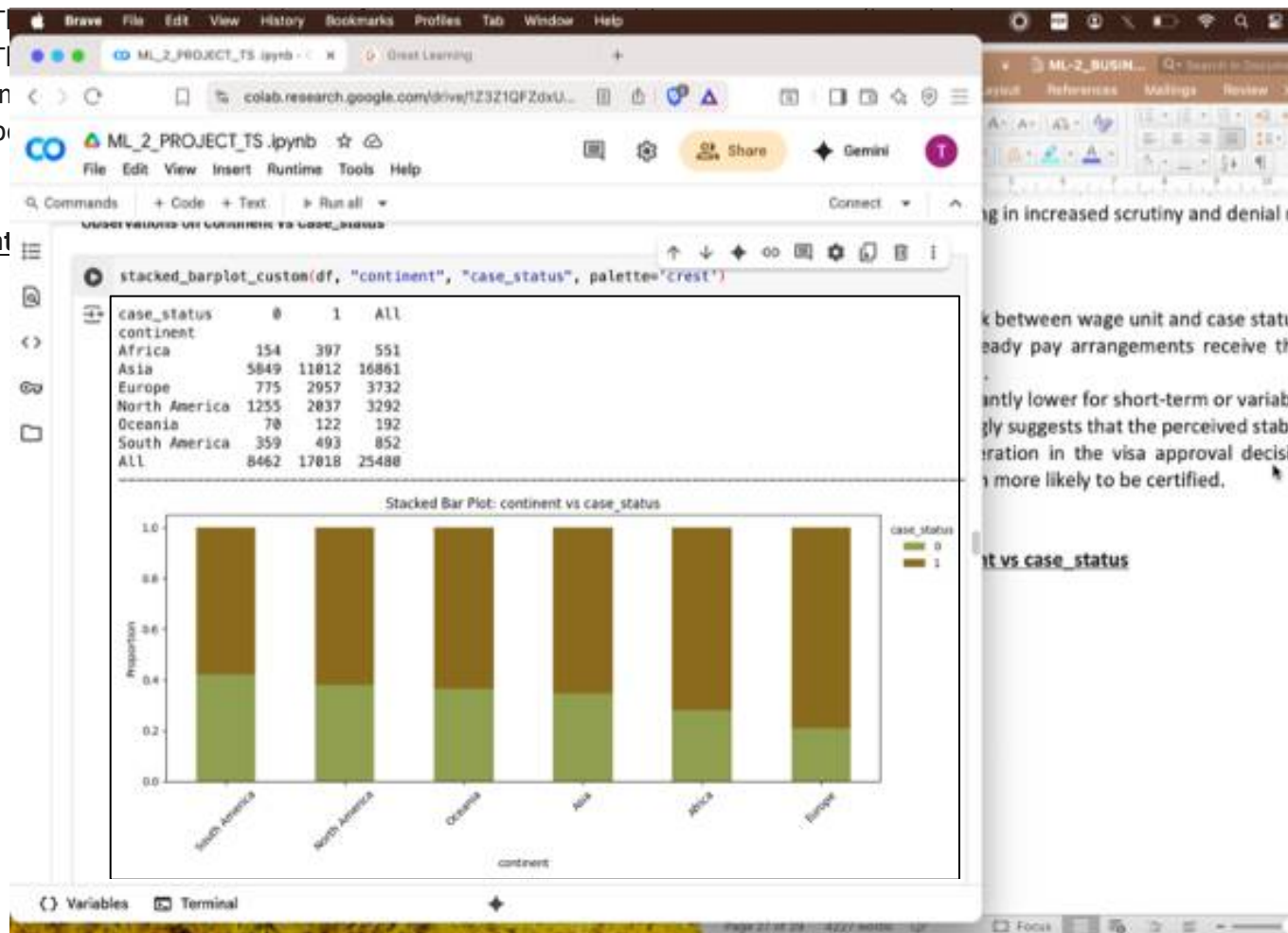
- Applicants earning an hourly pay have the most difficult odds. Only **747 of the 2,157** applications were certified, resulting in a **34.6%** visa approval rate.
- This shows that hourly-paid occupations are associated with less permanent, transient, or part-time work, resulting in increased scrutiny and denial rates.

Insights:

- There is a clear link between wage unit and case status likelihood.

- Long-term and steady pay arrangements receive the highest visa acceptance rate (yearly > weekly ≈ monthly).
 - T
 - T
- in
p

Observations



Observations:

European applicants have the highest certification rate

- Approval Rate: **79.2% (2,957 approved from 3,732 total)**
- European applicants have the greatest visa certification rates.
- A significant **majority of applications from Europe are approved.**

Very High Certification Rate for Oceanian Applicants

- Approval Rate: **63.5% (122 approved of 192 total).**
- **Second highest approval rate**, albeit with a lower sample size.
- Strong performance despite reduced candidate numbers.

Moderate Certification Rate for African and South American Applicants

- **Africa: 72.1%** approval rate (397 out of 551).
- **South America: 57.9%** approval percentage (493 out of 852).
- **Solid approval rates, particularly for African applicants.**

Lower Certification Rate for North American and Asian Applicants

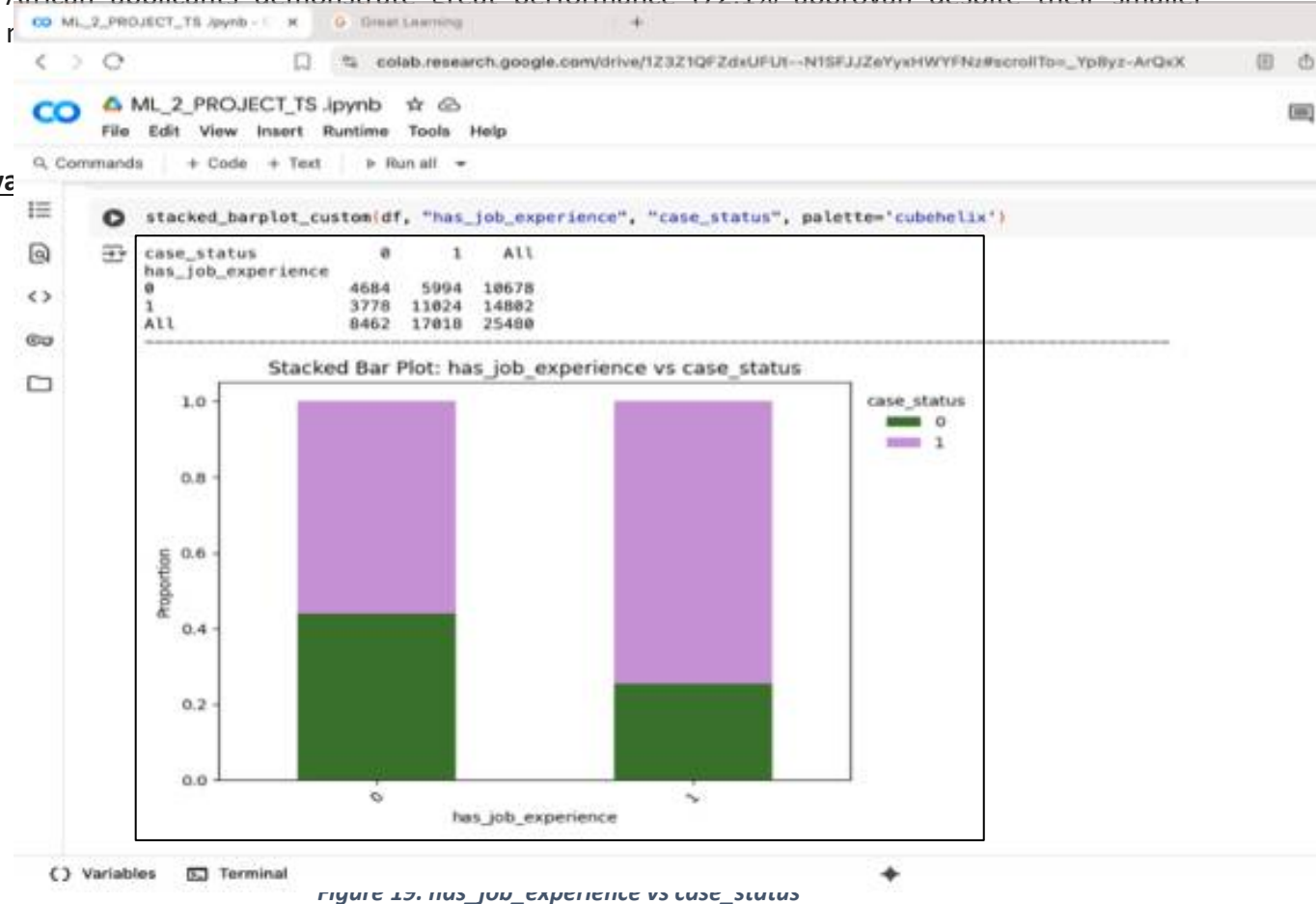
- **North America: 61.9%*** approval rate (2,037 approvals out of 3,292).
- **Asia: 65.3%** approval rate (11,012 accepted from 16,861)

- While still a majority of approvals, these rates are **lower than other locations**.
- **Asia is the greatest candidate pool (66.2% of total applications)**.

Insights:

- There is a 24.3 percentage point difference between the highest (Europe: 79.2%) and lowest (South America: 57.9%) approval percentages.
- European applicants have much higher success percentages than other regions.
- Asian candidates dominate in volume (66.2% of all applications), with a moderate approval rate (65.3%).
- African applicants demonstrate great performance (72.1% approval) despite their smaller

Observations:



Observations:

Higher Certification Rate: Experienced Applicants

- Approval Rate: **74.5%** (11,024 accepted from 14,802 total)
- Applicants with job experience have much better certification rates.
- Nearly three out of every four experienced applicants are approved.

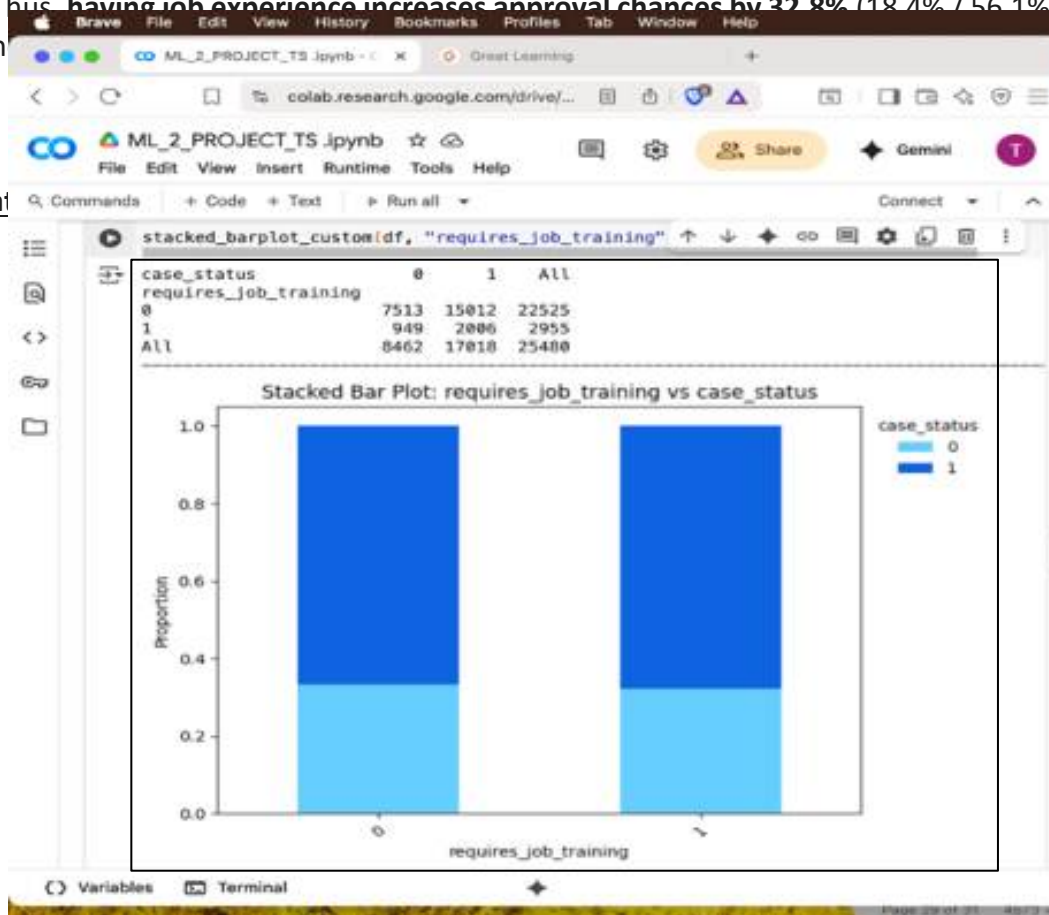
Lower Certification Rate for Inexperienced Applicants

- Approval Rate: **56.1%** (5,994 authorised from 10,678 total)
- Applicants without work experience have more challenging approval odds.
- A little more than half of novice applicants get approved.

Insights:

- 18.4% point difference between experienced (74.5%) and freshers (56.1%) applicants.
- Experience is an important factor in the approval process.
- Experienced applicants are 1.33 times more likely to be approved than novice applicants.
- The majority of candidates (58.1%) have job experience.
- Thus, having job experience increases approval chances by 32.8% ($18.4\% / 56.1\%$) compared to no experience.

Observations:



Observations:

Higher certification rate: No training required

- Approval rate: **66.7%** (15,012 accepted out of a total of 22,525).
- Positions requiring no job training have a substantially higher certification rate.
- Nearly two out of three applications are granted when no training is required.

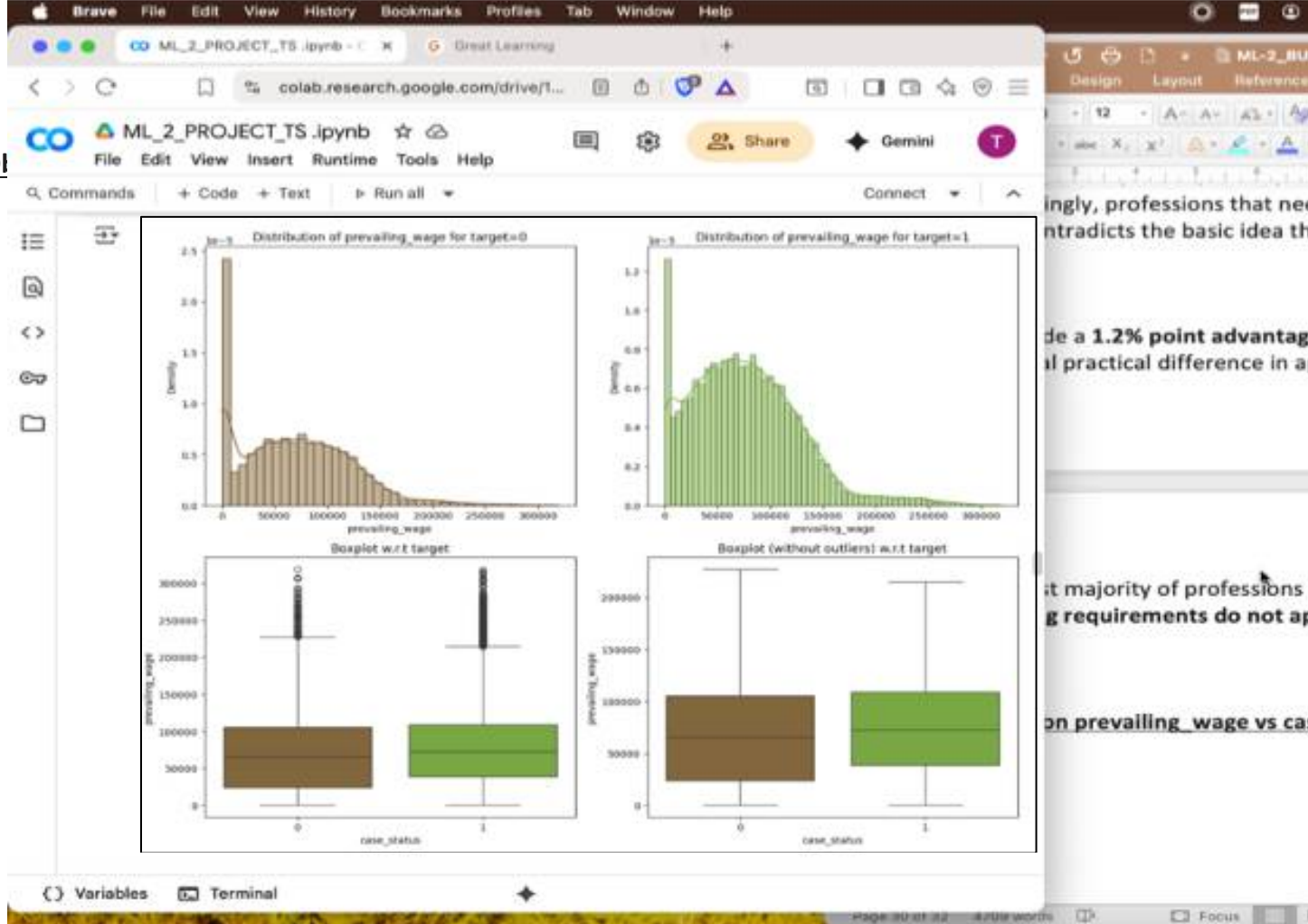
Slightly lower certification rate: Training is required

- Approval Rate: **67.9%** (2,006 approved from 2,955 total)
- Surprisingly, professions that need training have a marginally higher approval rate.
- This contradicts the basic idea that training requirements will negatively effect approval.

Insights:

- It include a **1.2% point advantage** for professions needing training (67.9% vs. 66.7%).
- Minimal practical difference in approval rates based on training requirements.
- The vast majority of professions (88.4%) require no training.
- **Training requirements do not appear to be a key issue in approval decisions.**

O!



Observations:

- The prevailing wage has highly right skewed distribution, and is quite comparable between certified and rejected visas; the main distinction is in the lowest earnings, where denied visas have a density of **around 2.5** and certified visas have a density of **around 1.30**.
- Also, certified visas have a somewhat higher density than denied visas.
- The distribution on the boxplot is quite similar, with the exception of a somewhat smaller number of denied visas.
- Offering a greater salary may marginally boost the likelihood of certification, even if it is not a reliable indicator in and of itself.

Observat

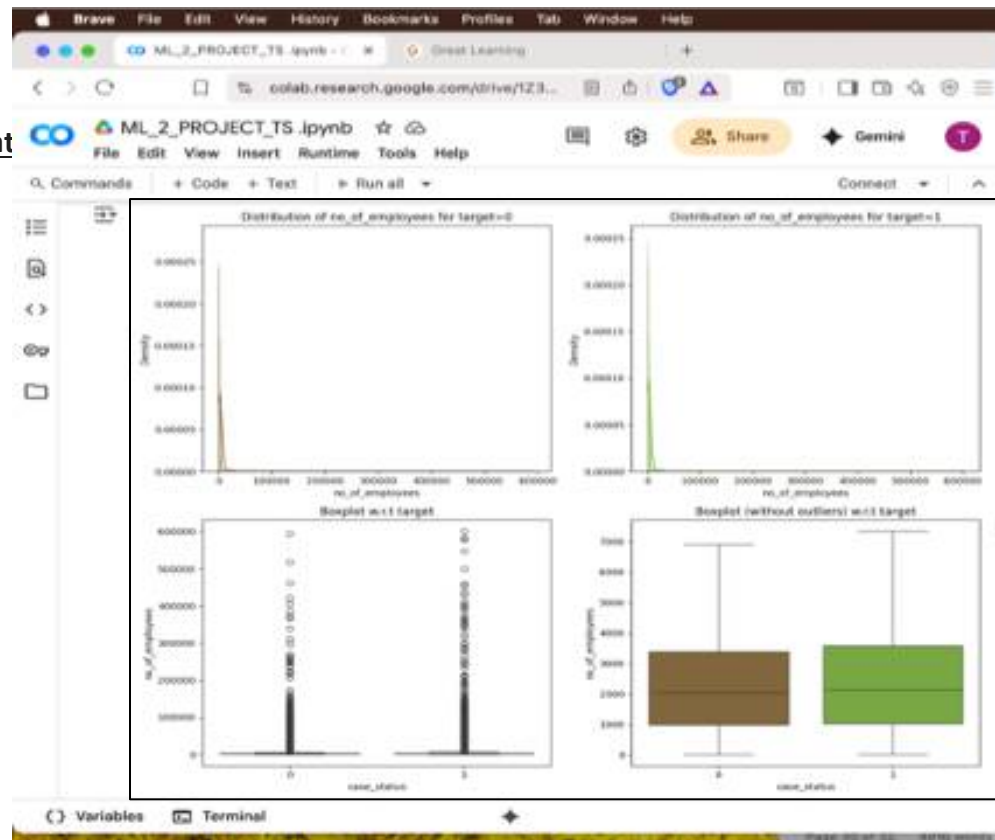


Figure 22. no_of_employees vs case_status

Observations:

- Density plot shows almost similar trend for denied & accepted visas status based on number of employees, with distribution is highly right skewed.
- The number of employees with denied visa has density **~0.00023** while, the density for accepted is **~0.00024**.
- The box plot shows no_of_employees with denied status has median around 2k while accepted status has slightly above 2k.
- Outliers are present in both case status.
- The average company size is also slightly higher for certified visas.
- This is because visas from bigger, more reputable organisations have a minor propensity to be certified.
- This probably reflects the idea that larger companies, which might have more financial stability and familiarity with the immigration procedure, are more legitimate and lower risk.
- Thus, company size (no_of_employees) is not a decisive feature, but rather a minor supporting trait.

Observati

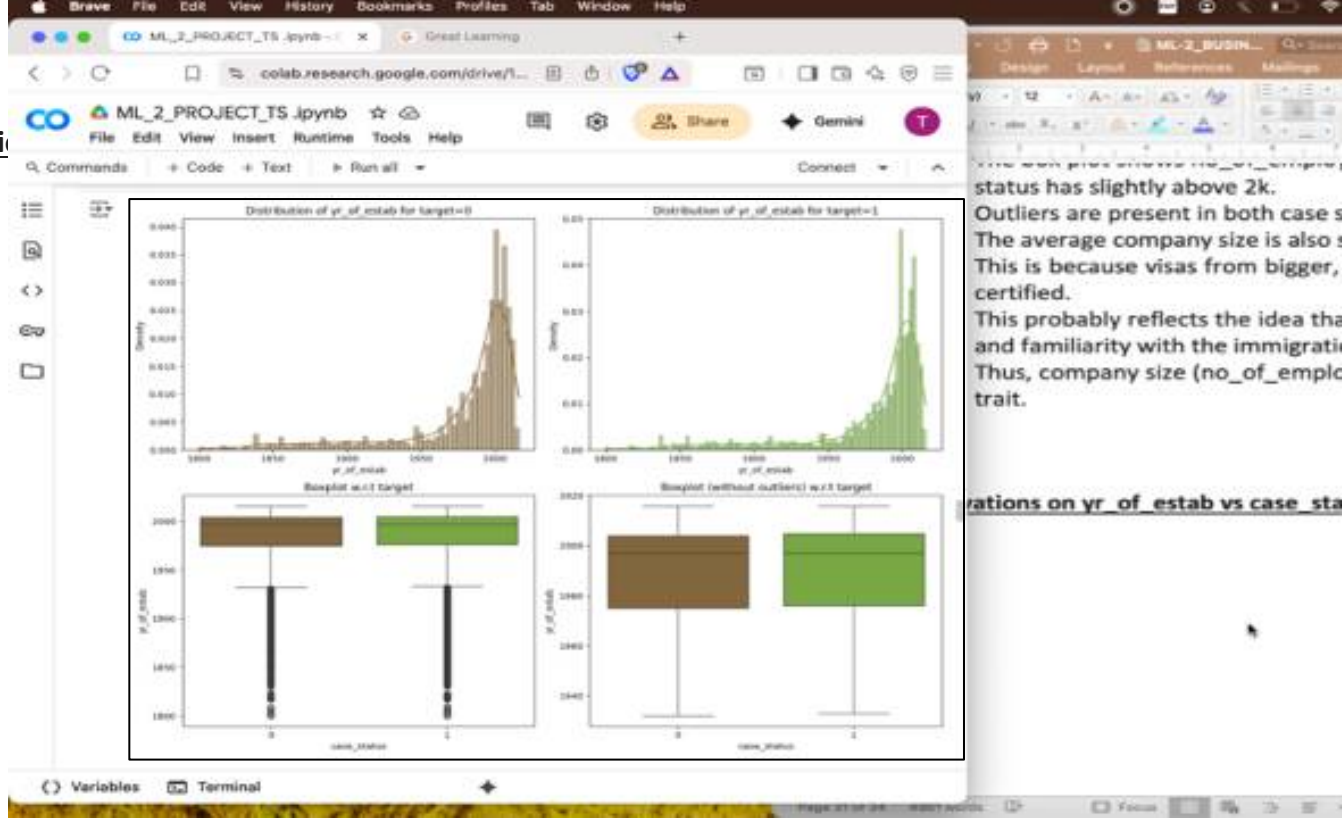
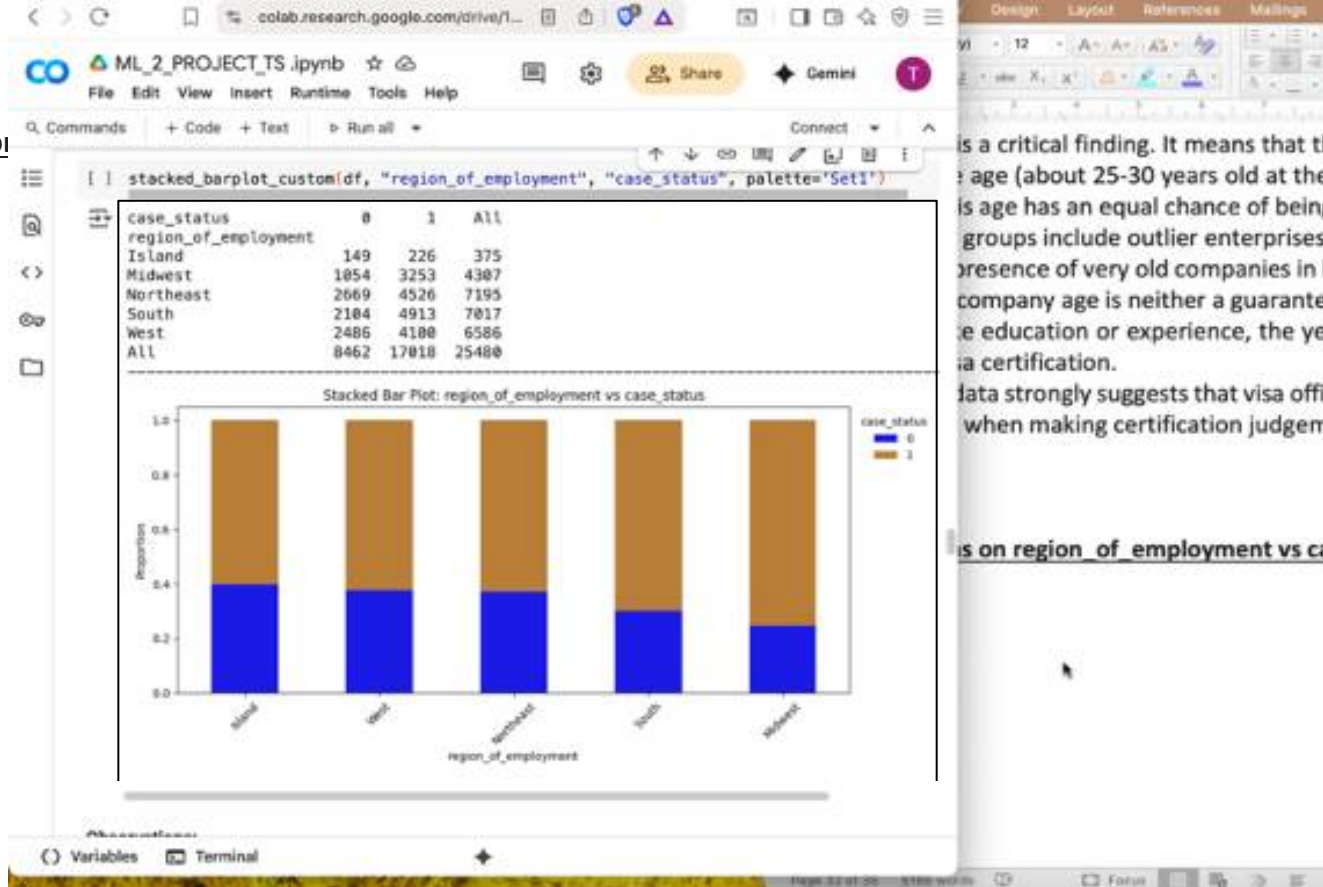


Figure 23: yr_of_estab vs $case_status$

Observations:

- Density plot shows highly left skewed distribution with similar trends in denied or accepted via status.
- **Density for denied case is ~0.26 while for accepted is ~0.025.**
- The box plot shows there are several outliers present on both cases of denied & accepted visa status.
- **Median year of establishment for a company is ~1995 in both the case of denied & rejected visa status.**
- This is a critical finding. It means that the "typical" or median company in both categories is the same age (about 25-30 years old at the time of application). A visa application from a company of this age has an equal chance of being approved or denied based only on this aspect.
- Both groups include outlier enterprises founded considerably earlier (e.g., in the 1800s).
- The presence of very old companies in both the accepted and refused visa groups demonstrates that company age is neither a guarantee of success nor a reason for denial.
- Unlike education or experience, the year of establishment (business age) is a neutral predictor of visa certification.
- The data strongly suggests that visa officers do not prefer older, "legacy" enterprises over newer ones when making certification judgements.

Observation



Observations:

Midwest-Highest Approval Rate

- The Midwest region has the highest certification rate.
- Approval Rate: **75.5%** (3,253 approved from 4,307 total)
- Three out of every four applications in the Midwest are approved.

High Certification Rate: South Region

- Approval Rate: **70.0%** (4,913 approvals out of 7,017 total)
- The South area demonstrates great performance, with a high approval rate.
- Applications here have very high prospects of success.

Moderate Certification Rate: West and Northeast Regions

- **West: 62.3%** approval rate (4,100 out of 6,586).
- **Northeast: 62.9%** approval rate (4,526 approvals out of 7,195).
- These big coastal regions exhibit moderate approval percentages.
- The majority approval rate remains high, but it is lower than in other regions.

Lowest Certification Rate: Island Territories

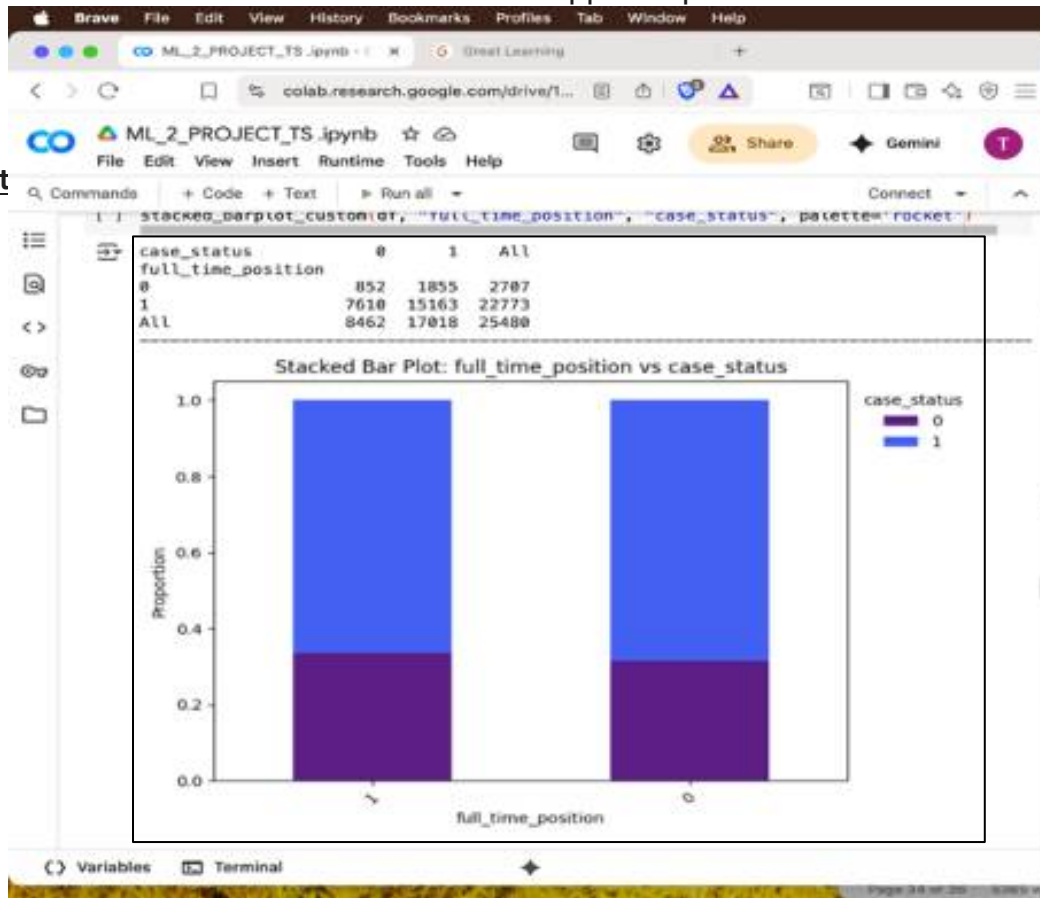
- Approval Rate: **60.3%** (226 approved of 375 total).
- The island territories have the lowest certification rate.
- This region faces the most demanding approval process.

Key insights:

- There is a **15.2 percentage point difference** between the highest (Midwest: 75.5%) and lowest (Island: 60.3%) approval percentages.
- Midwest applicants have much greater success percentages than other regions.

- Regional patterns matter: Heartland regions (Midwest, South) outperform coastal regions (Northeast, West).
- Island territories have the most difficult approval process.

Observations



Observations:

Higher Certification Rate: Full-Time Role

- Approval Rate: **66.6%** (15,163 approved of 22,773 total)
- Full-time positions have a strong majority approval rate.
- Nearly two out of every three full-time applications are granted.

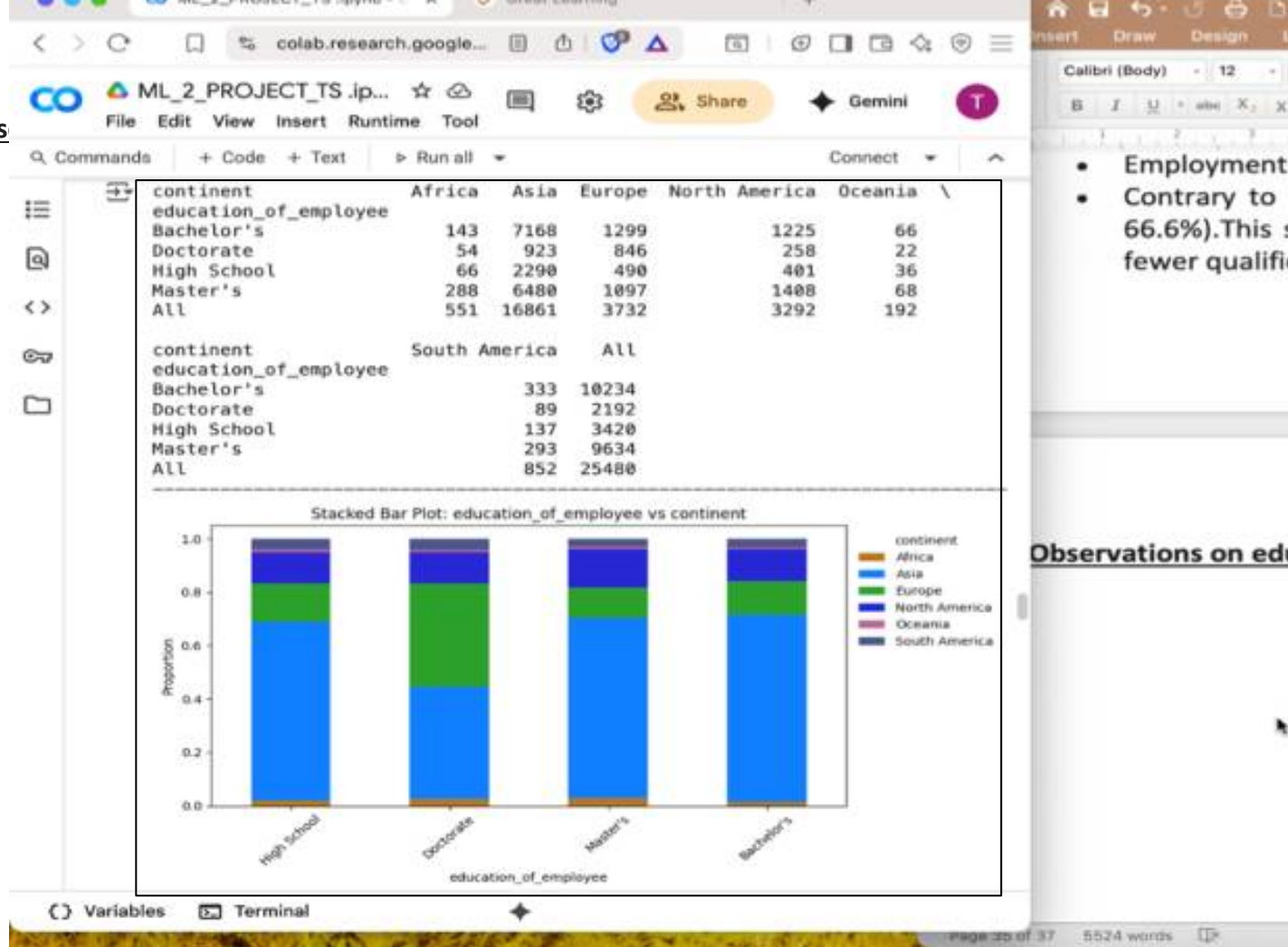
Part-time positions have the highest certification rates

- Approval Rate: **68.5%** (1,855 approved from 2,707 total)
- Surprisingly, part-time positions had a slightly higher approval rate.
- This goes against the initial notion that full-time positions would be preferred.

Insights:

- Part-time roles have a **1.9 percentage point advantage**(68.5% vs. 66.6%).
- There is a minimal practical difference in approval rates based on the employment type.
- The overwhelming majority of applicants (89.4%) are for full-time work.
- Employment type (full vs. part-time) does not appear to be a key influence in approval decisions.
- Contrary to popular belief, part-time jobs had a somewhat higher approval rate (68.5% vs. 66.6%). This suggests that part-time roles could be for specialized, high-demand skills where fewer qualified applicants exist.

Obs



Observations:

Asia dominates at all educational levels

- Asian applicants are the majority in all education categories:
 - Bachelor's: 7,168 out of 10,234 (**70%**).
 - Master's: 6,480 out 9,634 (**67%**)
 - Doctorate: 923 of 2,192 (**42%**).
 - High school: 2,290 out of 3,420 (**67%**)
- This reflects Asia's significant presence in the entire candidate pool.

Europe: Strong on Advanced Degrees

- Europe has unusually strong representation in higher education.
- Doctorate: 846 out of 2,192 (**39%**)—second only to Asia.
- Master's: **1,097** applications
- European candidates are more concentrated in advanced degree categories than other areas.

North America has a balanced educational distribution

- North American candidates are very evenly divided across education levels.
- Bachelor's: **1,225**.
- Master's: **1,408**

- High School: **401**.
- Doctorate: **258**.
- The applicants' educational backgrounds are various.

Africa and South America: Small but Qualified Pools

- Both regions have fewer applicants but stronger educational profiles:
- **Africa** has a significant number of Master's degrees (**288**) in comparison to its overall population.
- **South America**: Balanced across academic levels, with a focus on bachelor's (**333**) and master's (**293**).

Oceania: Small, but Highly Educated

- Despite having the smallest region (**192 total applicants**):
 - Strong representation of Bachelor's (**66**) and Master's (**68**) degrees.
 - Proportionately high education level among applicants.

Insights:

- Regional educational patterns differ greatly.
- Asia's dominance is steady across all educational levels.
- European applicants are disproportionately represented in the advanced degree categories.
- All regions have a substantial representation in higher education (Bachelor's and above)

Obs

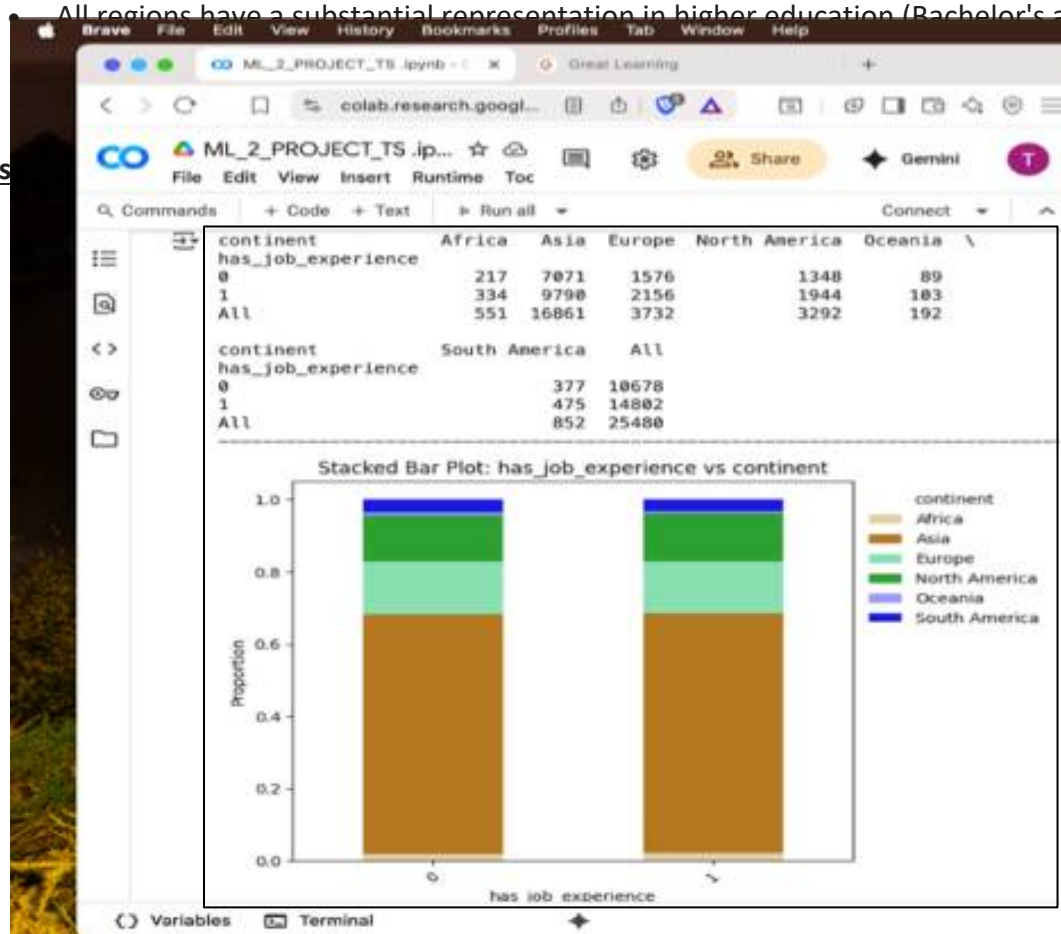


Figure 27. has_job_experience vs continent

Observations:

Experienced applicants form the majority everywhere

- Across all continents, applicants with work experience (Y:1) outweigh those without.
- This global pattern confirms that experienced professionals are the top applicants in all regions.

Asia dominates both experience categories

- Asia has the biggest number of experienced (**9,790**) and novice (**7,071**) applications.
- This reflects Asia's overwhelming domination of the whole candidate pool.

Europe has the highest experience rate

- Europe has the most experienced applicants:
- Experienced: 2,156 (**58%**).
- Inexperienced: 1,576 (**42%**).

European applicants are more likely to have professional experience.

Africa Demonstrates Notable Experience Level

- Despite its tiny population, Africa has a significant experience profile:
- Experienced: 334 (**61%**).
- Inexperienced: 217 (**39%**).
- African applicants have one of the highest experience rates across all areas.

Consistent Global Pattern

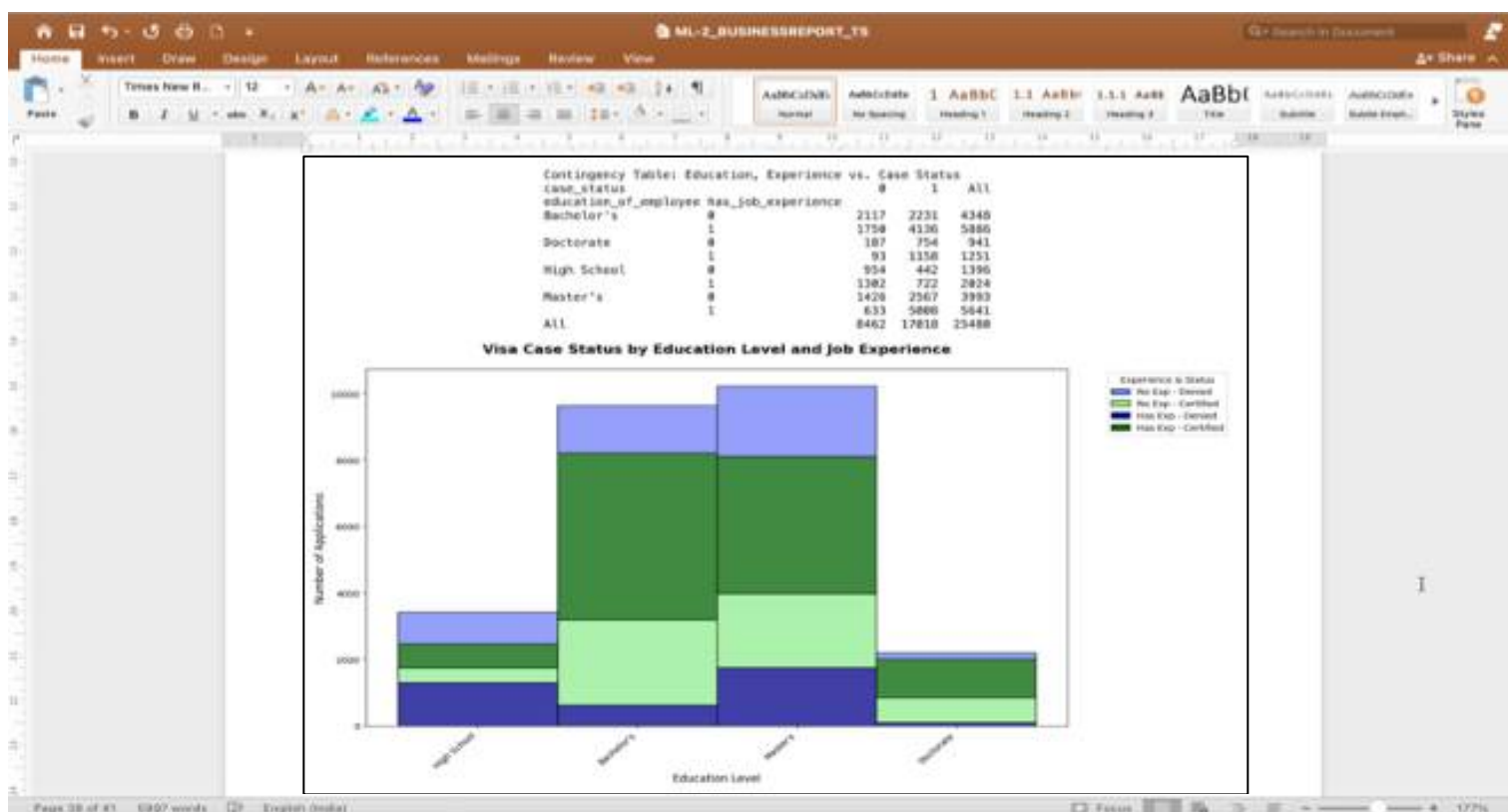
- In all regions, there are more experienced applicants than inexperienced ones.
- The ratio is largely stable across continents.

Insights:

- Experience is appreciated universally in all locations.
- The European and African candidates have the highest number of experienced professionals.
- Asian applicants lead in absolute terms for both experience categories.
- No region focusses heavily on inexperienced applicants.

3.3 Multivariate Analysis

Multivariate analysis is done on three or more variables. We will create contingency table for education_of_employee & has_job_experience vs case_status to analyse how education level of employees and job experience affects visa application status.



Observations:

Best combination: Advanced Degree and Experience

- Master's with Experience: Has the **most certified applications** (5,008).
- Doctorate with Experience has the **greatest certification rate** ($1,158 / 1,251 = 92.6\%$).
- This combination provides the most favourable profile for visa clearance.

Experience person with bachelor's degree

- Without experience: **51.3%** certification rate ($2,231 / 4,348$).
- With experience: certification rate of 70.3% ($4,136 / 5,894$)
- For bachelor's degree holders, gaining experience enhances their **chances of acceptance by 19 percentage points**.

High School Without Experience

- This group has the **lowest certification rate** (31.7 percent, $442 / 1,396$).
- It reflects the **most difficult profile for visa approval**.

Even doctorates require experience

- Without Experience: **80.1%** certification percentage ($754/941$) - Good
- With experience: **92.6%** certification rate ($1,158/1,251$) - Excellent
- Experience still provides a considerable **12.5 percentage point advantage** for the highest degree of schooling.

Experience Doesn't Fully Compensate for Low Education

- High School with Experience still has a certification percentage of 35.7% ($722 / 2,024$).
- This is lower than the bachelor's without experience (51.3%).

Insights:

- Advanced Education levels & experience are key factors to determine if the person's visa is approved or rejected.
- A doctorate with experience achieves 92.6%, whereas a doctorate without experience achieves "only" 80.1%.
- Similarly, experience can assist a bachelor's degree holder advance from average (51.3%) to good (70.3%) prospects, but it cannot overcome the restrictions of a high school education.

3.4 Overall EDA Insights

Education is the single most important factor

- Higher education greatly improves approval chances.
- Doctorate holders have the best success rate (**87%**).
- High school graduates have the toughest odds (**34% approval**).
- Each increase in education level considerably boosts approval chances.

Job experience provides a significant boost

- Experienced applicants are **33% more likely to get approved**.
- Experience is most important for Bachelor's holders (19% point gain).
- Doctorate holders receive a 13% point boost from experience.
- The winning combo is advanced degrees + experience.

Company stability outperforms company size (no_of_employees)

- Company age does not impact much; new and old enterprises have similar approval percentages.
- Company size indicates only a modest advantage for larger companies.
- What counts more is the work type (full-time/annual wage vs. hourly).

Job quality is more important than job location

- Full-time, salaried positions have a significantly higher approval rate.
- Hourly wage approval is quite low (35%).
- The Midwest has the highest approval rate (76%), but disparities between areas are lower than other characteristics.

The system promotes stable, professional employment

- Yearly salaries are preferable to hourly earnings.
- Full-time opportunities are preferred.
- Professional experience is quite valuable.
- Advanced schooling is highly preferred.

No single factor ensures success or failure

- Even Doctorate holders get denied sometimes.

- Some high school grads still receive approval.
- The choice examines numerous aspects together to determine visa status.

The best profile for visa approval:

- Education: Doctorate or Master's degree.
- Experience: At least some work experience.
- Job Type: Full-Time, Salary Position.
- Wage: meets or surpasses the prevailing wage.
- Region: Midwest or south areas have a minor edge.

The most challenging profile:

- Education: High school only.
- Experience: No work experience.
- Job Type: Hourly, Part-Time Position.
- Wage: On the low end of the scale.

Conclusion:

- **The EDA analysis shows a clear trend: the visa approval system is designed to certify educated, experienced professionals entering stable, well-compensated positions.**

4 DATA PREPROCESSING

In this section, we will handle missing values & duplicate values to ensure no data leakage.

4.1 Data Preprocessing

We converted all

The screenshot shows a Jupyter Notebook interface in a Brave browser. The code cell [54] contains the following Python code:

```
[54] # convert all columns with dtype object into category
for col in df.columns[df.dtypes=='object']:
    df[col] = df[col].astype('category')

df.info()
```

The output of the code is a summary of the DataFrame:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                ---
0   continent                             25480 non-null  category
1   education_of_employee                 25480 non-null  category
2   has_job_experience                    25480 non-null  int64
3   requires_job_training                25480 non-null  int64
4   no_of_employees                      25480 non-null  int64
5   yr_of_estab                          25480 non-null  int64
6   region_of_employment                 25480 non-null  category
7   prevailing_wage                      25480 non-null  float64
8   unit_of_wage                         25480 non-null  category
9   full_time_position                   25480 non-null  int64
10  case_status                          25480 non-null  int64
dtypes: category(4), float64(1), int64(6)
memory usage: 1.5 MB
```

Observations:

- We can see object dtype has been successfully converted to the category dtype.
- has_job_experience, requires_job_training, full_time_position, case_status : these we already label encoded to binary values (0/1). Our target (case_status) variable successfully label encoded (converted from 'Y'/'N' to 1/0). Therefore, they are suitable for machine learning models.

Observations:

- We can see object dtype has been successfully converted to the category dtype.
- has_job_experience, requires_job_training, full_time_position, case_status : these we already label encoded to binary features, including our target (case_status) variable successfully label encoded (converted from 'Y'/'N' to 1/0). Therefore, they are shown as int dtype.

4.2 Feature Engineering

We did not create any new features because:

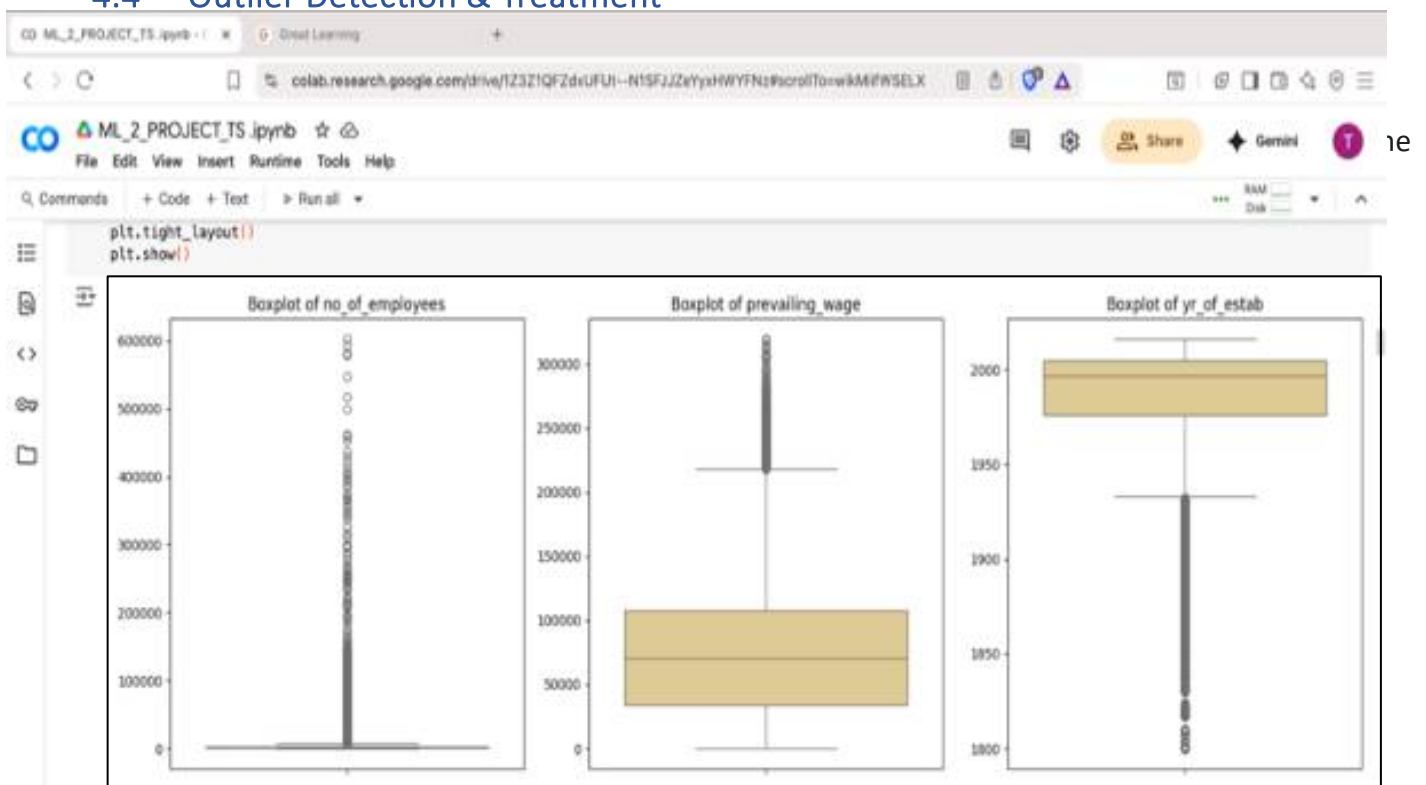
- The classifier models we'll create can automatically detect the interactions and non-linear patterns we identified in the EDA.
- Adding new features will not make any significant impact and could hinder model's generalisation.

4.3 Missing & Duplicate values

We already checked the missing and duplicate values in our data set in data overview section. We found:

- There are no missing & duplicate values in our dataset.
- Our data is clean to be used for model.

4.4 Outlier Detection & Treatment



Observations:

- The box plots shows there are several extreme outliers in all columns.
- We will further verify these outliers using IQR method.

Outlier Verification using IQR method

ML_2_PROJECT_TS.ipynb

File Edit View Insert Runtime Tools Help

Commands

+ Code + Text ▶ Run all ▼

lower_bound = Q1 - 1.5 * IQR

upper_bound = Q3 + 1.5 * IQR

return lower_bound, upper_bound

Check each column

for col in num_cols:

lower_bound, upper_bound = find_iqr_bounds(df[col])

outlier_count = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()

print(f"{col}:")

print(f" Lower bound: {lower_bound:.2f}, Upper bound: {upper_bound:.2f}")

print(f" Number of outliers: {outlier_count} ({outlier_count/len(df)*100:.2f}%)\\n")

no_of_employees:

Lower bound: -2701.00, Upper bound: 7227.00

Number of outliers: 1556 (6.11%)

prevailing_wage:

Lower bound: -76564.57, Upper bound: 218315.56

Number of outliers: 427 (1.68%)

yr_of_estab:

Lower bound: 1932.50, Upper bound: 2048.50

Number of outliers: 3260 (12.79%)

Observations:

- The box plot
- We will furth

Outlier Verification

Observations:

- There are m
- (1.68%) and
- We will chec

Outlier Treatment

We will not be treating any outliers because:

Companies that have much more employees (no. of employees) than the average (for example, major organisations with tens or hundreds of thousands of people). Large, well-established businesses are frequently seen as more reliable and authentic, which may have a favourable impact on the certification decision. This crucial signal would be lost if these values were removed or capped, which would stop the model from recognising the trend that applications from large companies had a higher chance of being verified.

We decided to not treat any outliers because:

- no_of_employees:** Companies that have much more employees (no_of_employees) than the average (for example, major organisations with tens or hundreds of thousands of people). Large, well-established businesses are frequently seen as more reliable and authentic, which may have a favourable impact on the certification decision. This crucial signal would be lost if these values were removed or capped, which would stop the model from recognising the trend that applications from large companies had a higher chance of being verified.
- prevailing_wage:** Wages are extremely high for specialised, in-demand occupations (e.g., senior surgeons, AI specialists) and very low for entry-level or part-time positions. A "speciality occupation," which is a crucial requirement for visa certification, is strongly indicated by exceptionally high earnings. On the other hand, extremely low pay could be a good excuse for rejection. The model's capacity to identify these essential linkages would be diminished if these extremes were capped.
- yr_of_estab:** Companies established in the 1800s or very early 1900s ("legacy" or old companies). A company's age is a major indicator of its stability and repute. A visa application from a century-old, well-established corporation is considered less risky than one from a new company. Removing these past data points would reduce the model's capacity to use corporate stability as a predictor.

38

EDA

- It is a good idea to explore the data once again after manipulating it.
- However, we have not manipulated the data by treating outliers, filling null values or removing duplicates, we will not be re exploring the data.

4.5 Data Preparation For Modeling

We already label encoded the categorical columns. **We did not do feature scaling** on the numerical predictors prior to model training because tree-based models are scale-invariant. We will create dummy variables of our independent categorical variables using **one-hot encoding**. Then, we'll **split the data into train, validation and test** to be able to evaluate the model that we build on the train data. This three split is done to **avoid any data-leakage**.

We will first we need to select our target (`case_status`) variable from the dataframe and separate it from the independent variables(predictors), then encode categorical variable and finally split the dataset.

Splitting the data into train, validation and test set (60:20:20 ratio):

- X train → 60% of feature data will be used to train the model.

The screenshot shows a Google Colab notebook with the following content:

```
X_train, y_train, X_test, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
```

Below the code, the first five rows of the `X_train` dataframe are displayed:

full_time_position	continent_Asia	continent_Europe	continent_North America	continent_Oceania	continent_South America	education_of_employee_Doctorate	education_of_employ
1	1	0	0	0	0	0	
1	1	0	0	0	0	0	
1	1	0	0	0	0	0	
1	1	0	0	0	0	0	
1	0	0	0	0	0	0	

Below the table, the data types and value counts for the `X_train` dataframe are shown:

```
# Check data types
print("\nData types:")
print(X_train.dtypes.value_counts())
```

Data types:
int64 21
Name: count, dtype: int64

Observations:

Train – Validation – Test

The screenshot shows a Jupyter Notebook interface. The code cell contains the following Python code:

```
print(y_train.value_counts(normalize=True))
print("\nTarget distribution in Validation set:")
print(y_val.value_counts(normalize=True))
print("\nTarget distribution in Test set:")
print(y_test.value_counts(normalize=True))
```

The output of the code is displayed in a box:

```
Training set shape: (15288, 21) (15288,)
Validation set shape: (5096, 21) (5096,)
Test set shape: (5096, 21) (5096,)

Target distribution in Training set:
case_status
1    0.667909
0    0.332091
Name: proportion, dtype: float64

Target distribution in Validation set:
case_status
1    0.667779
0    0.332221
Name: proportion, dtype: float64

Target distribution in Test set:
case_status
1    0.667975
0    0.332025
Name: proportion, dtype: float64
```

Below the output box, there is a section titled "Observations:" with the following bullet points:

- Data is imbalanced with 2:1 ratio i.e. for every 2 certified cases, there is 1 denied case
- There are no row-column mismatch in the split dataset, perfect alignment achieved after the split.
- We can proceed with model building.

On the right side of the notebook, there is a sidebar with a table titled "One- Hot Encoding" and a section titled "Train – Validation – Test".

Observations:

- Data is imbalanced
- There are **no row-column mismatch** in the split dataset, perfect alignment achieved after the split.
- We can proceed with model building.

5 MODEL BUILDING

5.1 Model Evaluation Criteria

The model may make two types of inaccurate predictions:

- **False Positive (Type I Error):** The model predicts that the visa application will be certified when, in fact, it should be denied.
- **False Negative (Type II Error):** The model predicts that the visa application will be denied, whereas in fact it should be certified.

Both type of error can lead to significant business consequences:

False Positives (Certified → Actually Denied)

- Improperly certified visas may replace qualified US workers.
- Employers may hire less qualified foreign labour than qualified citizens.
- Possible detrimental effect on domestic employment opportunities.
- Violation of the Immigration and Nationality Act's worker protection clauses

False Negatives (Denied to Certified)

- US firms are losing access to competent people that may contribute to economic growth.
- Companies confront manpower shortages, which may harm competitiveness.
- Missed possibilities for knowledge transfer and skill diversity.

- Potential

Given the equal i

- **F1-Score** of both fa
- The F1-s between incorrect

5.2 Initial M

Before building t
the train, validat
do not have to u

5.2.1 Decisic

Model Building

We build the mo

- We have s

Performance Eva

Training Set Perf.....

- The model performed perfectly on the training data, with all metrics at **1.0**.
- This shows that the decision tree has entirely memorised the training patterns.
- Accuracy, recall, precision, and F1-score have all hit 100%.

The screenshot displays a Google Colab notebook titled 'ML_2_PROJECT_TS.ipynb'. The notebook is open to a cell where a Decision Tree model is built and evaluated. The code in the cell is as follows:

```
[64] # Let's build the decision tree

dtree = DecisionTreeClassifier(random_state=42, class_weight='balanced')
dtree.fit(X_train, y_train)
```

The output of the code shows the model object: `DecisionTreeClassifier(class_weight='balanced', random_state=42)`. Below the code, a text box states: "We have successfully build the initial decision tree model."

The next cell in the notebook is titled "Checking Initial Decision Tree Model's Performance on Train set". The code in this cell is:

```
[65] # Evaluate
print("Training Performance-DT:")
dt_train_perf = model_performance_classification_sklearn(dtree, X_train, y_train)
dt_train_perf
```

The output of this cell shows the training performance metrics:

Training Performance-DT:				
Accuracy	Recall	Precision	F1	
1.0	1.0	1.0	1.0	1.0

Below the table, the text "Training Set Performance Observations:" is visible.

- This level of p

Validation Set Perfo

- Performance
- The F1 score
- A rec
- certif
- The p
- The 2

Confusion M

Training Set

- The r
- All **15,288** cases were accurately classified.
- 5,077 denials and 10,211 certificates were predicted correctly.
- This precise separation is typical of decision trees without constraints.

• This level of performance indicates **severe overfitting** on the training data.

```
[66] # Evaluate
print("Validation Performance-DT:")
dt_val_perf = model_performance_classification_sklearn(dtree, X_val, y_val)
dt_val_perf
```

	Accuracy	Recall	Precision	F1
0	0.658752	0.744931	0.744275	0.744603

Validation Set Performance Observations:

- Performance decreased dramatically to **65.88% accuracy** on validation data.
- The **F1 score of 74.46%** shows **moderate performance** on unseen data.
- A **recall of 74.49%** indicates that the model accurately detects three out of four actual certifications.

ML_2_PROJECT_TS.ipynb - X Great Learning

colab.research.google.com/drive/12 To exit full screen, press and hold **esc**

ML_2_PROJECT_TS.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Confusion Matrix Plot for Initial Decision Tree model

```
[67] print("Confusion Matrix-DT (Train):")
confusion_matrix_sklearn(dtree, X_train, y_train)
```

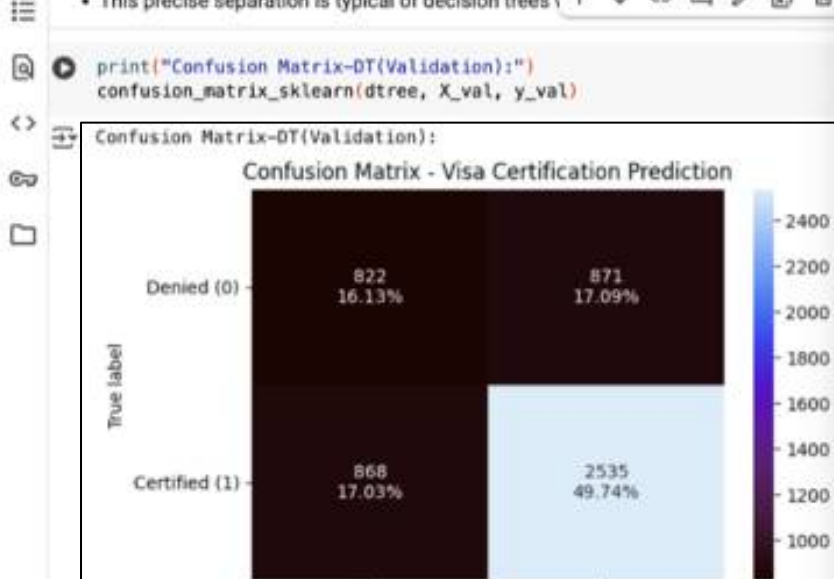
Confusion Matrix-DT (Train):

True label \ Predicted label	Denied (0)	Certified (1)
Denied (0)	5077 33.21%	0 0.00%
Certified (1)	0 0.00%	10211 66.79%

Training Set Confusion Matrix Observations:

- The model made no errors in the training set.
- All **15,288** cases were accurately classified.

Variables Terminal



Training Set Confusion Matrix

- The model made no error
- All 15,288 cases were correctly predicted
- 5,077 denials and 10,211 certifications
- This precise separation is typical of decision trees

Validation Set

- The model made 87 errors
- Correctly predicted 2535 cases
- Error rate is 3.3%
- There are 87 false positives (FP)
- There are 87 false negatives (FN)

5.2.2 Bagging

We will build a

Random Forest

- We have a model that we can fit it on

Performance

Word File Edit View Insert Format Tools Table Window Help

ML_2_PROJECT_TS.ipynb - X Great Learning

colab.research.google.com/dri...

ML_2_PROJECT_TS.ip... ☆ ⚙️

File Edit View Insert Runtime Toc

Commands + Code + Text Run all Connect

Random Forest

```
# Let's build the bagging Random Forest model
```

Word File Edit View Insert Format Tools Table Window Help

ML_2_PROJECT_TS.ipynb - X Great Learning

colab.research.google.com/dri...

ML_2_PROJECT_TS.ip... ☆ ⚙️

File Edit View Insert Runtime Toc

Commands + Code + Text Run all Connect

```
random_state=42  
class_weight='balanced')  
bagging_rf.fit(X_train, y_train)
```

RandomForestClassifier

```
RandomForestClassifier(class_weight='balanced', random_state=42)
```

Checking Initial Bagging (Random Forest) Model's Performance on Train set

```
# Evaluate  
print("Training Performance-RF:")  
rf_train_perf = model_performance_classification_sklearn(bagging_rf, X_train, y_train)  
rf_train_perf
```

Training Performance-RF:

	Accuracy	Recall	Precision	F1
0	0.999935	1.0	0.999902	0.999951

Training Performance Analysis:

- The model's performance on the training set is unrealistically perfect, sign of high overfitting.
- **Accuracy: ~99.99%:** The model accurately predicted almost all cases in the training

Training Performance Observations:

- The model's performance on the training set is unrealistically perfect, sign of high overfitting.
- **Accuracy: ~99.99%:** The model accurately predicted almost all cases in the training data.
- **The recall of 1.0 and precision of ~0.9999.** This signifies that the model made nearly no errors in recognising "Denied" applications during training. The algorithm detected all denials (Recall=1.0) and accurately predicted "Denied" in virtually all cases (Precision~1.0).

Conclusion:

- The model's
- It has overle
- new data.

Validation Performance-RF:

	Accuracy	Recall	Precision	F1
0	0.725275	0.851895	0.7639	0.805502

Validation Performance Analysis:

- The performance declines considerably on the validation set, confirming the overfitting observed during the training phase.
- **Accuracy: ~72.5%:** The model accurately predicts the outcome in around 7 out of 10 applications. This is a considerable decline from 99.99%, indicating overfitting.
- The **recall is 0.852**. This is the most significant positive metric. This means that the model correctly identifies 85.2% of all truly fraudulent or problematic ("Denied") applications. This is adequate for an initial model, as capturing denials is frequently a crucial.
- The **precision is 0.764**. This means that only 76.4% of the applications predicted by the model as "Denied" were really denied. The remaining 23.6% were false alarms (false positives), which means that legitimate applications were mistakenly identified for rejection. This could result in unnecessary manual scrutiny and applicant unhappiness.
- **F1-Score (0.806):** The harmonic mean of Precision and Recall yields a single score for the "Denied" class' performance. A score of 0.806 is acceptable, but there is space for growth, particularly on the Precision side.

Confusion matrix of bagging initial model

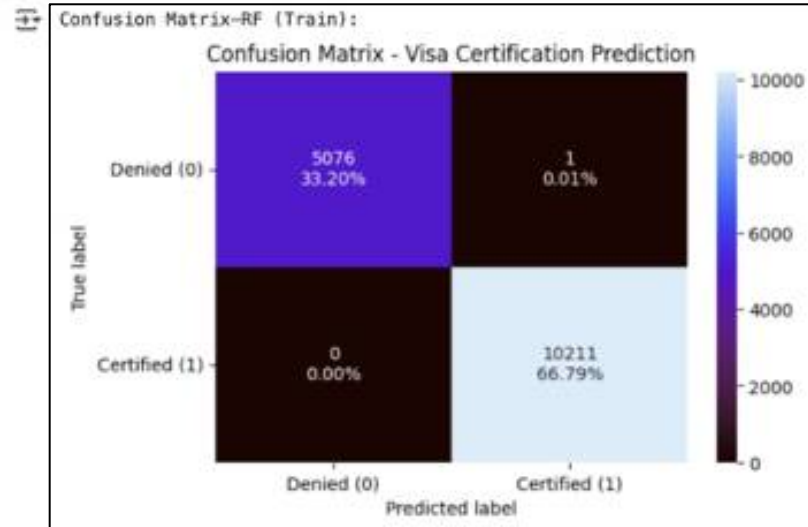
Validation Perform

- The perform
- during the t
- **Accuracy: ~**
- This is a con
- The **recall is**
- correctly ide
- adequate fo
- The **precisio**
- "Denied" w
- means that
- unnecessary
- **F1-Score (0**
- "Denied" cl
- particularly

Confusion Matrix

Confusion matrix of bagging initial model

```
[ ] print("Confusion Matrix-RF (Train):")
confusion_matrix_sklearn(bagging_rf, X_train, y_train)
```



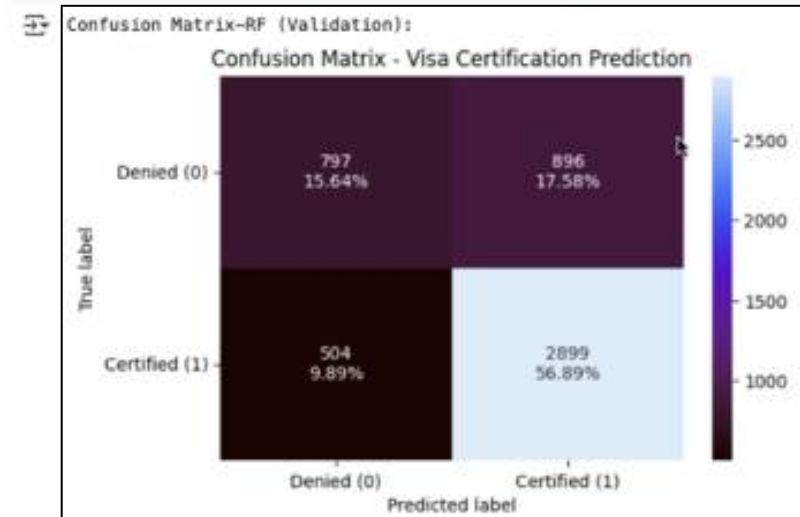
Training Confusion Matrix:

- There were 10,211 applications were successfully forecasted as Certified (true positives).
- Approximately 5076 applications were accurately forecasted as denied (True Negatives).

Training Confusion Matrix

- There were 10,211 applications were successfully forecasted as Certified (true positives).
- Approximately 5076 applications were accurately forecasted as denied (True Negatives).
- Only one application was incorrectly forecasted as denied when they were really certified (False Negative).
- 0 applications were incorrectly classified as denied when they were really certified (False Positive).

```
[ ] print("Confusion Matrix-RF (Validation):")
confusion_matrix_sklearn(bagging_rf, X_val, y_val)
```



Validation Confusion Matrix Observations:

Training Confusion Matrix:

- There were 10,211 applications were successfully forecasted as Certified (true positives).
- Approximately 5076 applications were accurately forecasted as denied (True Negatives).
- Only one application was incorrectly forecasted as denied when they were really certified (False Negative).
- 0 applications were incorrectly classified as denied when they were really certified (False Positive).

Validation Confusion

- 2,899 applica
- 797 applicati
- 896 applicati
- 4307 applica

5.2.3 Boosting

AdaBoost Initial Mo

- We have suc

Performance Evalu

Training Performan

- **Accuracy (0.741):** The model has a high level of accuracy, which means it accurately classifies a large portion
- **Recall (0.883):** A high recall indicates that the model is excellent at detecting positive instances while minimising false negatives.
- **Precision (0.765):** The precision score suggests a decent mix of true and false positives, but there is still space for improvement.
- **F1 Score (0.820):** The F1 score represents a good mix of precision and recall, indicating that the model performs well overall.

The top screenshot shows the Google Colab notebook interface. The notebook is titled 'ML_2_PROJECT_TS.ipynb'. The left sidebar shows the file explorer with a folder named 'Bagging Classifier Initial Model Building' and a file named '12 cells hidden'. The main area shows the 'Boosting Initial Model Building' section, with 'AdaBoost Initial Model Building' expanded. The code cell shows the following code:

```
[ ] ada_boost = AdaBoostClassifier(random_state=42)
```

The middle screenshot shows the same notebook interface, but with the 'Checking Performance of Adaboost model' section expanded. The code cell shows the following code:

```
[ ] #train set performance
print("Training Performance-Adaboost:")
ada_train_perf = model_performance_classification_sklearn(ada_boost, X_train, y_train)
ada_train_perf
```

The output of the code is a table showing the training performance of the Adaboost model:

	Accuracy	Recall	Precision	F1
0	0.741627	0.883851	0.765544	0.820455

The bottom screenshot shows the 'Training Performance Observations' section, which provides a detailed analysis of the training performance metrics:

- **Accuracy (0.741):** The model has a high level of accuracy, which means it accurately classifies a large portion of the training data.
- **Recall (0.883):** A high recall indicates that the model is excellent at detecting positive instances while minimising false negatives.
- **Precision (0.765):** The precision score suggests a decent mix of true and false positives, but there is still space for improvement.
- **F1 Score (0.820):** The F1 score represents a good mix of precision and recall, indicating that the model performs well overall.

The code cell at the bottom shows the following code:

```
[ ] #validation set performance
print("Validation Performance-Adaboost:")
ada_val_perf = model_performance_classification_sklearn(ada_boost, X_val, y_val)
ada_val_perf
```

```
[ ] #validation set performance
print("Validation Performance-Adaboost:")
ada_val_perf = model_performance_classification_sklearn(ada_boost, X_val, y_val)
ada_val_perf
```

Validation Performance-Adaboost:				
	Accuracy	Recall	Precision	F1
0	0.738226	0.891566	0.75869	0.819778

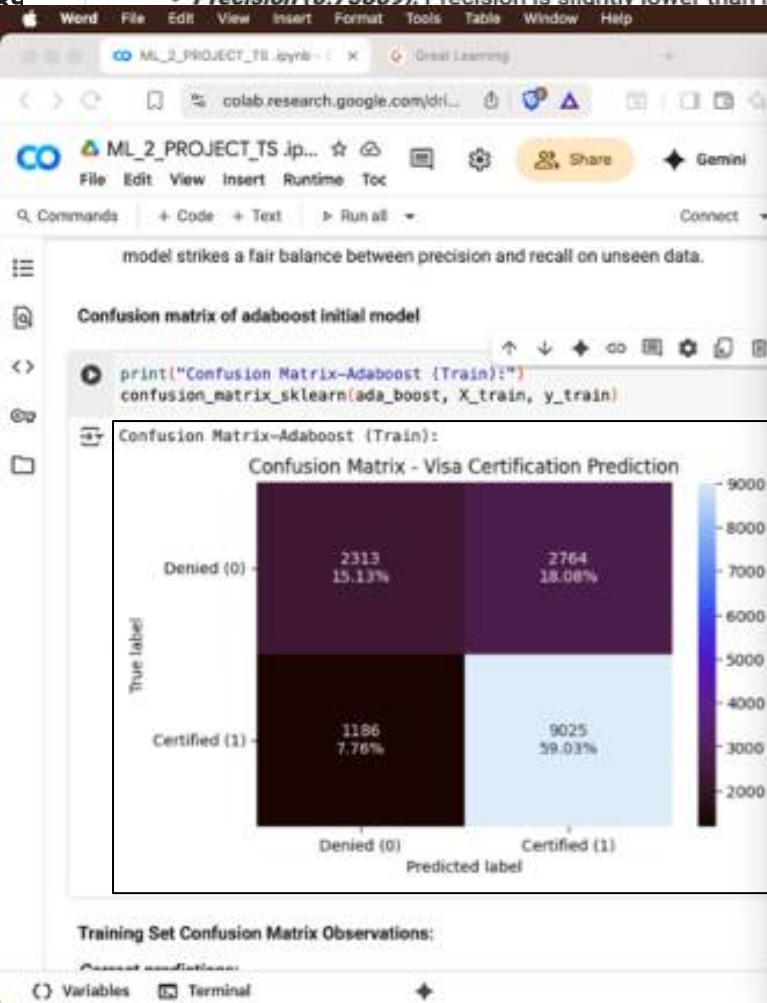
Validation Performance Observations:

- **Accuracy (0.7382):** The accuracy on the validation set is slightly lower than the training accuracy, which could indicate overfitting.
- **Recall (0.8915):** The recall remained strong, comparable to the training set, indicating consistent performance in detecting good instances.
- **Precision (0.75869):** Precision is slightly lower than in training, indicating a minor risk of false positives during validation.

Validation Perform

- **Accuracy (0.7382):** The accuracy on the validation set is slightly lower than the training accuracy, which could indicate overfitting.
- **Recall (0.8915):** The recall remained strong, comparable to the training set, indicating consistent performance in detecting good instances.
- **Precision (0.75869):** Precision is slightly lower than in training, indicating a minor risk of false positives during validation.
- **F1 Score (0.819778):** The F1 score provides a fair balance between precision and recall on unseen data.

Confusion Matrix



Validation Performance Observations:

- **Accuracy (0.7382):** The accuracy on the validation set is slightly lower than the training accuracy, which could indicate overfitting.
- **Recall (0.8915):** The recall remained strong, comparable to the training set, indicating consistent performance in detecting good instances.
- **Precision (0.75869):** Precision is slightly lower than in training, indicating a minor risk of false positives during validation.
- **F1 Score (0.819778):** The F1 score provides a fair balance between precision and recall on unseen data.

Confusion Matrix

Training Set Conf

Correct predictions:

- Certified visas accurately predicted (TP): **9025** (59.03%).
- Denied visas accurately predicted (TN): **2313** (~15.13%).

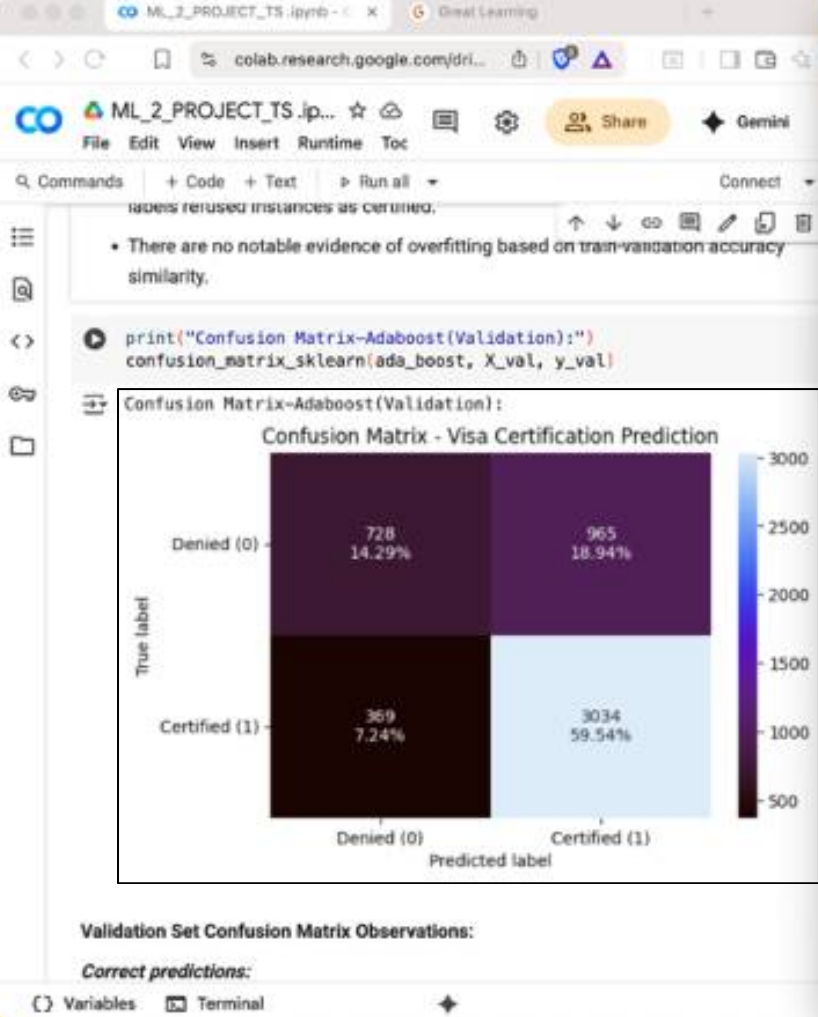
Incorrect predictions:

- Certified predicted as denied (FN): **1186** (approx. 7.76%)
- Denied predicted as Certified (FP): **2764** (~18.08%).

Conclusions:

- The model performs better at recognising certified visas than refused visas.

- False Positive
- refused
- There are



Conclusions:

- The model performs better
- False Positives (18%) are refused instances as certified
- There are no notable evidence of overfitting based on train-validation accuracy similarity.

Validation Set (

Correct predictions:

- Certified v
- Denied vis

Incorrect predictions:

- Certified p
- Denied pre

Conclusions:

- Performan
- The model
- The false p
- some deni

5.2.4 Gradient

Model Building

```

AdaBoost Initial Model Building

[ ] 11 cells hidden

Gradient Boost Initial Model Building

gb_model = GradientBoostingClassifier(random_state=42)
gb_model.fit(X_train, y_train)

GradientBoostingClassifier
GradientBoostingClassifier(random_state=42)

Checking Performance of Gradient Boosting Initial Model

[ ] # Train-set Performance of Gradient Boosting Initial Model
print("Training Performance-Gradient Boosting Model:")
gb_train_perf = model_performance_classification_sklern(gb_model, X_train
  
```

- We have suc and fit it on t

Performance Evalua

Training Performa

- **Accuracy** (training da
- **Recall** (0.8 cases and l
- **Precision** (l is still room
- **F1 Score** (0 the model

Validation Perform

- **Accuracy** (could indic
- **Recall** (0.8 consistent
- **Precision** (positives d
- **F1 Score** (0 a solid bala

GradientBoostingClassifier(random_state=42)

Checking Performance of Gradient Boosting Initial Model

```
# Train-set Performance of Gradient Boosting Initial Model
print("Training Performance-Gradient Boosting Model:")
gb_train_perf = model_performance_classification_sklearn(gb_model, X_train,
gb_train_perf
```

Training Performance-Gradient Boosting Model:

Brave File Edit View History Bookmarks Profiles Tab Window Help

ML_2_PROJECT_TS.ipynb - X Great Learning

colab.research.google.com/dri...

ML_2_PROJECT_TS.ip... ☆

File Edit View Insert Runtime Toc

Commands + Code + Text Run all Connect

- **Precision (0.7752):** The precision shows a good ratio of true positives to false positives, but there is still room for improvement.
- **F1 Score (0.8303):** The F1 score represents a good balance of precision and recall, showing that the model performs well during training.

```
# Validation-set Performance of Gradient Boosting Initial Model
print("Validation Performance-Gradient Boosting Model:")
gb_val_perf = model_performance_classification_sklearn(gb_model, X_val, y_
gb_val_perf
```

Validation Performance-Gradient Boosting Model:

	Accuracy	Recall	Precision	F1
0	0.748823	0.878636	0.775214	0.823691

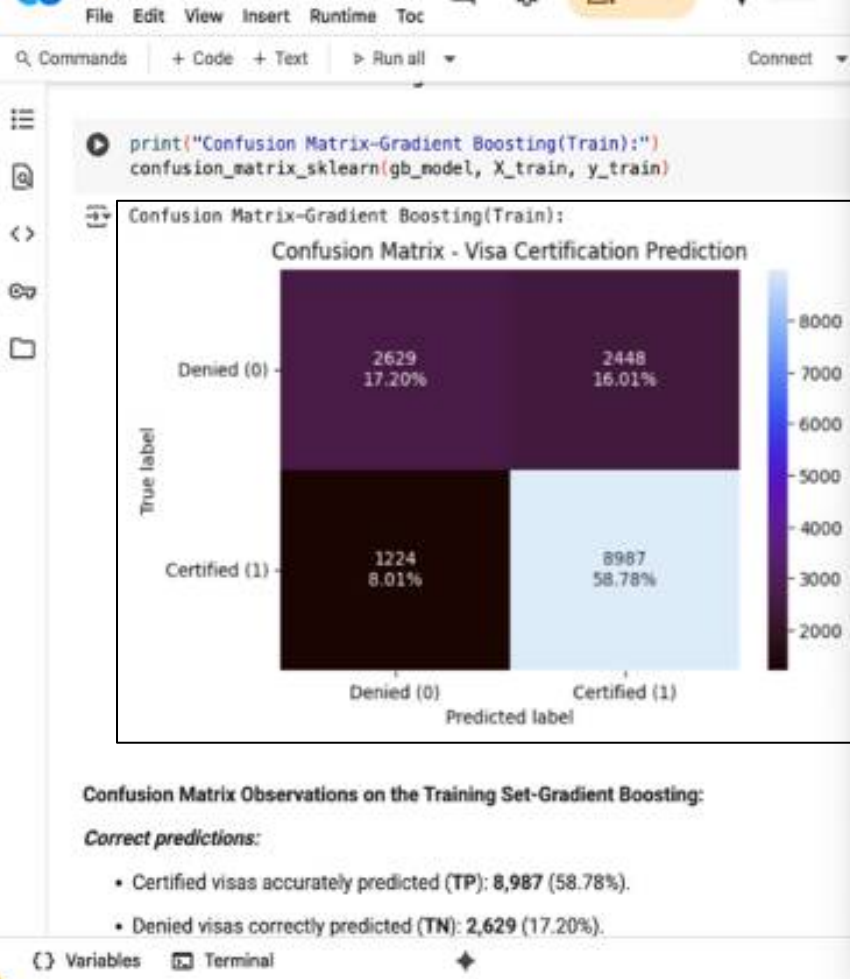
Validation Performance Observations for the Gradient Boosting Model:

- **Accuracy (0.7488):** The validation set has slightly lower accuracy than the training set, which could indicate some degree of overfitting.
- **Recall (0.8786):** The recall is still high, similar to the training performance, demonstrating consistent effectiveness in identifying the positive cases.
- **Precision (0.7752):** Precision is slightly lower than in training, indicating a small rise in false positives during validation.
- **F1 Score (0.8236):** The F1 score is similar to the training score, indicating that the model retains a solid balance of precision and recall when tested on unseen data.

Confusion Matrix of Gradient Boosting Initial Model

Variables Terminal

Confusion Ma



Confusion Ma

Correct predic

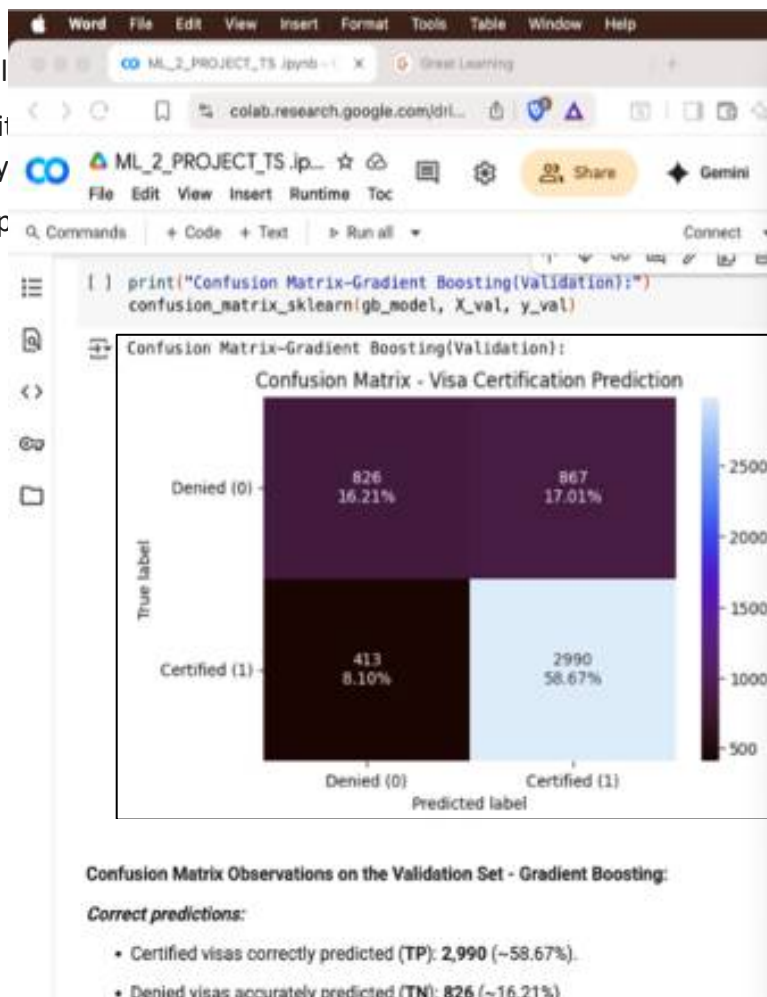
- Certified visas accurately predicted (TP): 8,987 (58.78%).
- Denied visas correctly predicted (TN): 2,629 (17.20%).

Incorrect predictions:

- Certified predicted as denied (FN): 1,224 (8.01%).
- Denied predicted as Certified (FP): 2,448 (~16.01%)

Conclusions:

- The model
- False posit
- incorrectly
- Balanced p



- Recall (0.8786):** The consistent effectiveness
- Precision (0.7752):** Pr
- F1 Score (0.8236):** The a solid balance of prec

Confusion Matrix

Page 54 of 55 8814 words

- The model predicts certified
- False positives (~16%) rema
- Balanced performance with r

Confusion Matrix Observations on the Validation Set - Gradient Boosting:

Correct predictions:

- Certified visas correctly predicted (TP): **2,990** (~58.67%).
- Denied visas accurately predicted (TN): **826** (~16.21%)

Incorrect predictions:

- Certified predicted as denied (FN): **413** (~8.10%).
- Denied predicted as Certified (FP): **867** (17.01%).

Conclusions:

- Validation performance roughly corresponds to training outcomes, showing strong generalisation.

- The cas
- The der

5.2.5

- We

Performan

Training Performance OI

- **Accuracy (0.8515)**: significant amount
- **Recall (0.9340)**: 1 positive cases, re
- **Precision (0.8566)**: The precision score reflects a high ratio of true positives to false positives, indicating that the model is reliable in making positive predictions.

The screenshot displays a Google Colab notebook titled 'ML_2_PROJECT_TS.ipynb'. The notebook is open to a cell containing XGBoost hyperparameters and a performance evaluation script. The hyperparameters are listed as follows:

```
colsample_bylevel=None, colsample_bynode=None, colsample_bytree=None, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric='logloss', feature_types=None, feature_weights=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=None, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=None, max_leaves=None, min_child_weight=None, missing=None, monotone_constraints=None, multi_strategy=None, n_estimators=None, n_jobs=None, num_parallel_tree=None, ...)
```

Below the hyperparameters, the notebook shows the output of the training performance evaluation. The output is a table with the following data:

	Accuracy	Recall	Precision	F1
0	0.851583	0.934091	0.856655	0.893699

The notebook also includes a section titled 'Checking Performance of XGBoost Initial Model' and a section titled 'Training Performance Observations for XGBoost:'. The observations are as follows:

- **Accuracy (0.8515)**: The model performs well on the training set, indicating that it can classify a significant amount of the training data.
- **Recall (0.9340)**: The recall score is quite high, indicating that the model excels at identifying positive cases, resulting in a low rate of false negatives.

- **F1 Score (0.893)**: The F1 score shows a good balance of precision and recall, indicating that the model performs very well during the training phase.

```
[ ] print("Validation Performance-XGBoost:")
xgb_val_perf = model_performance_classification_sklearn(xgb_model, X_val, y_val)
xgb_val_perf
```

Validation Performance-XGBoost:				
	Accuracy	Recall	Precision	F1
0	0.731947	0.859242	0.767253	0.810646

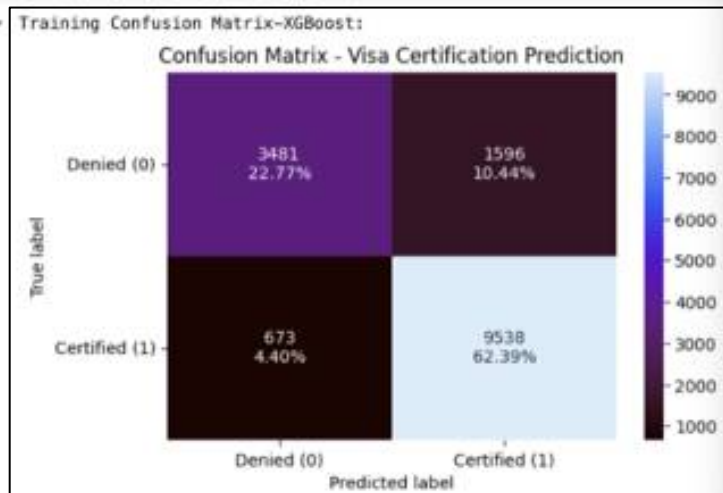
Validation and Performance Observations for XGBoost:

- **Accuracy (0.7319)**: The accuracy on the validation set is much lower than that on the training set, which could imply overfitting, as the model performs well on training data but poorly on unknown data.
- **Recall (0.8592)**: The recall remains quite high, although lower than the training recall, indicating that the model is still effective at identifying positive cases.
- **Precision (0.7672)**: The precision is slightly lower than the training precision, indicating that the model is still effective at identifying negative cases.
- The **F1 score (0.8106)** is a balanced measure of precision and recall, indicating that the model is still effective at identifying both positive and negative cases.

Confusion Matrix

Confusion Matrix of XGBoost Initial Model

```
print("Training Confusion Matrix-XGBoost:")
confusion_matrix_sklearn(xgb_model, X_train, y_train)
```



Observations of Confusion Matrix on Training Set-XGBoost:

Observations of Confusion Matrix on Training Set-XGBoost:

Correct predictions:

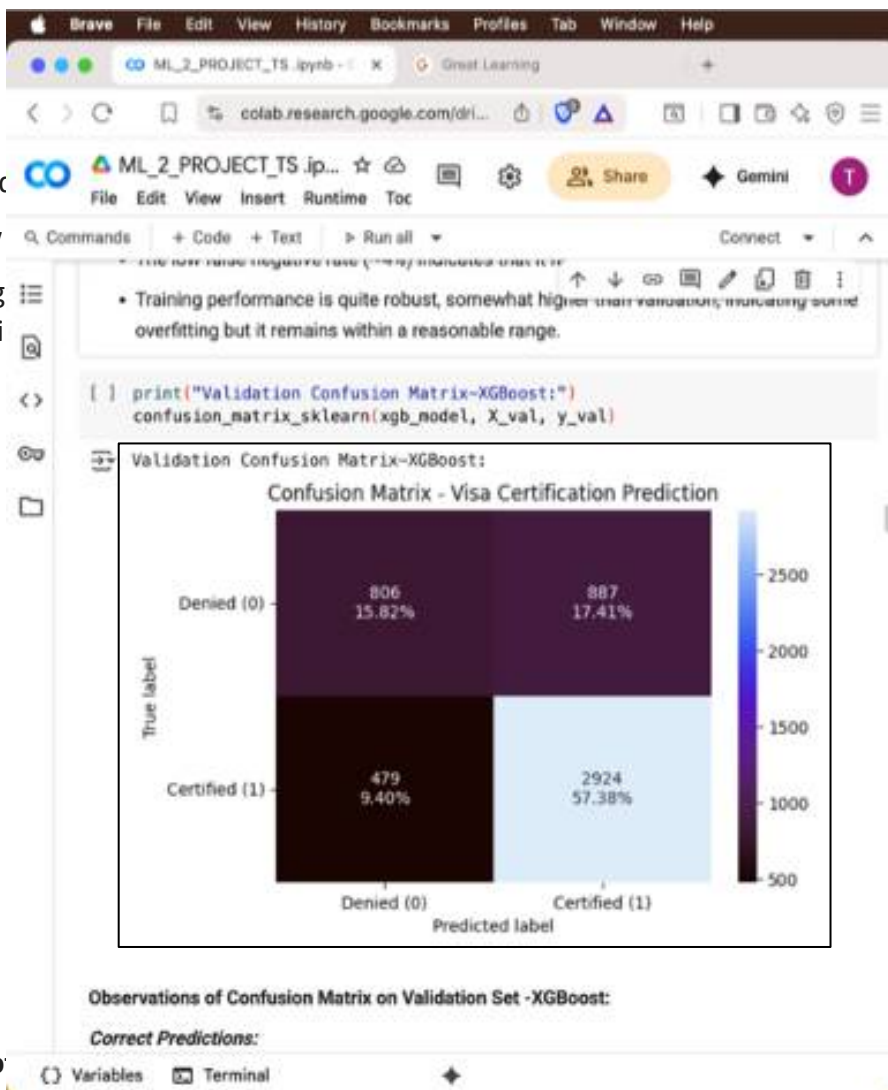
- Certified visas accurately predicted (TP): **9,538** (about 62.39%).
- Denied visas correctly predicted (TN): **3,481** (22.77%).

Incorrect predictions:

- Certified predicted as denied (FN): **673** (~4.40%).
- Denied predicted as Certified (FP): **1,596** (~10.44%).

Conclusions:

- The model achieves good performance on the validation set, indicating some overfitting.
- The low false negative rate (~9%) and false positive rate (~17%) on validation compared to training, but performance is still solid.
- Training performance is quite robust, somewhat higher than validation, indicating some overfitting but it remains within a reasonable range.



Observations of Confusion Matrix

Correct predictions:

- Certified visas accurately predicted
- Denied visas correctly predicted

Incorrect predictions:

- Certified predicted as Denied
- Denied predicted as Certified

Conclusions:

- The model achieves good performance on the validation set, indicating some overfitting.
- The low false negative rate (~9%) and false positive rate (~17%) on validation compared to training, but performance is still solid.
- Training performance is quite robust, somewhat higher than validation, indicating some overfitting but it remains within a reasonable range.

Observations of

Correct Predictions:

- Certified visas correctly predicted (TP): **2,924** (~57.38%)
- Denied visas correctly predicted (TN): **806** (~15.82%)

Incorrect Predictions:

- Certified predicted as Denied (FN): **479** (~9.40%)
- Denied predicted as Certified (FP): **887** (~17.41%)

Conclusions:

- Slight drop in performance compared to the training set, indicating some overfitting.
- Higher false negatives ~9% and false positives ~17% on validation compared to training, but performance is still solid.
- Model Retains a good ability to correctly predict certified cases, making it useful in practical scenarios where missing genuine certified cases is costly.

6 MODEL BUILDING

Oversampling is of minority class oversampling.

Checking Class Distribution

Checking class distribution before oversampling

```
[ ] # Checking class distribution before oversampling
print("Before Oversampling:")
print("Counts of label 'Denied': {}".format(sum(y_train == 0)))
print("Counts of label 'Certified': {}".format(sum(y_train == 1)))
print("Proportions: {:.2f}% Denied, {:.2f}% Certified\n".format(
    sum(y_train == 0)/len(y_train)*100,
    sum(y_train == 1)/len(y_train)*100
))
```

```
Before Oversampling:
Counts of label 'Denied': 5077
Counts of label 'Certified': 10211
Proportions: 33.21% Denied, 66.79% Certified
```

Observations:

Observations:

- Before oversampling, the dataset has a significant imbalance between the two labels, 'Denied' and 'Certified'. Specifically, there are:
- 5077 instances of 'Denied' (33.21%) and 10211 instances of 'Certified' (66.79%).
- To overcome these issues, oversampling (SMOTE-Synthetic Minority Oversampling)

- Before oversampling, the dataset has a significant imbalance between the two labels, 'Denied' and 'Certified'. Specifically, there are:
- 5077 instances of 'Denied' (33.21%) and 10211 instances of 'Certified' (66.79%).
- To overcome these issues, oversampling (SMOTE-Synthetic Minority Oversampling)

Applying SMOTE

We applied SMOTE = 1

Note: To ensure the model is performed on valid data on unseen data with

Checking Class Distribution

Checking class distribution after oversampling

```
[ ] # Check class distribution after oversampling
print("After Oversampling:")
print("Counts of label 'Denied': {}".format(sum(y_train_over == 0)))
print("Counts of label 'Certified': {}".format(sum(y_train_over == 1)))
print("Proportions: {:.2f}% Denied, {:.2f}% Certified\n".format(
    sum(y_train_over == 0)/len(y_train_over)*100,
    sum(y_train_over == 1)/len(y_train_over)*100
))
```

```
After Oversampling, shape of X_train: {}
print("After Oversampling, shape of y_train: {}".format(y_train_over.shape))

After Oversampling:
Counts of label 'Denied': 10211
Counts of label 'Certified': 10211
Proportions: 50.00% Denied, 50.00% Certified

After Oversampling, shape of X_train: (20422, 21)
After Oversampling, shape of y_train: (20422,)
```

Observations:

Observations:

- After applying oversampling, the dataset now has a balanced class distribution between the two labels, 'Denied' and 'Certified'. Specifically, there are:
 - 10211 instances of 'Denied' (50%)
 - 10211 instances of 'Certified' (50%).
- The change of the training features (X_train) and labels (y_train) show that the dataset

- After applying oversampling, the dataset now has a balanced class distribution between the two labels, 'Denied' and 'Certified'. Specifically, there are:
 - 10211 instances of 'Denied' (50%)
 - 10211 instances of 'Certified' (50%).
- The change of the training features (X_train) and labels (y_train) show that the dataset

- The shapes of the data were **successful**
- This is an efficient way to increase information.

6.1 Decision Tree

Model Building

- We successfully

Performance Evaluation

Training Performance

- **Accuracy (1.0)**: The model correctly classifies all instances, indicating it has memorised the training data.
- **Recall (1.0)**: A recall of 1.0 shows that the model correctly recognises all positive cases in the training data, demonstrating its ability to capture all instances of the minority class.
- **Precision (1.0)**: A perfect precision score indicates that all of the model's positive predictions are true positives, with no false positives in the training set.
- **F1 Score (1.0)**: An F1 score of 1.0 indicates an optimum balance of precision and recall, indicating that the model performs extraordinarily well on oversampled training data.
- These perfect results are due to synthetic data and do not reflect real-world performance.

10211 instances of 'Certified' (50%).

Word File Edit View Insert Format Tools Table Window Help

ML_2_PROJECT_TS.ipynb - x Great Learning

colab.research.google.co...

ML_2_PROJECT_TS.ipynb ☆

File Edit View Insert Runtime Toc

Commands + Code + Text Run all Connect

Decision Tree Model Building using

```
[ ] # Initialize and train
dt_model_over = DecisionTreeClassifier(random_state=42)
dt_model_over.fit(X_train_over, y_train_over)
```

DecisionTreeClassifier

DecisionTreeClassifier(random_state=42)

Checking Performance of Decision Tree-oversampled Model

```
[ ] # Training performance (on oversampled data)
print("TRAINING PERFORMANCE-DT (Oversampled Data):")
dt_train_perf_over = model_performance_classification_sklearn(dt_model_over, X_train_over, y_train_over)
dt_train_perf_over
```

	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0

Training Performance Observations for Decision Tree Model (Oversampled Data):

- **Accuracy (1.0)**: The model achieves complete accuracy on the training set, which means it correctly classifies all instances. However, this could indicate overfitting, as the model may have memorised the training data instead of learning generalisable patterns.
- **Recall (1.0)**: A recall of 1.0 shows that the model correctly recognises all positive cases in the training data, demonstrating its ability to capture all instances of the minority class.

Variables Terminal

Page 60 of 61 9933 words

```
print("VALIDATION PERFORMANCE-DT (Original Validation Data):")
dt_val_perf_over = model_performance_classification_sklearn(dt_model_over,
dt_val_perf_over
```

VALIDATION PERFORMANCE-DT (Original Validation Data):				
	Accuracy	Recall	Precision	F1
0	0.658163	0.733764	0.749175	0.74139

Validation Perform:

- **Accuracy (0.658163):** The validation set's accuracy is much lower than that of the training set, indicating that the model may not generalise effectively to new data, which is a common problem when overfitting occurs.
- **Recall (0.733764):** The recall score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.
- **Precision (0.749175):** The precision score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.
- **F1 Score (0.74139):** The F1 score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.

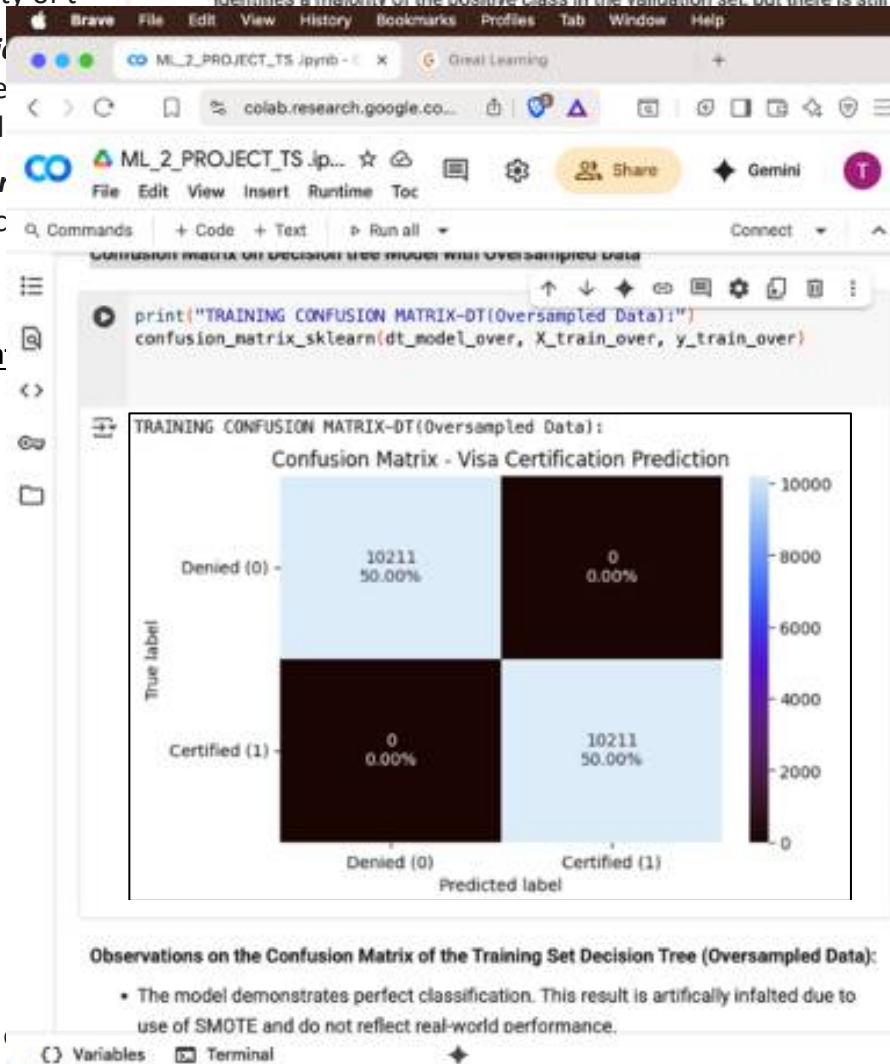
Validation Performance Observations for Decision Tree Model (Original Validation Data):

- **Accuracy (0.6581):** The validation set's accuracy is much lower than that of the training set, indicating that the model may not generalise effectively to new data, which is a common problem when overfitting occurs.
- **Recall (0.7337):** The recall score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.

Training Perform

- **Accuracy** correctly memorised
- **Recall (1.0000):** The recall score is perfect, indicating that the model correctly identifies all positive instances in the training data.
- **Precision**

Confusion Ma



Observations

- The model demonstrates perfect classification. This result is artificially inflated due to use of SMOTE and do not reflect real-world performance.
- **True Negatives (TN): 10,211** (all denied visas were correctly anticipated).
- **True Positives (TP): 10,211** (all certified visas were correctly predicted).
- **No false negatives (FN) or false positives (FP).**
- Accuracy, precision, recall, and F1-score are all 100% on the training data.
- This indicates that the model overfits substantially to the oversampled training data.

Validation Performance Obser

- **Accuracy (0.6581):** The validation set's accuracy is much lower than that of the training set, indicating that the model may not generalise effectively to new data, which is a common problem when overfitting occurs.
- **Recall (0.7337):** The recall score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.
- **Precision (0.7491):** The precision score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.
- **F1 Score (0.7413):** The F1 score is moderate, indicating that the model correctly identifies a majority of the positive class in the validation set, but there is still room for improvement.

Confusion Matrix

```
[ ] print("VALIDATION CONFUSION MATRIX-DT(Original Validation Data):")
confusion_matrix_sklearn(dt_model_over, X_val, y_val)
```

VALIDATION CONFUSION MATRIX-DT(Original Validation Data):



Observations on the Confusion Matrix:

- The model demonstrates high performance on the SMOTE and do not
- **True Negatives (TN):**

Observation

- Perf
- **True**
- **True Positiv**
- **False Nega** predicted a
- **False Positi**
- The mo with ba

Decision Tree Model Building using Oversampled Data

[] 11 cells hidden

6.2 Bagging

Model Building

```
rf_model_over = RandomForestClassifier(
    RandomForestClassifier(random_state=42)
```

Checking Performance of Random Forest-oversampled Model

```
[ ] # Training performance (on oversampled data)
print("Training Performance-RF(Oversampled Data):")
rf_train_perf_over = model_performance_classification_sklearn(rf_model_over, X_train_over, y_train_over)
rf_train_perf_over
```

Training Performance-RF(Oversampled Data):

	Accuracy	Recall	Precision	F1
0	0.999951	1.0	0.999902	0.999951

Random Forest Training Performance (oversampled data):

- **Accuracy: 0.99995** The model obtains an extremely high accuracy of roughly 99.995% on the training set, indicating great performance but causing overfitting.
- **Recall (1.0):** A perfect recall score of 1.0 means that the model correctly identified ev

Performance E

Random Forest Train

- **Accuracy: 0.9999**: The model's accuracy on the training set, indicating near-perfect performance.
- **Recall (1.0)**: The model's ability to correctly identify all positive instances (Certified) in the training set.
- **Precision (0.9999)**: The model's ability to correctly identify all positive instances (Certified) in the training set.
- **F1 Score (0.9999)**: The harmonic mean of precision and recall, indicating near-perfect performance.

```
high at around 99.99%, suggesting remarkable performance.
```

```
[ ] # Validation performance (on original validation data)
print("Validation Performance-RF (Original Validation Data):")
rf_val_perf_over = model_performance_classification_sklearn(rf_model_over,
rf_val_perf_over
```

	Accuracy	Recall	Precision	F1
0	0.717033	0.811637	0.775189	0.792995

Validation Performa

- **Accuracy (0.717033)**: The accuracy on the validation set is substantially lower, at around 71.70%, demonstrating that the model struggles to generalise to unseen data.
- **Recall (0.811637)**: The recall score of roughly 81.16% indicates that the model correctly detects a significant number of the positive cases ('Certified') in the validation set, but there is still room for improvement.
- **Precision (0.775189)**: The precision score of roughly 77.52% indicates that the model correctly identifies a significant number of the positive cases ('Certified') in the validation set, but there is still room for improvement.
- **F1 Score (0.792995)**: The F1 score of roughly 79.30% reveals a decent performance is not as strong as the training set performance.

Validation Performance of Random Forest (Original Data):

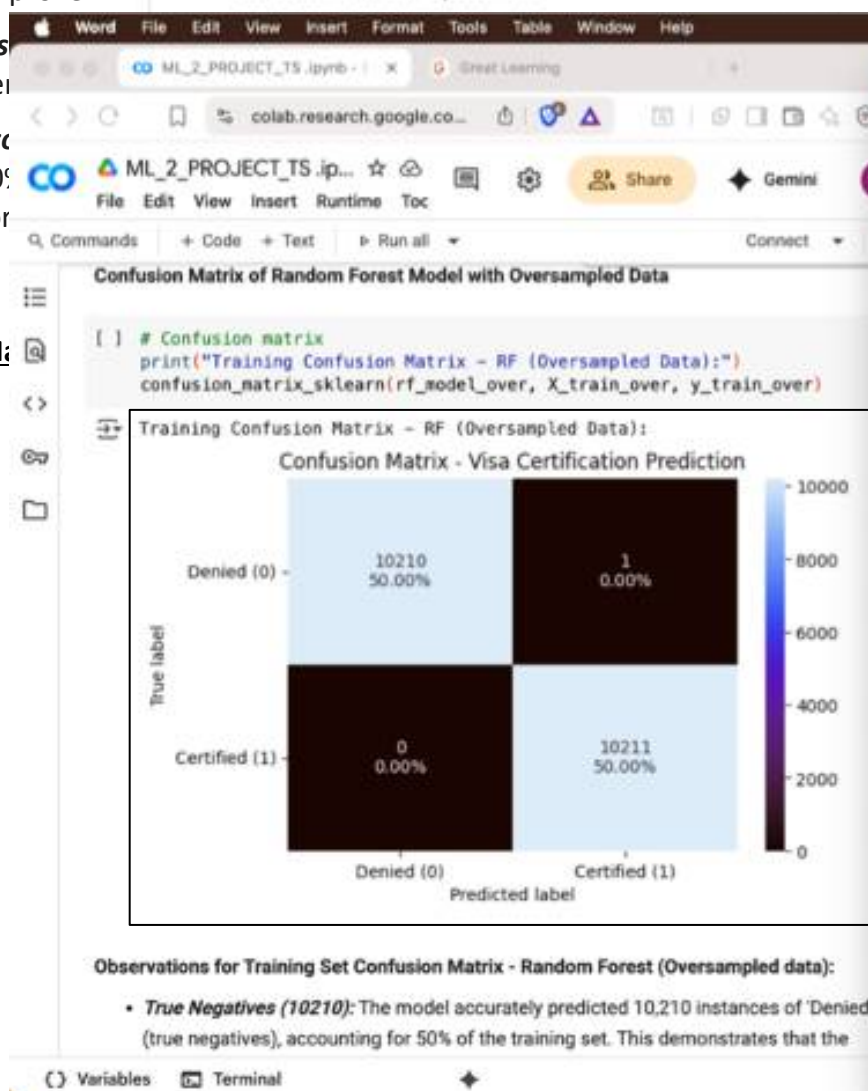
- **Accuracy (0.717033)**: The accuracy on the validation set is substantially lower, at around 71.70%, demonstrating that the model struggles to generalise to unseen data.
- **Recall (0.811637)**: The recall score of roughly 81.16% indicates that the model correctly detects a significant number of the positive cases ('Certified') in the validation set, but there is still room for improvement.

Random Forest Train

- **Accuracy: 0.9999**: The model's accuracy on the training set, indicating near-perfect performance.
- **Recall (1.0)**: The model's ability to correctly identify all positive instances (Certified) in the training set.
- **Precision (0.9999)**: The model's ability to correctly identify all positive instances (Certified) in the training set.

- **Precision (0.775189)**: The precision score of roughly 77.52% indicates that the model correctly identifies a significant number of the positive cases ('Certified') in the validation set, but there is still room for improvement.
- **F1 Score (0.792995)**: The F1 score of roughly 79.30% reveals a decent performance is not as strong as the training set performance.

Confusion Matrix



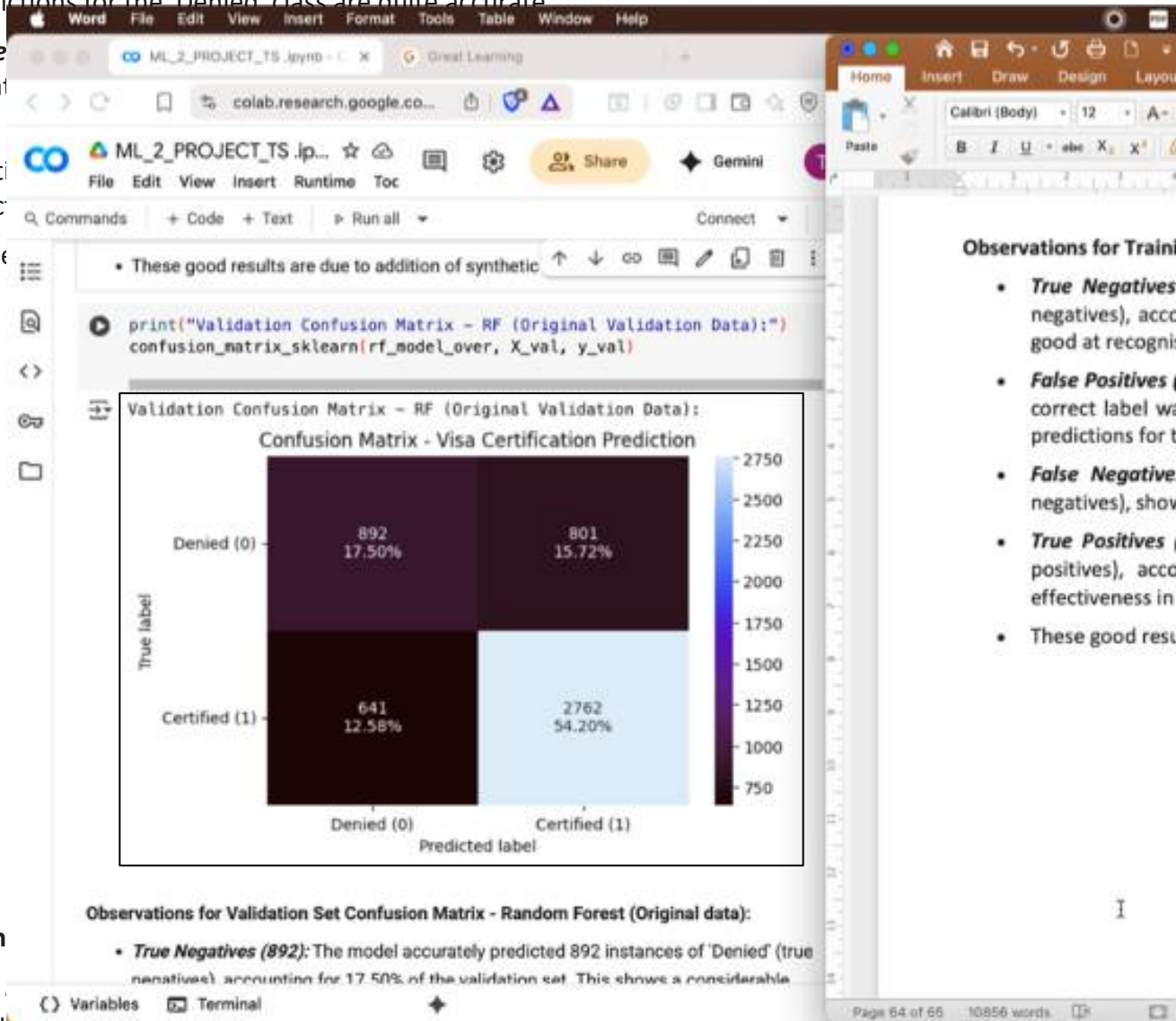
Validation Performance of Ra

- **Accuracy (0.717033)**: The accuracy on the validation set is substantially lower, at around 71.70%, demonstrating that the model struggles to generalise to unseen data.
- **Recall (0.811637)**: The recall score of roughly 81.16% indicates that the model correctly detects a significant number of the positive cases ('Certified') in the validation set, but there is still room for improvement.
- **Precision (0.775189)**: The precision score of roughly 77.52% indicates that the model correctly identifies a significant number of the positive cases ('Certified') in the validation set, but there is still room for improvement.
- **F1 Score (0.792995)**: The F1 score of roughly 79.30% reveals a decent performance is not as strong as the training set performance.

Confusion Matrix

Observations for Training Set Confusion Matrix - Random Forest (Oversampled data):

- **True Negatives (10210):** The model accurately predicted 10,210 instances of 'Denied' (true negatives), accounting for 50% of the training set. This demonstrates that the model is quite good at recognising the negative class in the training data.
- **False Positives (1):** There was just one instance when the model predicted 'Certified' when the correct label was 'Denied' (false positive). This is a very low rate, indicating that the model's predictions for the 'Denied' class are quite accurate.
- **False Negatives (641):** The model misclassified 641 instances of 'Certified' as 'Denied' (false negatives), or 12.58% of the validation set. This shows a significant decline in recall for the majority class compared to the training set.
- **True Positives (2,762):** The model correctly predicted 2,762 instances of 'Certified' (true positives), representing 54.20% of the validation set. While this is somewhat effective in identifying the majority class, it falls short of the training performance.
- These good results are due to addition of synthetic



Observation

- **True Negatives (892):** The model accurately predicted 892 instances of 'Denied' (true negatives), accounting for 17.50% of the validation set. This shows a considerable reduction in performance when compared to the training set.
- **False Positives (801):** The model misclassified 801 cases as 'Certified' when they were actually 'Denied' (false positives), accounting for 15.72% of the validation set. This shows that the model is unable to appropriately classify the majority of unseen data.
- **False Negatives (641):** The model misclassified 641 instances of 'Certified' as 'Denied' (false negatives), or 12.58% of the validation set. This shows a significant decline in recall for the majority class compared to the training set.
- **True Positives (2,762):** The model correctly predicted 2,762 instances of 'Certified' (true positives), representing 54.20% of the validation set. While this is somewhat effective in identifying the majority class, it falls short of the training performance.

Boosting Model Building using Oversampled Data

6.3 Boosting

6.3.1 AdaBoost

Model Building

- We have s

Performance Eva

Observations for

- **Accuracy** (0.7693): The model achieves an accuracy of ~76.93% on the training set. This shows that the model is reasonably good at identifying patterns in the training data.
- **Recall (0.764959)**: The recall score of about 76.50% indicates that the model correctly identifies a significant proportion of the positive cases ('Certified') in the training set. This is a good trait, indicating that the model is reasonably effective at detecting positive cases.
- **Precision (0.771685)**: The precision score of around 77.20% suggests that the model's positive predictions are reasonably dependable, with a high proportion of genuine positive cases.
- **F1 Score (0.768308)**: The F1 score of around 76.83% is the harmonic mean of precision and recall, showing that the model performs sufficiently but not as well as the Random Forest model. This shows that, while the model is useful, there is room for additional improvement.

Observations for

- **Accuracy** (0.699765): The validation set has an accuracy of about 69.98%, which is lower than the training accuracy. This implies that the model is having difficulty generalising to previously unseen data.

The screenshot displays a Google Colab notebook titled "ML_2_PROJECT_TS.ipynb". The notebook is open in a browser window with the URL "colab.research.google.co...". The notebook interface shows a menu bar with "File", "Edit", "View", "Insert", "Runtime", and "Toc". Below the menu bar, there are tabs for "Commands", "+ Code", "+ Text", and "Run all". The notebook content includes a code cell with the following code:

```
AdaBoostClassifier
AdaBoostClassifier(random_state=42)
```

The output of the code cell shows the training performance of the AdaBoost model on oversampled data:

	Accuracy	Recall	Precision	F1
0	0.769317	0.764959	0.771685	0.768308

The notebook also includes a section titled "Checking Performance of AdaBoost-Oversampled Model" with the following code:

```
# Training performance (on oversampled data)
print("Training Performance-AdaBoost (Oversampled Data):")
ada_train_perf_over = model_performance_classification_sklearn(ada_model_oversampled,
ada_train_perf_over)
```

The output of the code cell shows the training performance of the AdaBoost model on oversampled data:

	Accuracy	Recall	Precision	F1
0	0.769317	0.764959	0.771685	0.768308

The notebook also includes a section titled "Observations for Training Performance of AdaBoost (Oversampled Data):" with the following text:

- **Accuracy (0.7693)**: The model achieves an accuracy of ~76.93% on the training set.

The screenshot also shows a second Google Colab notebook titled "ML_2_PROJECT_TS.ipynb" with the following code:

```
# Validation performance (on original validation data)
print("Validation Performance-AdaBoost (Original Data):")
ada_val_perf_over = model_performance_classification_sklearn(ada_model_oversampled,
ada_val_perf_over)
```

The output of the code cell shows the validation performance of the AdaBoost model on original data:

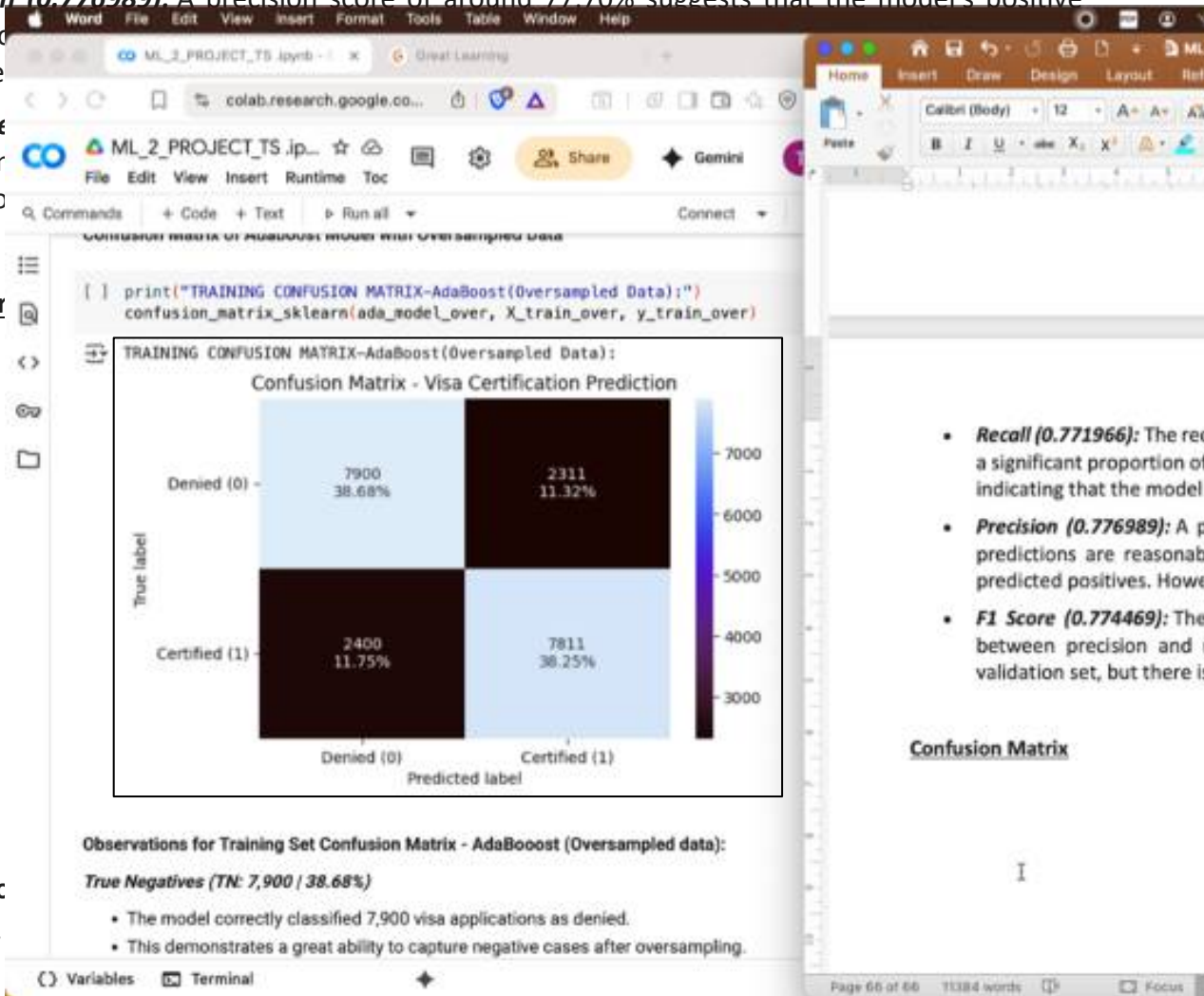
	Accuracy	Recall	Precision	F1
0	0.699765	0.771966	0.776989	0.774469

The notebook also includes a section titled "Observations for Validation Performance of AdaBoost (Original Data):" with the following text:

- **Accuracy (0.699765)**: The validation set has an accuracy of about 69.98%, which is lower than the training accuracy. This implies that the model is having difficulty generalising to previously unseen data.
- **Recall (0.771966)**: The recall score of about 77.20% indicates that the model correctly identifies a significant proportion of the positive cases ('Certified') in the validation set. This is a good trait, indicating that the model is reasonably effective at detecting positive cases.
- **Precision (0.776989)**: A precision score of around 77.70% suggests that the model's positive predictions are reasonably dependable, with a high proportion of genuine positive cases.

- **Recall (0.771966):** The recall score of about 77.20% indicates that the model correctly recognises a significant proportion of the positive cases ('Certified') in the validation set. This is a good trait, indicating that the model is reasonably effective at detecting positive cases.
- **Precision (0.776989):** A precision score of around 77.70% suggests that the model's positive predictions are reasonably accurate, with about 77.7% of predicted positives being correct.
- **F1 Score (0.774469):** The F1 score, which is the harmonic mean of precision and recall, is approximately 77.45%, indicating a balanced performance between precision and recall in the validation set, but there is still room for improvement.

Confusion Matrix



Observations for True Negatives

- The model correctly classified 7,900 visa applications as denied.
- This demonstrates a great ability to capture negative cases after oversampling.

False Positives (FP: 2,311 | 11.32%)

- These are applications that were actually denied but were incorrectly classified as certified.
- The misclassification rate of approximately 11% implies a risk of granting certification to denied cases.

False Negatives (FN: 2,400 | 11.75%)

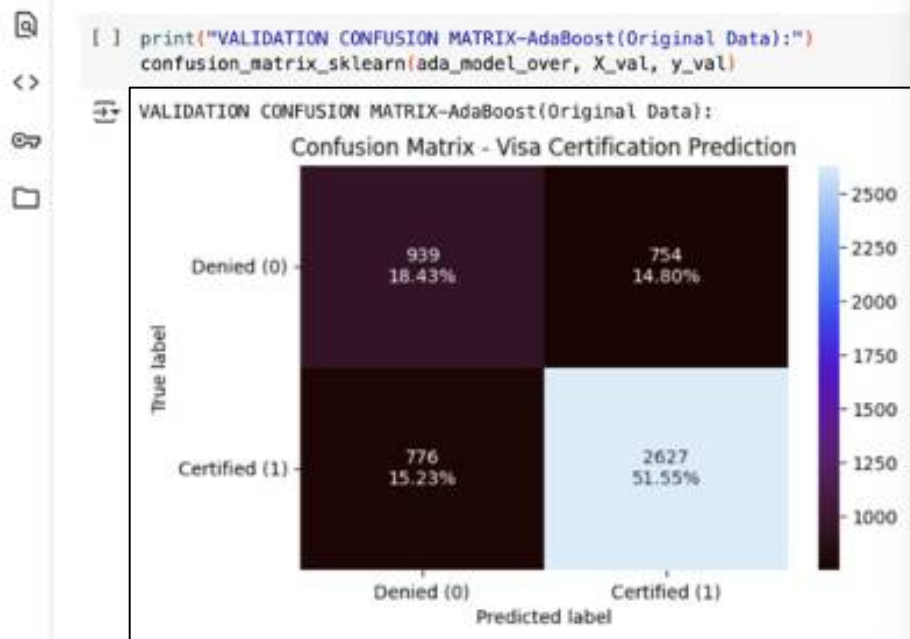
- These are applications that have actually been certified but are predicted to be denied.
- This represents missed opportunities for real approvals, but the error rate is moderate.

True Positives (TP: 7,811 | 38.25%)

- The model accurately identified 7,811 Certified apps.
- Balanced performance is seen, with about equal shares of correct predictions in both classes.

Thus, we can say oversampling effectively reduced the bias toward the majority class.

Thus, we can say oversampling effectively reduced the bias toward the majority class.



Observations for Validation Set Confusion Matrix - AdaBoost (Original data):

True Negatives: 939 (18.43%)

- Only 939 denied applications were accurately predicted.

Observatio

True Negai

- Only 939 denied applications were accurately predicted.
- Significantly lower than during training, indicating while oversampling improved minority class recognition, the model still performs weaker on denied cases in real-world (imbalanced) validation data.

False Positives (FP: 754; 14.80%)

- These are denied applications that have been wrongly labelled as certified.
- This may lead to approving undesirable candidates, which may have compliance issues.

False Negatives (FN: 776; 15.23%)

- Certified applications were misclassified as denied.
- Rejecting legitimate applications can lead to disappointment and a loss of good candidate.

True Positives (TP: 2,627; 51.55%)

- The majority of certified applications were accurately predicted.
- This indicates that the model is biased towards predicting certification.

Conclusions:

- There is a stronger bias towards certification predictions; denial predictions are weak.
- Indicates some degree of overfitting and difficulties owing to class imbalance.
- However, this does not mean that oversampling failed, but rather demonstrates the impact of applying a balanced-trained model to an uneven real-world distribution.

cases.

False Negatives (FN: 2

- These are appl
- This represents

True Positives (TP: 7,8

- The model acc
- Balanced perfo

Thus, we can say ov

6.3.2 Gradient

Model Building

- We have suc

Performance Evalu

Observations for Tr

- **Accuracy (0.801244)**: This indicates the model's performance on the oversampled data.
- **Recall (0.830967)**: A recall score of about 83.10% shows that the model correctly recognises around 83.10% of the positive cases ('Certified') in the training set. This
- **Precision (0.784341)**: A precision score of around 78.43% suggests that the model's positive predictions are reasonably dependable, with a high rate of actual positives among predicted positives. However, there are still some false positives.
- **F1 Score (0.806981)**: The F1 score of around 80.70% indicates that the model performs well overall. This score indicates that the model is effective at recognising positive cases while also retaining a reasonable level of accuracy in its predictions.

Observations for Valid

- **Accuracy (0.739992)**: The validation set has an accuracy of about 73.99%, which is lower than the training accuracy. This suggests that the model has difficulty generalising to previously unknown data.
- **Recall (0.835145)**: The recall score of about 83.51% indicates that the model is effective at identifying a significant number of positive cases ('Certified') from the validation set. This is an excellent indicator because it shows that the model can still detect positive cases even when using previously unseen data.

Gradient Boosting - Oversampled Data

Word File Edit View Insert Format Tools Table Window Help

ML_2_PROJECT_TS.ipynb

colab.research.google.co...

File Edit View Insert Runtime Toc

Commands + Code + Text Run all Connect

GradientBoostingClassifier(random_state=42)

Checking Performance of Gradient Boosting-Oversampled Model

```
[ ] # Training performance (on oversampled data)
print("Training Performance - GB (Oversampled Data):")
gb_train_perf_over = model_performance_classification_sklearn(gb_model_ov
gb_train_perf_over
```

	Accuracy	Recall	Precision	F1
0	0.801244	0.830967	0.784341	0.806981

Observations for Training Performance of Gradient Boosting (Oversampled Data):

- **Accuracy (0.801244)**: On the training set, the model achieved an accuracy of roughly 80.12%. This indicates strong performance, implying that the model is successfully capturing patterns in the oversampled data.
- **Recall (0.830967)**: A recall score of about 83.10% shows that the model correctly recognises around 83.10% of the positive cases ('Certified') in the training set. This
- **Precision (0.784341)**: A precision score of around 78.43% suggests that the model's positive predictions are reasonably dependable, with a high rate of actual positives among predicted positives. However, there are still some false positives.
- **F1 Score (0.806981)**: The F1 score of around 80.70% indicates that the model performs well overall. This score indicates that the model is effective at recognising positive cases while also retaining a reasonable level of accuracy in its predictions.

```
[ ] # Validation performance (original validation data)
print("Validation Performance - GB (Original Data):")
gb_val_perf_over = model_performance_classification_sklearn(gb_model_ov
gb_val_perf_over
```

	Accuracy	Recall	Precision	F1
0	0.739992	0.835145	0.788131	0.810957

Observations for Validation Performance of Gradient Boosting (Original Data):

- **Accuracy (0.739992)**: The validation set has an accuracy of about 73.99%, which is lower than the training accuracy. This suggests that the model has difficulty generalising to previously unknown data.
- **Recall (0.835145)**: The recall score of about 83.51% indicates that the model is effective at identifying a significant number of positive cases ('Certified') from the validation set. This is an excellent indicator because it shows that the model can still detect positive cases even when using previously unseen data.
- **Precision (0.788131)**: A precision score of around 78.81% suggests that the model's positive predictions are reasonably dependable, with a high rate of actual positives among predicted positives. However, there are still some false positives.

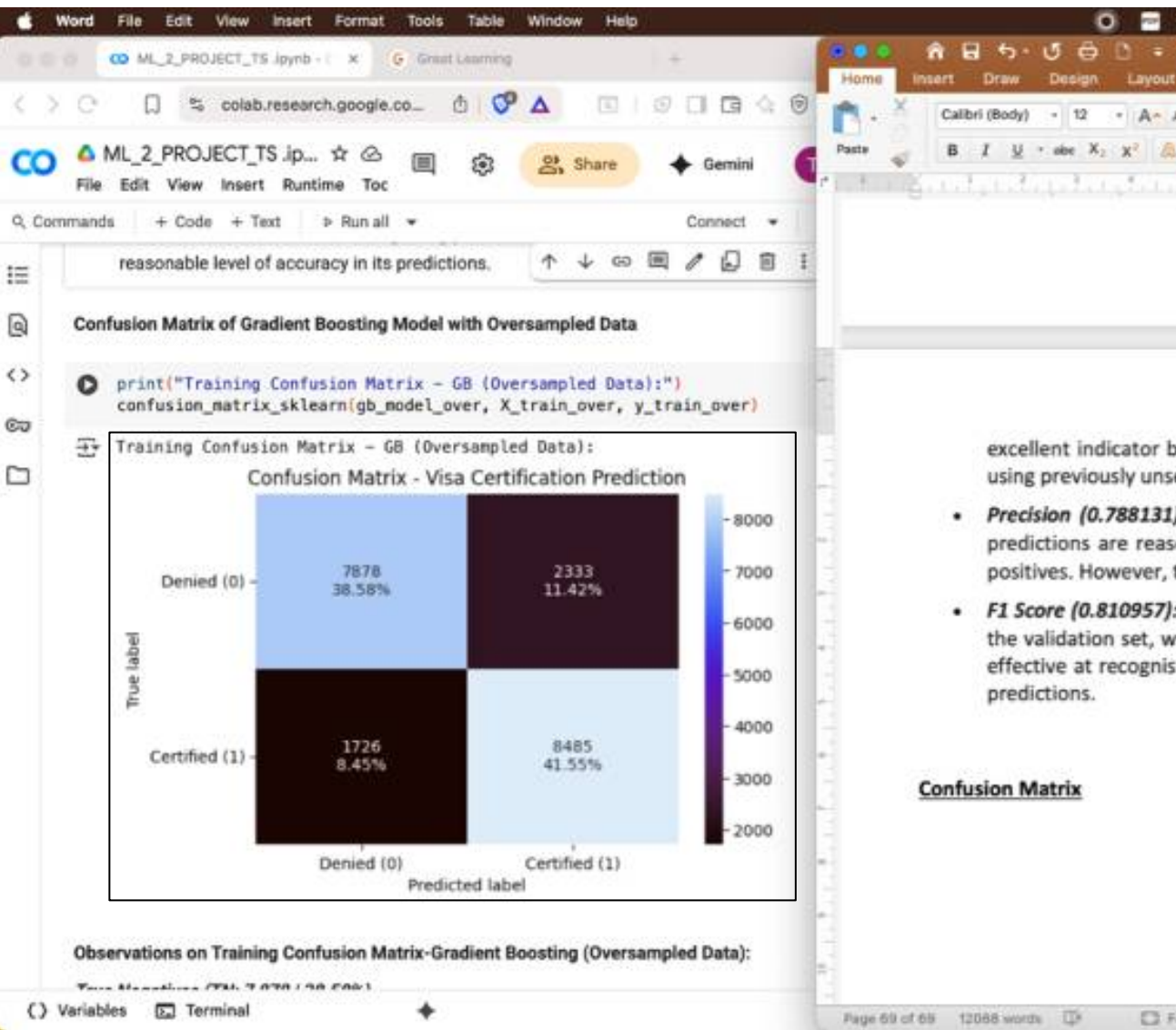
Variables Terminal

Page 68 of 69 11908 wo

excellent indicator because it shows that the model can still detect positive cases even when using previously unseen data.

- **Precision**: predictions are positive.
- **F1 Score**: the validation set, effective predictions.

Confusion Matrix



Observation:

True Negatives

- The model correctly classified 7,878 applications as denied.
- Strong performance on negative cases, indicating oversampling, aided the model in learning minority tendencies.

False Positives (FP: 2,333 | 11.42%)

- Applications that were actually denied but incorrectly predicted as Certified.
- Represents the business risk of mistakenly giving certification.
- However, the error rate is moderate and manageable when compared to the improvements in actual positive predictions.

False Negatives (FN: 1,726 | 8.45%)

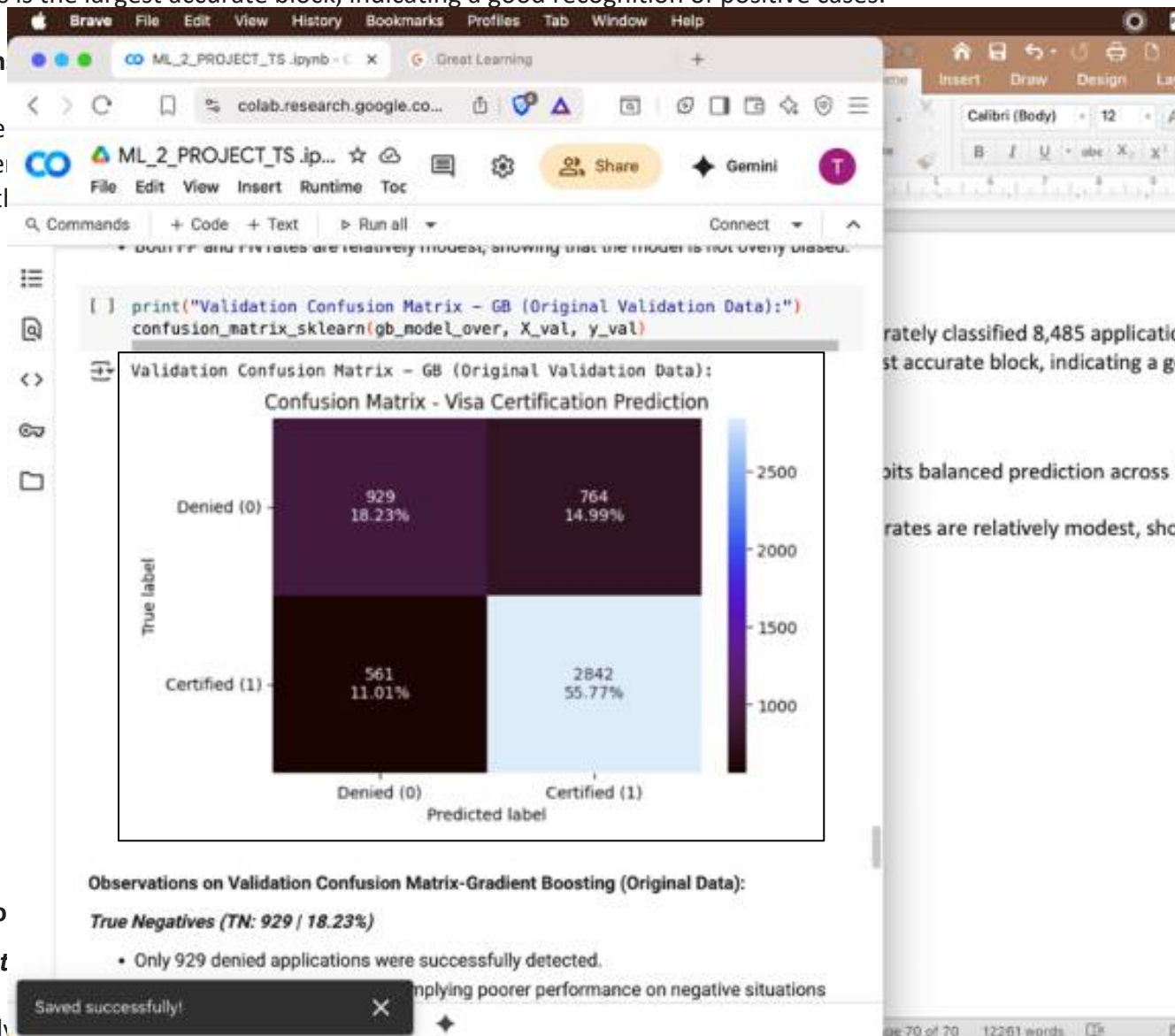
- Applications that were truly certified but reported as denied.
- This mistake rate is lower than in previous models (such as AdaBoost), indicating improved minority recall.

True Positives (8,485 | 41.55%)

- The model accurately classified 8,485 applications as Certified.
- This is the largest accurate block, indicating a good recognition of positive cases.

Conclusion

- The
- over
- Botl



Observatio

True Negat

- Only
- This is much lower than training, implying poorer performance on negative situations when applied to real-world distribution.
- Still, the model outperforms models trained without oversampling.

False Positives (FP: 764 | 14.99%)

- Applications were denied due to being misclassified as Certified.
- This creates a real-world risk of approving ineligible candidates.

False Negatives (FN: 561 | 11.01%)

- Certified applications were wrongly rejected as Denied.
- Although there are fewer lost opportunities than with FP.

True Positives (TP: 2,842 | 55.77%)

- The majority of Certified applications were predicted accurately.
- The model aligns more strongly with the Certified class, which accounts for around 67% of the validation data distribution.

Conclusions:

- The
- This
- This

6.3.3 X

Model Build

- We

Performance

Observations on Train

- **Accuracy (0.878611)**: The model obtains an accuracy of around 87.86%, suggesting excellent performance on the training set.
- **Recall (0.906473)**: A recall score of approximately 90.65% indicates that the model correctly recognises a large majority of positive examples ('Certified').
- **Precision (0.858627)**: The precision score of about 85.86% shows that the majority of forecasts are actual positives, with some false positives included.
- **F1 Score (0.881902)**: The F1 score, which is the harmonic mean of precision and recall, is approximately 88.19%.

The screenshot displays a Google Colab notebook titled 'ML_2_PROJECT_TS.ipynb'. The notebook is open in a web browser, showing the 'Code' tab. The code defines an XGBoost model with various parameters and evaluates its performance on oversampled data. The output shows the training performance metrics: Accuracy (0.878611), Recall (0.906473), Precision (0.858627), and F1 (0.881902).

```
feature_types=None, feature_weights=None, gamma=None,
grow_policy=None, importance_type=None,
interaction_constraints=None, learning_rate=None, max_bin=None,
max_cat_threshold=None, max_cat_to_onehot=None,
max_delta_step=None, max_depth=None, max_leaves=None,
min_child_weight=None, missing=None, monotone_constraints=None,
multi_strategy=None, n_estimators=None, n_jobs=None,
num_parallel_tree=None, ...)
```

Checking Performance of XGBoost-Oversampled Model

```
[ ] # Training performance on oversampled data
print("TRAINING PERFORMANCE - XGB (Oversampled Data):")
xgb_train_perf_over = model_performance_classification_sklearn(xgb_model,
xgb_train_perf_over
```

	Accuracy	Recall	Precision	F1
0	0.878611	0.906473	0.858627	0.881902

Observations on Training Performance of XGBoost (Oversampled Data):

- **Accuracy (0.878611)**: The model obtains an accuracy of around 87.86%, suggesting excellent performance on the training set.
- **Recall (0.906473)**: A recall score of approximately 90.65% indicates that the model correctly recognises a large majority of positive examples ('Certified').
- **Precision (0.858627)**: The precision score of about 85.86% shows that the majority of forecasts are actual positives, with some false positives included.

```
print("VALIDATION PERFORMANCE - XGB (Original Validation Data):")
xgb_val_perf_over = model_performance_classification_sklearn(xgb_model_over, X_val, y_val)
xgb_val_perf_over
```

	Accuracy	Recall	Precision	F1
0	0.72292	0.827799	0.773264	0.799803

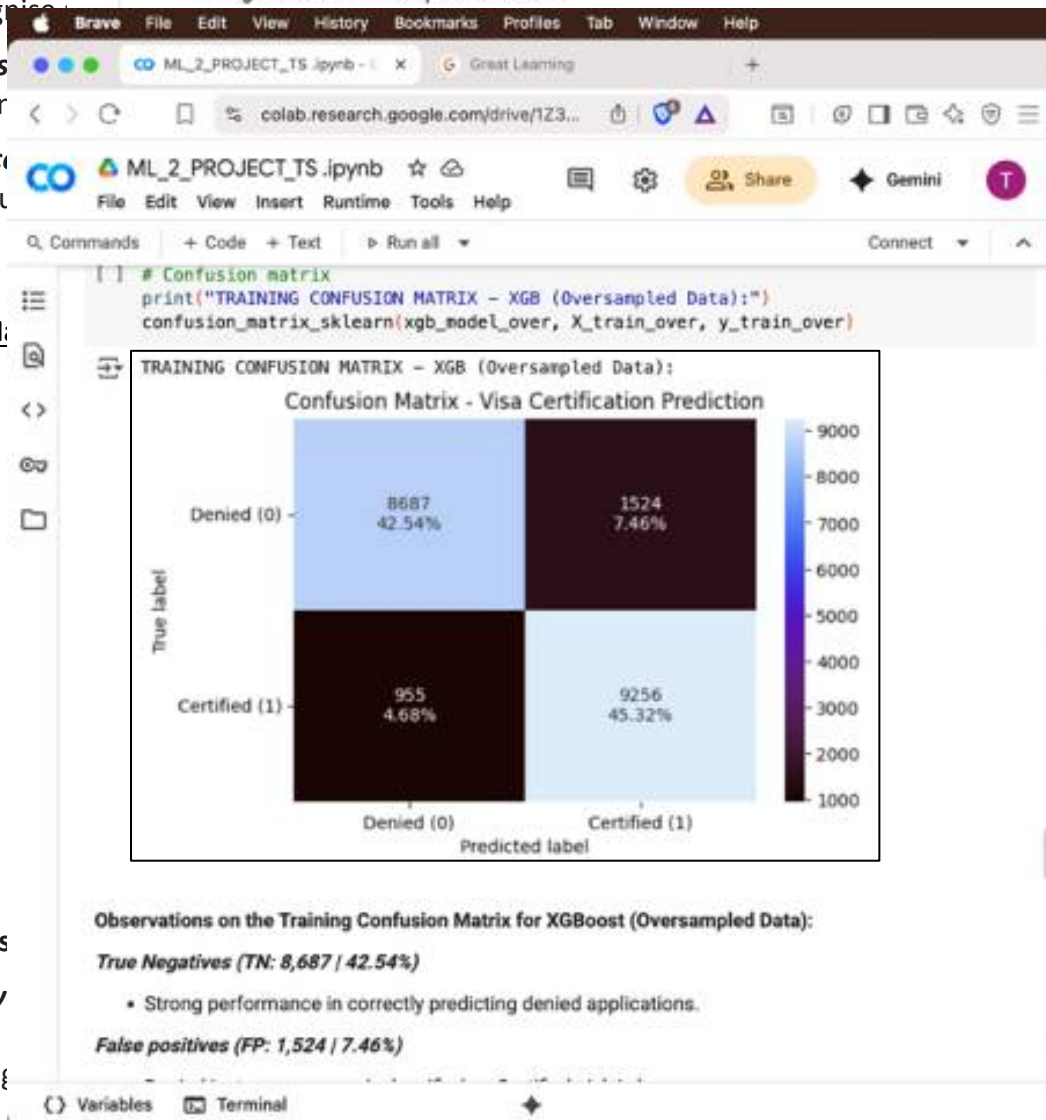
Observations on t

- **Accuracy** (performance)
- **Recall** (0.827799) recognises
- **Precision** (0.773264) depend
- **F1 Score** (0.799803) althou

Observations on the Validation Performance of XGBoost (Original Validation Data):

- **Accuracy (0.72292):** The accuracy on the validation set is roughly 72.29%, showing a decrease in performance when compared to the training set.
- **Recall (0.827799):** A recall score of around 82.78% indicates that the model continues to recognise a significant number of positive cases.

Confusion Ma



Observations

True Negative

- Strong

False positives (FP: 1,524 | 7.46%)

- Denied instances were misclassified as Certified; risk is low.

False Negatives (FN: 955; 4.68%)

- Certified applications are wrongly predicted as denied; the error rate is low.

True Positives (TP: 9,256 | 45.32%)

Observations

- **Accuracy** (performance)
- **Recall** (0.827799) recognises
- **Precision** (0.773264) depend

Observations on the Va

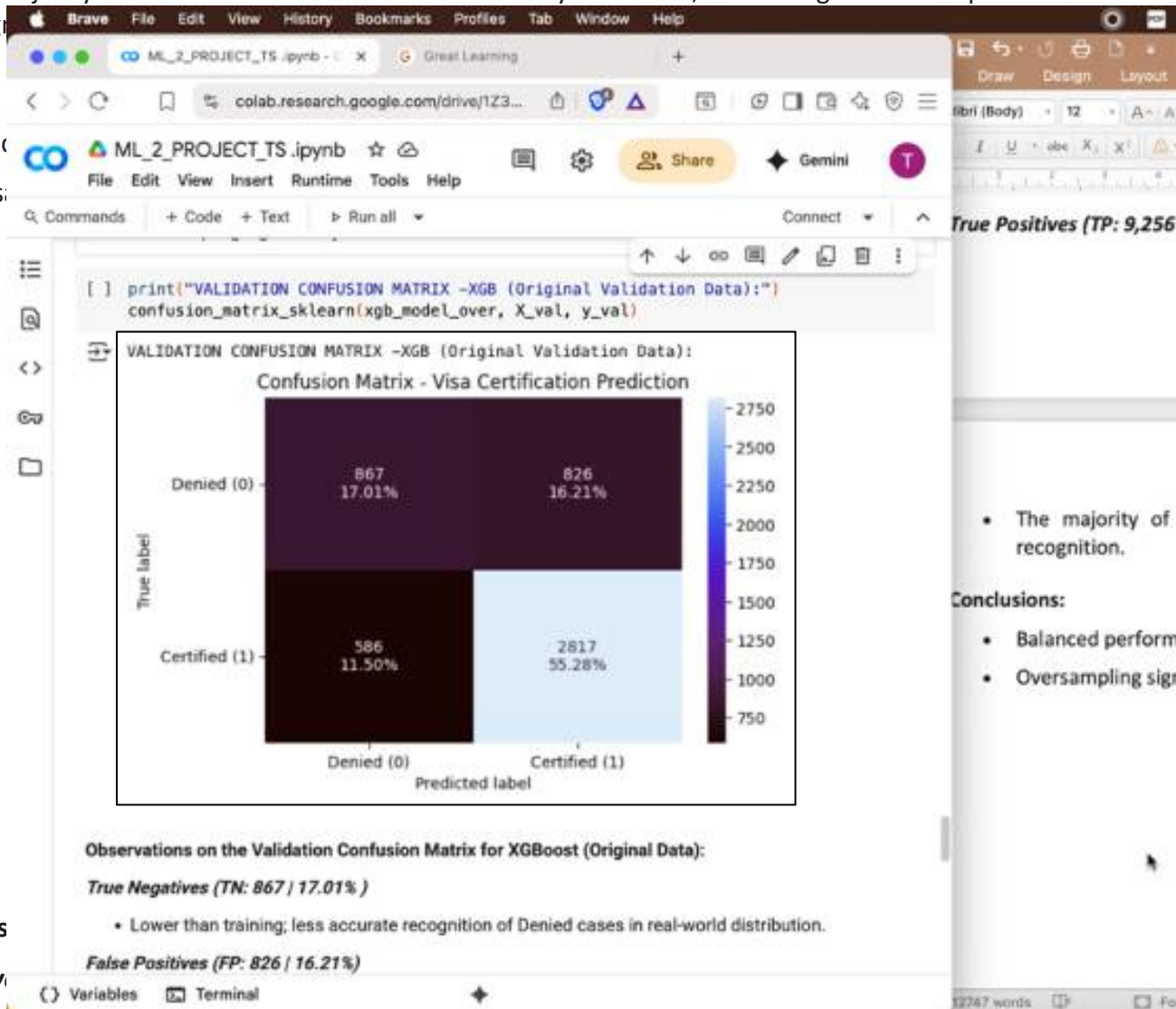
- **Accuracy** (0.72292) performance wh
- **Recall** (0.827799) recognise a signi
- **Precision** (0.773264) dependable, but
- **F1 Score** (0.799803) although lower t

Confusion Matrix

- The majority of certified instances were correctly classified, indicating excellent positive recognition.

Conclusions:

- Balanced performance
- Oversampling significantly improved



Observations

True Negatives

- Lower than training; less accurate recognition of Denied cases in real-world distribution.

False Positives (FP: 826 | 16.21%)

- Denied applications were incorrectly predicted as Certified, posing a significant business risk.

False Negatives (FN: 586 | 11.50%)

- Certified cases predicted as denied; moderate error but lower impact than FP.

True Positives (TP: 2,817 | 55.28%)

- Strong recognition of Certified apps; matches with class distribution during validation.

Conclusions:

- The model exhibits a bias towards Certification predictions, while Denied predictions are weaker.

7 MODEL BUILD

Undersampling is a method of reducing the number of instances in the majority class to improve the model's ability to learn from the minority class, resulting in better performance. We will use the `RandomUnderSampler` from the `imblearn` library.

Class distribution before undersampling

Observations:

- The dataset is biased towards the majority class ('Certified').
- This **imbalance** may result in a model that is biased towards the majority class, risking its ability to accurately predict the minority class ('Denied').
- Undersampling can help to address this issue by reducing the number of cases in the majority class, resulting in a more balanced dataset.

Applying RandomUnderSampler

We applied `RandomUnderSampler` with `random_state = 42` to the training data.

Note: To ensure that the undersampling is performed on valid data, we will use the `train_test_split` function to split the data into training and testing sets before applying the undersampling technique.

Class distribution after undersampling

Observations:

- Class distribution is balanced after applying undersampling technique.
- Both 'Denied' & 'Certified' class is now at 50% - 50%
- The shape of `X_train` is (10154, 21) & `y_train` is (10154,)

```
resulting in better predicted performance on minority class instance
RandomUnderSampler to implement this strategy.

Checking class distribution before undersampling

[ ] print("Before Undersampling:")
    print("Counts of label 'Denied': {}".format(sum(y_train == 0)))
    print("Counts of label 'Certified': {}".format(sum(y_train == 1)))
    print("Proportions: {:.2f}% Denied, {:.2f}% Certified\n".format(
        sum(y_train == 0)/len(y_train)*100,
        sum(y_train == 1)/len(y_train)*100
    ))

Before Undersampling:
Counts of label 'Denied': 5077
Counts of label 'Certified': 10211
Proportions: 33.21% Denied, 66.79% Certified

Observations:

• The dataset is biased, with just 33.21% labelled as 'Denied' and 66.79% as 'Certified'.
• This imbalance may result in a model that is biased towards the majority class ('Certified')
  its ability to accurately predict the minority class ('Denied').
• Undersampling can help to address this issue by reducing the number of cases in the majority
  class, resulting in a more balanced dataset.
```

```
Word File Edit View Insert Format Tools Table Window Help

CO ML_2_PROJECT_TS.ipynb - x G Great Learning

colab.research.google.com/drive/1Z3Z1QF...

CO ML_2_PROJECT_TS.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all RAM Disk

sum(y_train_under == 1)/len(y_train_under)*100
))

print("After Undersampling, shape of X_train: {}".format(X_train_under.shape))
print("After Undersampling, shape of y_train: {}".format(y_train_under.shape))

After Undersampling:
Counts of label 'Denied': 5077
Counts of label 'Certified': 5077
Proportions: 50.00% Denied, 50.00% Certified

After Undersampling, shape of X_train: (10154, 21)
After Undersampling, shape of y_train: (10154,)

Observations:

• Class distribution is balanced after applying undersampling technique.
• Both 'Denied' & 'Certified' class is now at 50% - 50%
• The shape of X_train is (10154, 21) & y_train is (10154,).

> Decision Tree - Undersampled Data

[ ] 11 cells hidden
```

7.1 Decision Tree

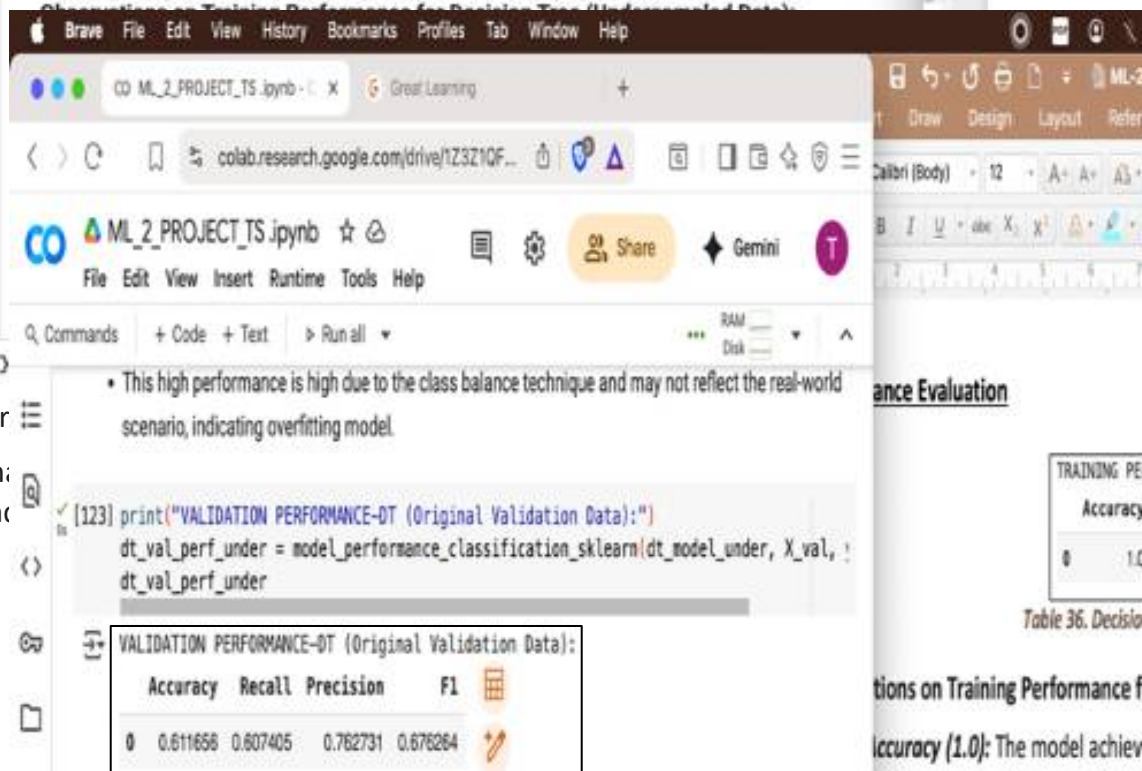
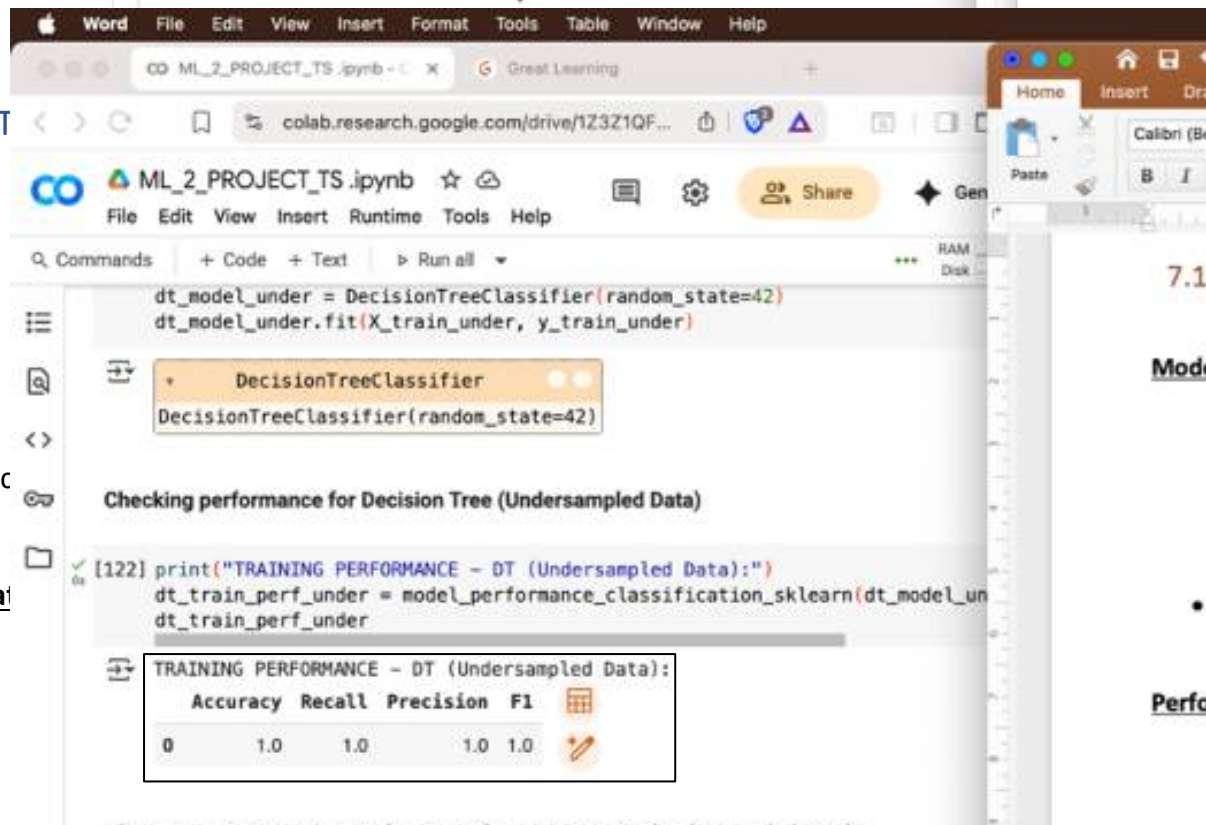
Model Building

- We have successfully

Performance Evaluation

Observations on Training Performance

- Accuracy (1.0)**: The model achieved 100% accuracy on the training data.
- Recall (1.0)**: The model achieved 100% recall, meaning it correctly identified all positive instances ('Certified').
- Precision (1.0)**: The model achieved 100% precision, meaning all predicted positive instances were true positives.
- F1 Score (1.0)**: The F1 score is 1.0, indicating a perfect balance of precision and recall.
- This high performance is high due to the class balance technique and may not reflect the real-world scenario, indicating overfitting model.



Observations on Validation Performance

- Accuracy (0.611656)**: The validation set has an accuracy of roughly 61.17%, showing a considerable decrease in performance when compared to the training set.
- Recall (0.607405)**: A recall score of around 60.74% indicates that the model only recognises a subset of positive events ('Certified').
- Precision (0.762731)**: A precision score of roughly 76.27% suggests that a significant majority of positive predictions are true positives, but there are also false positives.
- F1 Score (0.676264)**: An F1 score of approximately 67.63% indicates a moderate balance of precision and recall, but it is significantly lower than the training F1 score.

Observations on Validation Performance for Decision Tree (Original Validation Data):

- Accuracy (0.611656)**: The validation set has an accuracy of roughly 61.17%, showing a considerable decrease in performance when compared to the training set.
- Recall (0.607405)**: A recall score of around 60.74% indicates that the model only recognises a subset of positive events ('Certified').
- Precision (0.762731)**: A precision score of roughly 76.27% suggests that a significant majority of positive predictions are true positives, but there are also false positives.
- F1 Score (0.676264)**: An F1 score of approximately 67.63% indicates a moderate balance of precision and recall, but it is significantly lower than the training F1 score.

Conclusions:

- When compared to its high performance metrics on the training set, the model exhibits a

Performance Evaluation

TRAINING PERFORMANCE - DT (Undersampled Data):

	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0

Table 36. Decision Tree Performance

Observations on Training Performance

Accuracy (1.0): The model achieved 100% accuracy on the training data, indicating that all training data has been classified correctly.

Recall (1.0): A 100% recall score indicates that the model has correctly identified all positive instances ('Certified').

Precision (1.0): A precision score of 1.0 indicates that all predicted positive instances are true positives, with no false positives.

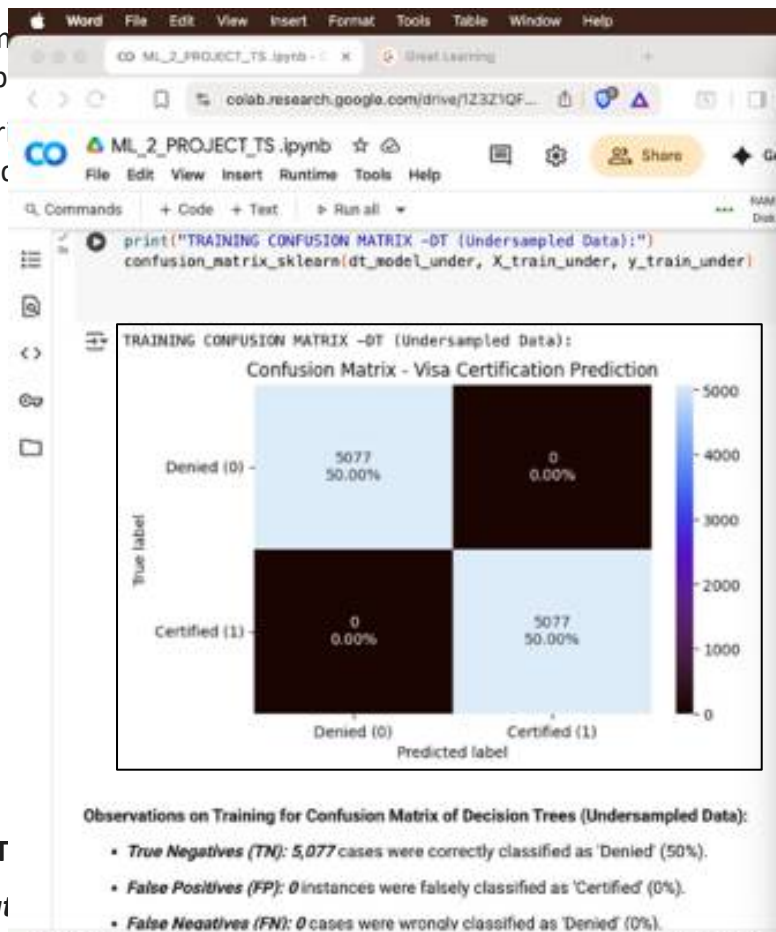
F1 Score (1.0): A score of 1.0 indicates a perfect balance of precision and recall, reflecting remarkable performance on the training data.

This high performance is high due to the class balance technique and may not reflect the real-world scenario, indicating overfitting.

Conclusions:

- When compared to its high precision and recall, but it is a considerable reduction.
- This disparity indicates that the training data, most likely due to overfitting, is not representative of the validation data.

Confusion Matrix

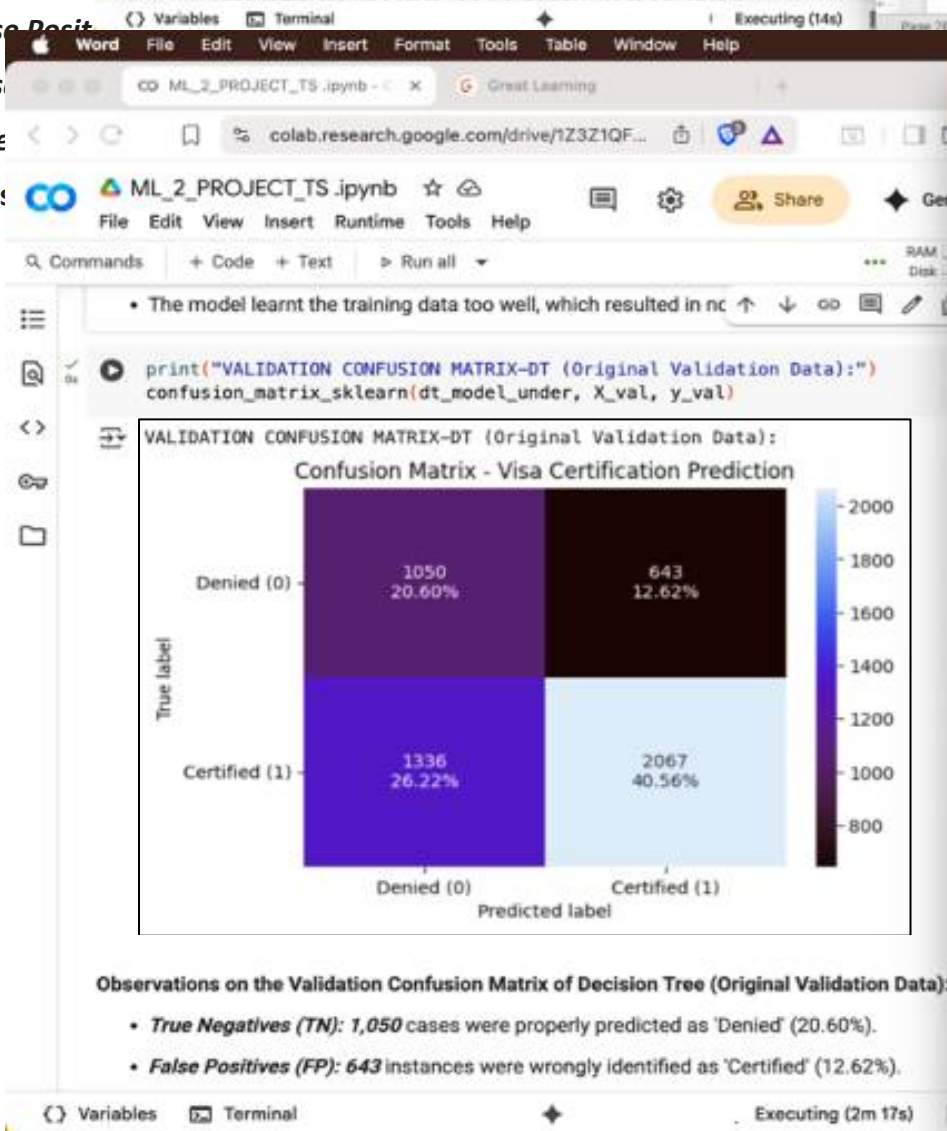


Observations on T

- True Negatives
- False Positives
- False Negatives
- True Positives

Conclusion:

- The model learnt the training data too well, which resulted in no generalization to the validation data.
- The model's performance on the validation data is poor, indicating overfitting.



subset of positive events ('Ce

- Precision (0.762731): A precision of 0.762731 indicates that 76.27% of the positive predictions are true.

- F1 Score (0.676264): An F1 score of 0.676264 indicates a balance between precision and recall, but it is still a considerable reduction.

Conclusions:

- When compared to its high precision and recall, but it is a considerable reduction.
- This disparity indicates that the training data, most likely due to overfitting, is not representative of the validation data.

Confusion Matrix

Observations on Tra

- True Negatives
- False Positives
- False Negatives
- True Positives

Conclusions:

- The Decision Tree model's performance on the validation data is poor, indicating overfitting.
- The model's performance on the validation data is poor, indicating overfitting.

Observations on the Validation Confusion Matrix of Decision Tree (Original Validation Data):

- **True Negatives (TN): 1,050** cases were properly predicted as 'Denied' (20.60%).
- **False Positives (FP): 643** instances were wrongly identified as 'Certified' (12.62%).
- **False Negatives (FN): 1,336** instances mistakenly classified as 'Denied' (26.22%).
- **True Positives (TP): 2,067** instances were accurately identified as 'Certified' (40.56%).

Conclusion:

- The model's performance is poor on the validation data, with a high false negative rate.
- This demonstrates the importance of proper model evaluation and data augmentation.
- The training model's performance is high, but it fails to generalize to new data.

The screenshot displays a Jupyter Notebook interface with the following content:

Building Random Forest Model (Undersampled Data)

```
[126] rf_model_under = RandomForestClassifier(random_state=42)
      rf_model_under.fit(X_train_under, y_train_under)
```

A variable inspector shows the `RandomForestClassifier` object with `random_state=42`.

Checking performance for Random Forest (Undersampled Data)

```
[127] print("TRAINING PERFORMANCE-RF(Undersampled Data):")
      rf_train_perf_under = model_performance_classification_sklearn(rf_model_under, X_train_under, y_train_under)
      rf_train_perf_under
```

The output is a table showing perfect training performance:

	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0

Observations on Random Forest Training Performance (Undersampled Data):

- **Accuracy (1.0):** The model has a high accuracy of 100% on the training set.
- **Recall (1.0):** A 100% recall score means that all positive cases ('Certified') were accurately identified.
- **Precision (1.0):** A score of 100% indicates that all positive predictions are true positives.

The bottom of the notebook shows a terminal window with the command `!pip install -U tensorflow` and the output `Requirement already satisfied: tensorflow in /usr/local/lib/python3.10/dist-packages (2.15.0)`.

7.2 Random Forest

Model Building

- We have

Performance Evaluation

Observations on

- **Accuracy**
- **Recall (1.0)** identified
- **Precision**
- **F1 score**

Conclusions:

- The Random Forest model performs perfectly on undersampled training data, suggesting overfitting.

- The perfect s and specific p

Observations on Val

- **Accuracy (0.684851)**: The validation set has an accuracy of about 68.49%, indicating m performance.
- **Recall (0.683808)**: A recall score of around 68.38% indicates that the model correctly de substantial proportion of positive events ('Certified').
- **Precision (0.814491)**: A precision score of around 81.45% implies that the majority of p predictions are true positives.
- **F1 Score (0.74345)**: An F1 score of approximately 74.35% indicates a healthy balance o and recall.

Conclusions:

- The l declin
- While is not
- This p unkn

Confusion M

Observation

- **True**

colab.research.google.com/drive/1Z3Z1QF...

ML_2_PROJECT_TS.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

rf_val_perf_under

VALIDATION PERFORMANCE-RF (Original Validation Data):

	Accuracy	Recall	Precision	F1
0	0.684851	0.683808	0.814491	0.74345

Observations on Validation Performance of Random Forest (Original Validation Data):

- **Accuracy (0.684851)**: The validation set has an accuracy of about 68.49%, indicating m performance.
- **Recall (0.683808)**: A recall score of around 68.38% indicates that the model correctly de substantial proportion of positive events ('Certified').
- **Precision (0.814491)**: A precision score of around 81.45% implies that the majority of p predictions are true positives.
- **F1 Score (0.74345)**: An F1 score of approximately 74.35% indicates a healthy balance o and recall.

Conclusions:

Word File Edit View Insert Format Tools Table Window Help

ML_2_PROJECT_TS.ipynb

colab.research.google.com/drive/1Z3Z1QF...

Commands + Code + Text Run all

Confusion Matrix for Random Forest (Undersampled Data)

```
print("TRAINING CONFUSION MATRIX-RF (Undersampled Data):")
confusion_matrix_sklearn(rf_model_under, X_train_under, y_train_under)
```

TRAINING CONFUSION MATRIX-RF (Undersampled Data):

Confusion Matrix - Visa Certification Prediction

True label \ Predicted label	Denied (0)	Certified (1)
Denied (0)	5077 50.00%	0 0.00%
Certified (1)	0 0.00%	5077 50.00%

Observations on Training for Confusion Matrix of Random Forest (Undersampled Data):

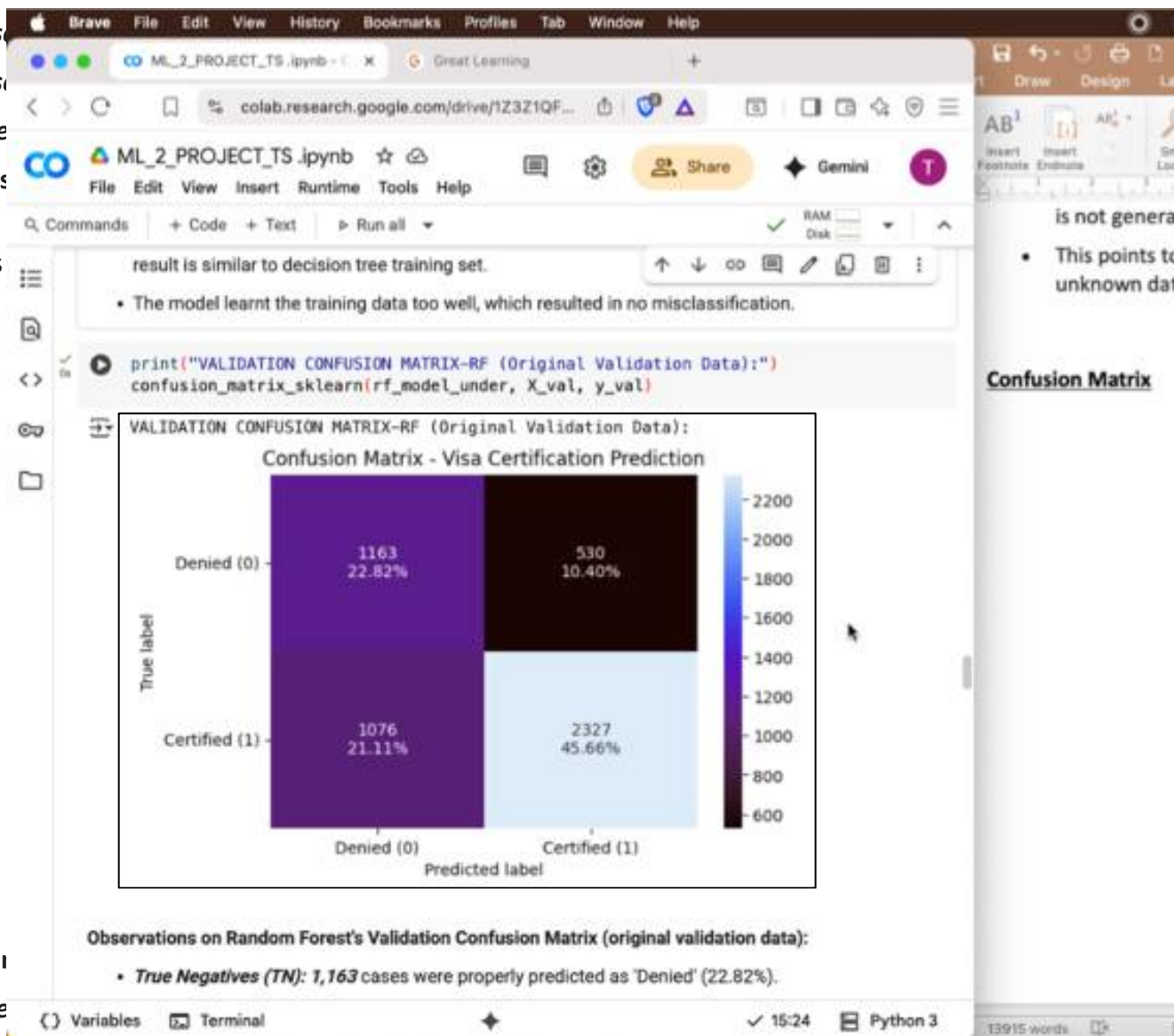
- **True Negatives (TN):** 5,077 cases were correctly classified as 'Denied' (50%).

Variables Terminal 15:24 Page 78 of 78 13908 words

- False
- False
- True

Conclusions

- The
- This
- The



Observation

- True
- **False Positives (FP): 530** occurrences were wrongly identified as 'Certified' (10.40%).
- **False Negatives (FN): 1,076** cases were mistakenly classified as 'Denied' (21.11%).
- **True Positives (TP): 2,327** instances were accurately classified as 'Certified' (45.66%).

Conclusions:

- The validation set had 1,076 false negatives and 530 false positives, indicating a significant number of misclassifications by the model.
- This implies that the model does not generalise effectively to prior unseen data, proving that it is most likely overfitting.
- The validation set performs significantly worse than the training set, emphasising the need for improved model robustness and generalisation.

7.3 Boosting Models - UnderSampled Data

7.3.1 AdaBoost Model – UnderSampled Data

Model Building

- We have suc

Performance Evalua

Observations on Ad

- **Accuracy (0.690762)**: On the training set, the model achieves an average accuracy of 69.08%.
- **Recall (0.741383)**: A recall score of around 74.14% shows that the model correctly detects a substantial p
- **Precision (0.673225)**: A precision score of around 67.32% implies that a large proportion of positive predictions are true positives.
- **F1 Score (0.705662)**: An F1 score of approximately 70.57% indicates a solid balance of precision and recall.

Conclusion:

- The AdaBoost model performs well on the validation set, with higher accuracy and F1 score compared to the training set.
- The metrics show that the model recognises patterns in the data without overfitting, implying a high level of generalisation.

Observations for Validat

- **Accuracy (0.709969)**: The validation set has an accuracy of about 70.99%, which is slightly higher than the training set.
- **Recall (0.750808)**: A recall score of approximately 75.08% indicates that the model correctly recognises a significant number of positive cases.
- **Precision (0.802198)**: A precision score of around 80.22% implies that a large proportion of positive predictions are true positives.
- **F1 Score (0.775653)**: An F1 score of approximately 77.57% indicates a solid balance of precision and recall.

Conclusions:

- The AdaBoost model outperforms on the validation set, with higher accuracy and F1 score compared to the training set.

AdaBoost - Undersampled Data

```
[131] ada_model_under = AdaBoostClassifier(random_state=42)
ada_model_under.fit(X_train_under, y_train_under)
```

Checking performance for AdaBoost Model (Undersampled Data)

```
[132] print("TRAINING PERFORMANCE-AdaBoost(Undersampled Data):")
ada_train_perf_under = model_performance_classification_sklearn(ada_model_under, X_train_under, y_train_under)
```

	Accuracy	Recall	Precision	F1
0	0.690762	0.741383	0.673225	0.705662

Observations on AdaBoost Training Performance (Undersampled Data):

- **Accuracy (0.690762)**: On the training set, the model achieves an average accuracy of 69.08%.
- **Recall (0.741383)**: A recall score of around 74.14% shows that the model correctly detects a

Conclusion:

- The AdaBoost model performs well on the validation set, with higher accuracy and F1 score compared to the training set.
- The metrics show that the model recognises patterns in the data without overfitting, implying a high level of generalisation.

```
[133] print("VALIDATION PERFORMANCE-AdaBoost (Original Validation Data):")
ada_val_perf_under = model_performance_classification_sklearn(ada_model_under, X_train_under, y_train_under)
```

	Accuracy	Recall	Precision	F1
0	0.709969	0.750808	0.802198	0.775653

Observations for Validation Performance of AdaBoost (Original Validation Data):

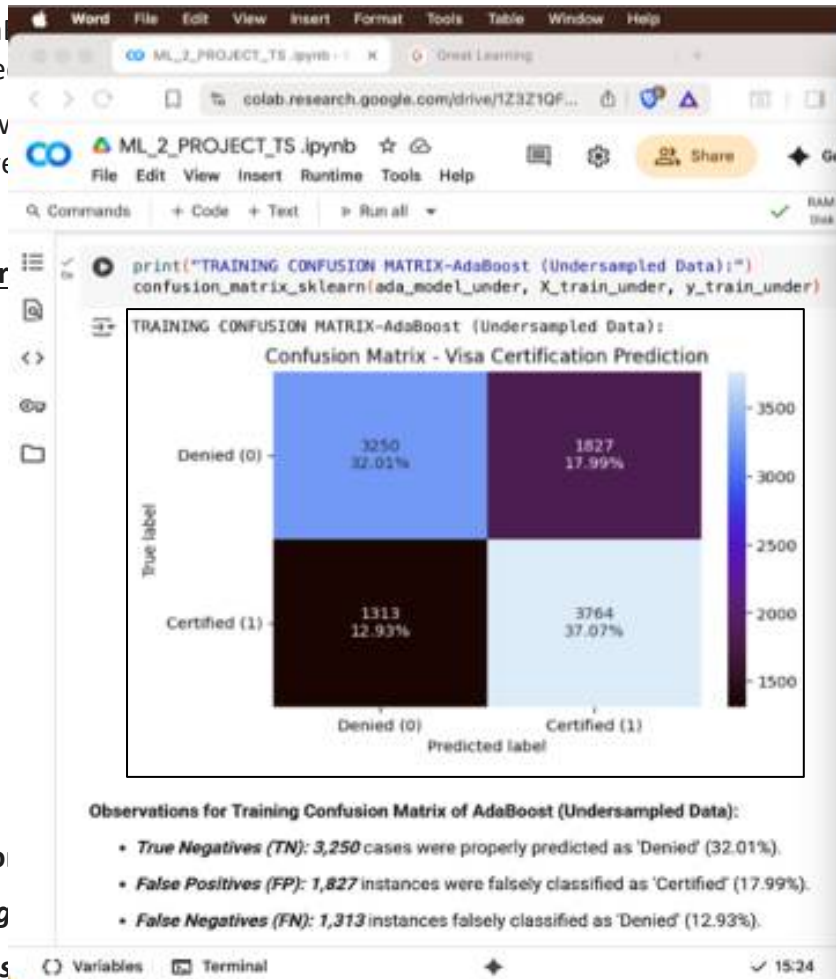
- **Accuracy (0.709969)**: The validation set has an accuracy of about 70.99%, which is slightly higher than the training set.
- **Recall (0.750808)**: A recall score of approximately 75.08% indicates that the model correctly recognises a significant number of positive cases.
- **Precision (0.802198)**: A precision score of around 80.22% implies that a large proportion of positive predictions are true positives.
- **F1 Score (0.775653)**: An F1 score of approximately 77.57% indicates a solid balance of precision and recall.

Conclusions:

- The AdaBoost model outperforms on the validation set, with higher accuracy and F1 score compared to the training set.

- The AdaBoost model compares the training data to the test data.
- This shows that the model is not overfitting.

Confusion Matrix

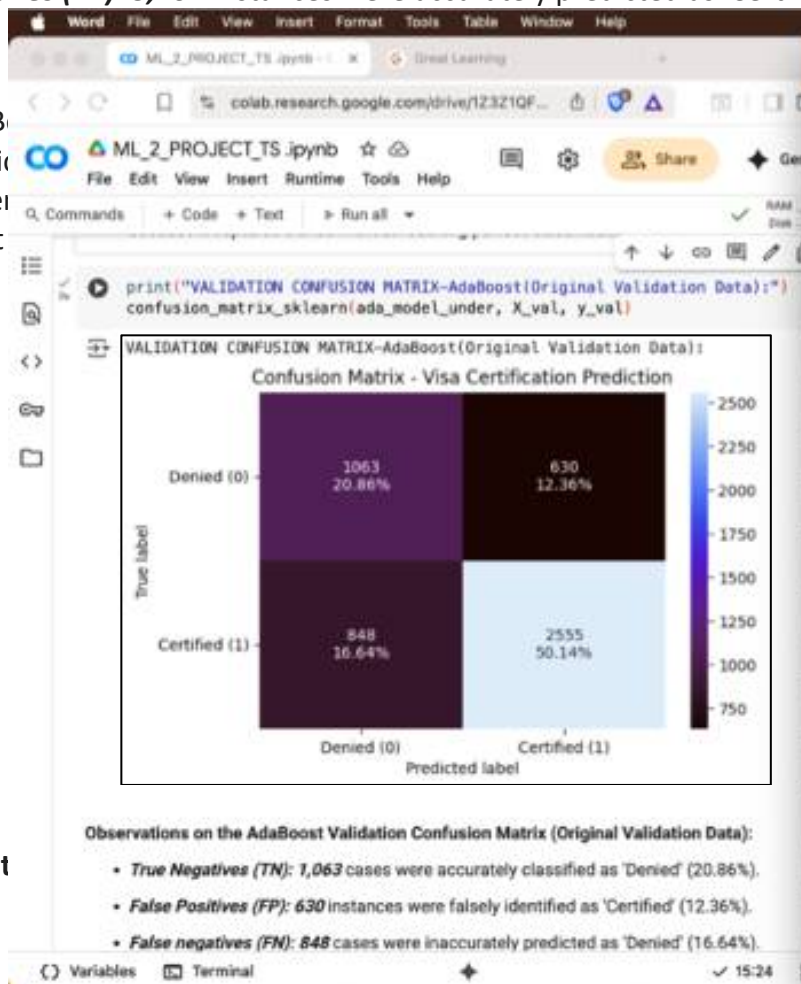


Observations for

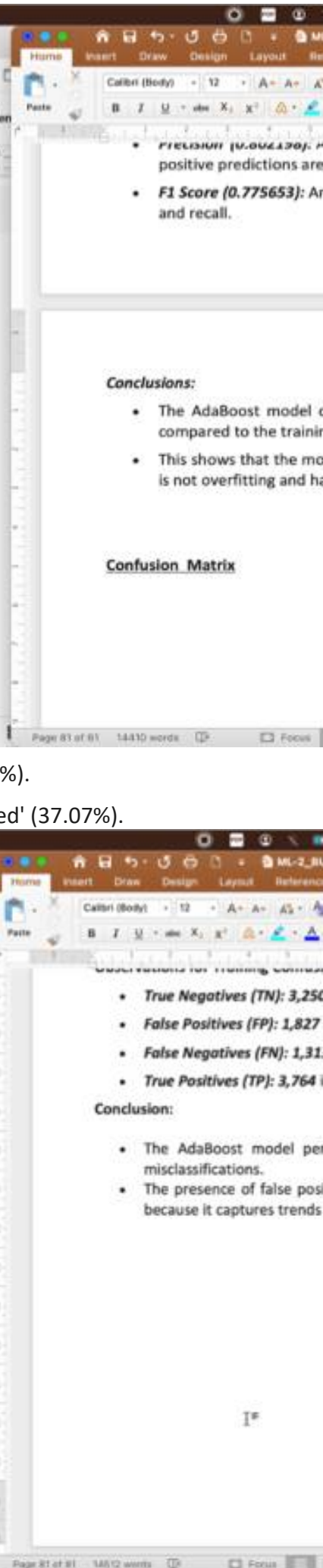
- **True Negatives (TN):** 3,250 cases were properly predicted as 'Denied' (32.01%).
- **False Positives (FP):** 1,827 instances were falsely classified as 'Certified' (17.99%).
- **False Negatives (FN):** 1,313 instances falsely classified as 'Denied' (12.93%).
- **True Positives (TP):** 3,764 instances were accurately predicted as 'Certified' (37.07%).

Conclusion:

- The AdaBoost model performs well on the training data.
- The presence of false positives is not a major concern because it captures trends.



Observations on t



- **True Negatives (TN): 1,063** cases were accurately classified as 'Denied' (20.86%).
- **False Positives (FP): 1,063** cases were incorrectly classified as 'Denied' (20.86%).
- **False negatives (FN): 1,063** cases were incorrectly classified as 'Approved' (20.86%).
- **True Positives (TP): 1,063** cases were accurately classified as 'Approved' (20.86%).

Conclusions:

- The model achieved a modest performance.
- The results show no symptoms of overfitting.
- The performance metrics are consistent across different data samples.

7.3.2 Gradient Boosting

Model Building

- We have seen how to build a Gradient Boosting model.

Performance Evaluation

Observations for

- **Accuracy (0.72336):** The model's overall accuracy is approximately 72.34%, indicating a moderate level of performance.
- **Recall (0.746898):** A recall score of around 74.69% suggests that the model correctly identifies a substantial portion of positive events ('Certified').
- **Precision (0.713318):** A precision score of about 71.33% indicates that a large proportion of positive predictions are true positives.
- **F1 Score (0.729722):** The F1 score, which is the harmonic mean of precision and recall, is approximately 72.97%.

Conclusions:

- The Gradient Boosting model shows no evidence of overfitting.
- The metrics show that the model efficiently captures patterns in the data without attaining perfect classification, implying a high level of generalisation.

The screenshot displays a Google Colab notebook interface. The notebook title is 'ML_2_PROJECT_TS.ipynb'. The code cell shows the following:

```
[136] gb_model_under = GradientBoostingClassifier(random_state=42)
      gb_model_under.fit(X_train_under, y_train_under)
```

A variable inspector window shows the object `GradientBoostingClassifier` with attribute `random_state=42`.

Below the code, a text block titled 'Checking performance for Gradient Boosting Model (Undersampled Data)' contains the following code:

```
[137] print("TRAINING PERFORMANCE-GB(Undersampled Data):")
      gb_train_perf_under = model_performance_classification_sklearn(gb_model_under, X_train_under, y_train_under)
```

The output of the code is a table showing the training performance metrics:

	Accuracy	Recall	Precision	F1
0	0.72336	0.746898	0.713318	0.729722

Below the table, a text block titled 'Observations for Training Performance of Gradient Boosting (Undersampled Data):' provides a detailed analysis of the metrics:

- **Accuracy (0.72336):** On the training set, the model achieved an accuracy of roughly 72.34%.
- **Recall (0.746898):** A recall score of around 74.69% suggests that the model correctly recognises a substantial portion of positive events ('Certified').
- **Precision (0.713318):** A precision score of about 71.33% indicates that a large proportion of positive predictions are true positives.

The bottom of the notebook shows the 'Variables' and 'Terminal' tabs, along with the status '15:24 Python 3'.

```
[138] print("VALIDATION PERFORMANCE-GB (Original Validation Data):")
gb_val_perf_under = model_performance_classification_skllearn(gb_model_under,
gb_val_perf_under
```

VALIDATION PERFORMANCE-GB (Original Validation Data):				
	Accuracy	Recall	Precision	F1
0	0.716052	0.739348	0.817945	0.776663

Observations for Validation Performance

- **Accuracy (0.716052):** The accuracy score on the validation set is ~71.61%, suggesting a decline as compared to the training set.
- **Recall (0.739348):** A recall score of around 73.93% indicates that the model continues to recognise a significant number of positive cases.
- **Precision (0.817945):** A precision score of around 81.79% implies that the majority of predicted positive cases are true positives.
- **F1 Score (0.776663):** An F1 score of approximately 77.67% indicates a solid balance of precision and recall.

Conclusions:

- The Gradient Boosting model outperforms on the validation set, with only a modest reduction in accuracy compared to the training set.
- However, the improvement in precision and F1 score indicates that the model is effectively balancing precision and recall.

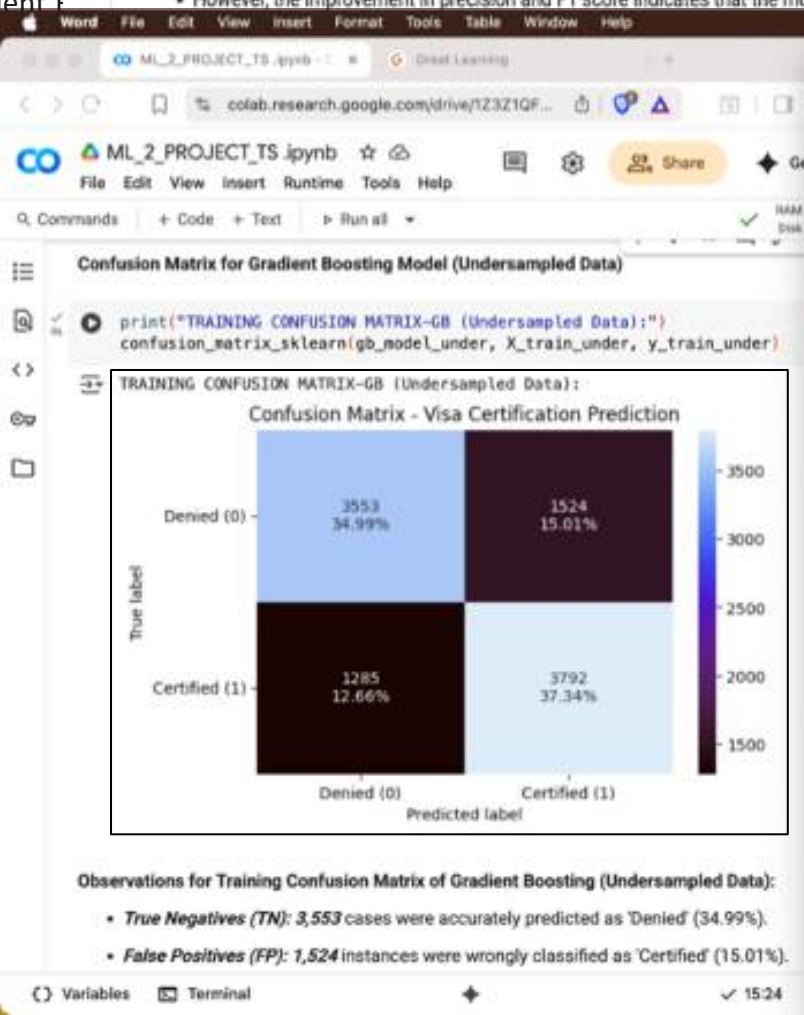
Observations for Validation Performance of Gradient Boosting (Original Validation Data):

- **Accuracy (0.716052):** The accuracy score on the validation set is ~71.61%, suggesting a decline as compared to the training set.
- **Recall (0.739348):** A recall score of around 73.93% indicates that the model continues to recognise a significant number of positive cases.
- **Precision (0.817945):** A precision score of around 81.79% implies that the majority of predicted positive cases are true positives.
- **F1 Score (0.776663):** An F1 score of approximately 77.67% indicates a solid balance of precision and recall.

Conclusions:

- The Gradient Boosting model outperforms on the validation set, with only a modest reduction in accuracy compared to the training set.
- However, the improvement in precision and F1 score indicates that the model is effectively balancing precision and recall.

Confusion Matrix



Observations for

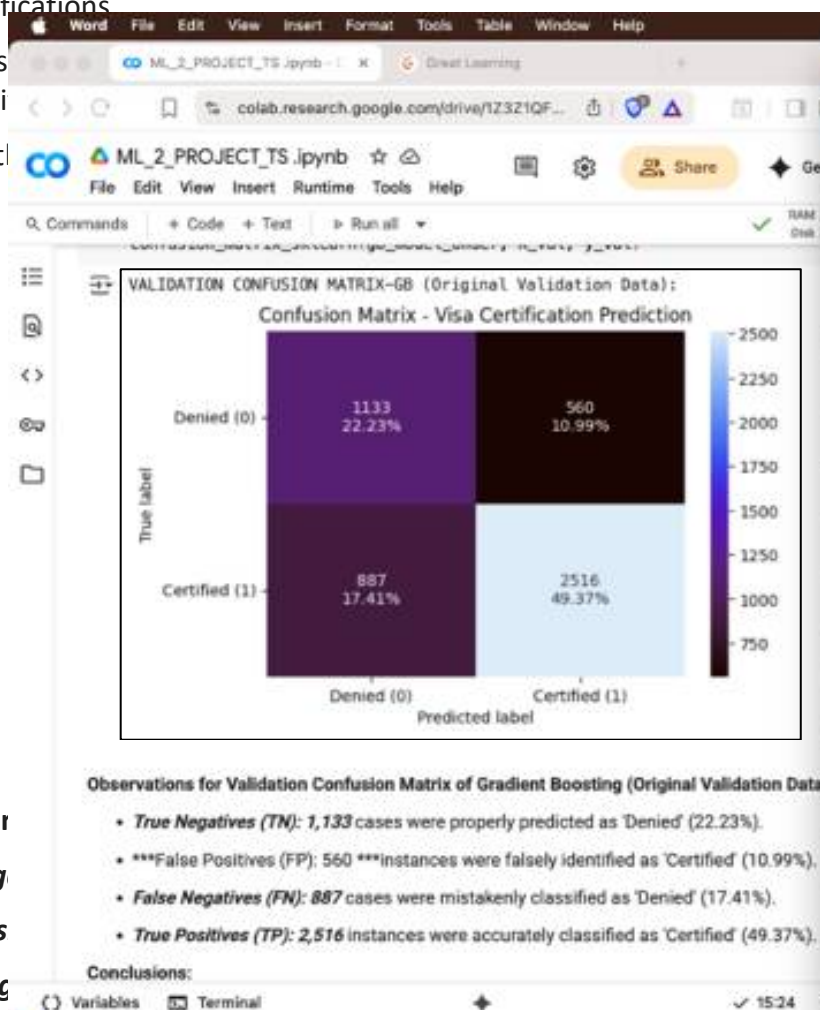
- **True Negatives (TN):** 3,553 cases were accurately predicted as 'Denied' (34.99%).
- **False Positives (FP):** 1,524 instances were wrongly classified as 'Certified' (15.01%).
- **False Negatives (FN):** 1,285 instances were wrongly classified as 'Denied' (12.66%).
- **True Positives (TP):** 3,792 instances were accurately identified as 'Certified' (37.34%).

- **Precision (0.716052):** The precision score on the validation set is ~71.61%, suggesting a decline as compared to the training set.
 - **F1 Score (0.776663):** An F1 score of approximately 77.67% indicates a solid balance of precision and recall.
- Conclusions:
- The Gradient Boosting model outperforms on the validation set, with only a modest reduction in accuracy compared to the training set.
 - However, the improvement in precision and F1 score indicates that the model is effectively balancing precision and recall.

Confusion Matrix

Conclusions:

- The Gradient Boosting model performs well on the training set, with a fair proportion of misclassifications.
- False positives and false negatives trends without attaining perfect classification.
- Overall, the model shows a high level of performance on the validation set.



Observations for

- **True Negatives (TN): 1,133** cases were properly predicted as 'Denied' (22.23%).
- **False Positives (FP): 560** instances were falsely identified as 'Certified' (10.99%).
- **False Negatives (FN): 887** cases were mistakenly classified as 'Denied' (17.41%).
- **True Positives (TP): 2,516** instances were accurately classified as 'Certified' (49.37%).

Conclusions:

- The model performs well on the validation set, with a high number of true positives (2,516) and a modest number of misclassifications.
- The results show that the Gradient Boosting model generalises well to new data, with no evidence of overfitting.
- The performance indicators indicate a high level of model accuracy.

7.3.3 XGBo

Model Building

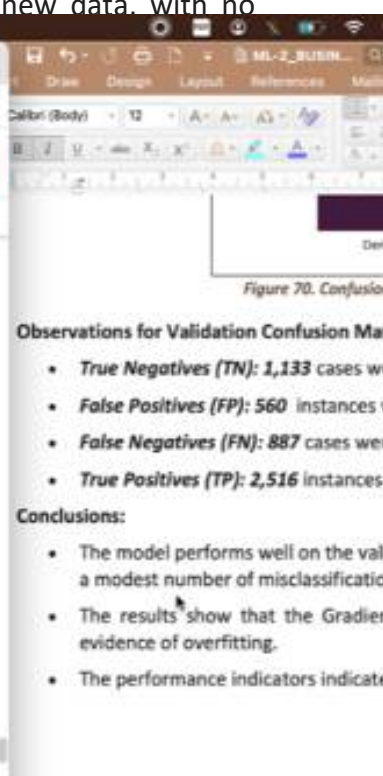
```
[141] xgb_model_under = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')
xgb_model_under.fit(X_train_under, y_train_under)

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, feature_weights=None, gamma=None,
               grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=None, max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=None, max_leaves=None,
               min_child_weight=None, missing=None, monotone_constraints=None,
               multi_strategy=None, n_estimators=None, n_jobs=None,
               num_parallel_tree=None, objective=None,
               print_eval_method=None, random_state=42,
               reg_alpha=None, reg_lambda=None, scale_pos_weight=None,
               subsample=None, tree_method=None, verbosity=None,
               watchlist=None, weight_posterior=None)

Checking performance for XGBoost Model (Undersampled Data)

[142] print("TRAINING PERFORMANCE-XGB (Undersampled Data):")
xgb_train_perf_under = model_performance_classification_sklearn(xgb_model_under, X_train_under, y_train_under)

TRAINING PERFORMANCE-XGB (Undersampled Data):
```



- We have succe

Performance Evaluati

Checking performance for XGBoost Model (Undersampled Data)

```
[142] print("TRAINING PERFORMANCE-XGB (Undersampled Data):")
xgb_train_perf_under = model_performance_classification_sklearn(xgb_model_under, X_t
xgb_train_perf_under
```

TRAINING PERFORMANCE-XGB (Undersampled Data):

	Accuracy	Recall	Precision	F1
0	0.865555	0.861926	0.86998	0.865935

Observations on the Training Performance of XGBoost (Undersampled Data):

Observations on the

- **Accuracy (0.865555):** The model obtains a high accuracy of around 86.56% on the training data.
- **Recall (0.861926):** A recall score of around 86.19% indicates that the model detects a substantial proportion of positive cases.
- **Precision (0.86998):** A precision score of around 86.99% implies that a significant number of positive predictions are true positives.
- **F1 Score (0.865935):** An F1 score of approximately 86.59% indicates a moderate balance between precision and recall.

Conclusions:

- The XGBoost model performs well on the training data, with high accuracy, recall, precision, and F1 scores.
- The high scores across all criteria indicate that the model is capturing noise and unique patterns.

- The high scores across all criteria indicate that the model learn capturing noise and unique patterns.

```
[143] print("VALIDATION PERFORMANCE-XGB (Original Validation Data):")
xgb_val_perf_under = model_performance_classification_sklearn(xgb_model_un
xgb_val_perf_under
```

VALIDATION PERFORMANCE-XGB (Original Validation Data):

	Accuracy	Recall	Precision	F1
0	0.690345	0.694387	0.814547	0.749683

Observations for Validation Performance of XGBoost (Original Validation Data):

- **Accuracy (0.690345):** The accuracy on the validation set is ~69.03%, suggesting a significant drop compared to the training set.
- **Recall (0.694387):** A recall score of around 69.44% indicates that the model detects a good proportion of positive cases.
- **Precision (0.814547):** A precision score of around 81.45% implies that a significant number of positive predictions are true positives.
- **F1 Score (0.749683):** An F1 score of approximately 74.97% indicates a moderate balance between precision and recall.

Conclusions:

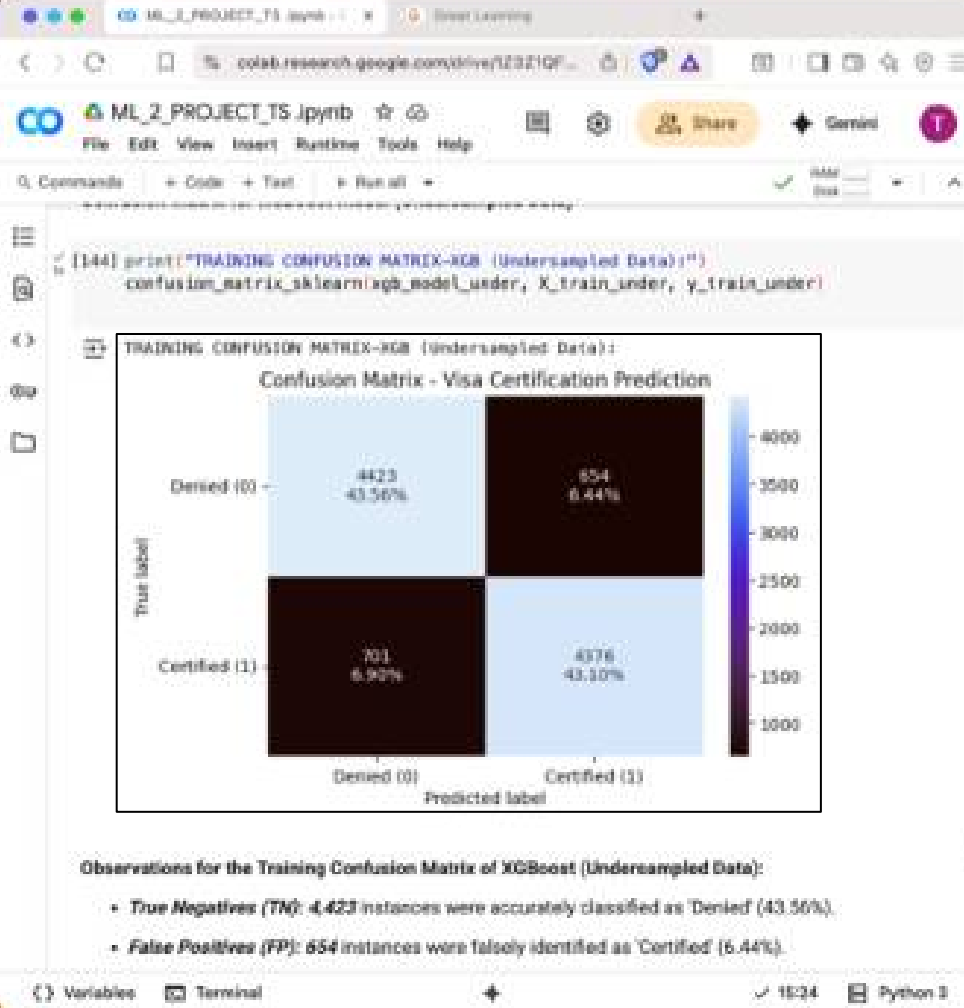
- The XGBoost model performs significantly worse on the validation set, with lower accuracy, recall, precision, and F1 scores compared to the training set.

Conclusions:

- The XGBoost model achieves higher accuracy, recall, precision, and F1 scores on the training data compared to the validation data.
- This indicates that the model is not generalising effectively to unseen data, which confirms concerns of overfitting.

- The misclassification rate is low, indicating good model performance.
- The model's ability to correctly identify 'Certified' instances is high, suggesting it is effective in detecting potential security vulnerabilities.

Confusion Matrix

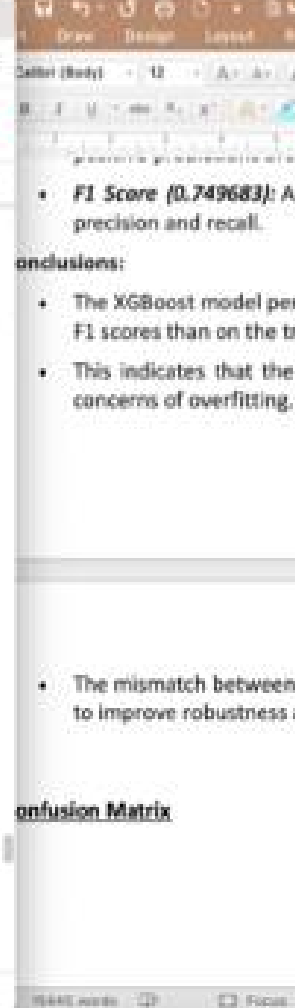
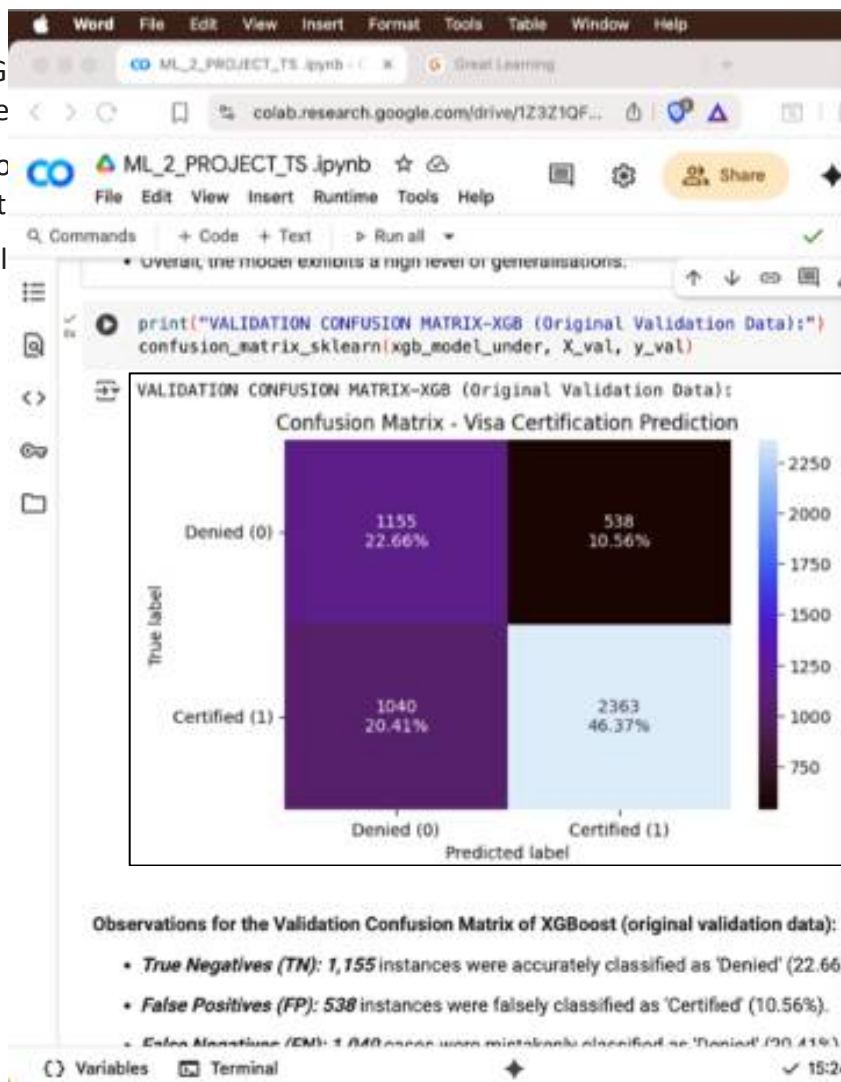


Observations for

- **True Negatives (TN):** 4,423 instances were accurately classified as 'Denied' (43.56%).
- **False Positives (FP):** 654 instances were falsely identified as 'Certified' (6.44%).
- **False negatives (FN):** 701 cases were inaccurately predicted as 'Denied' (6.90%).
- **True Positives (TP):** 4,376 instances were accurately identified as 'Certified' (43.10%).

Conclusions:

- The XGBoost model demonstrates high performance in classifying 'Denied' instances (True Negatives).
- The model's ability to correctly identify 'Certified' instances is high, suggesting it is effective in detecting potential security vulnerabilities.
- Overall, the model's performance is strong, with a high True Positive Rate and a low False Positive Rate.



Confusion Matrix

Observations for the Training

- **True Negatives (TN):** 4,423 instances were accurately classified as 'Denied' (43.56%).
- **False Positives (FP):** 654 instances were falsely identified as 'Certified' (6.44%).
- **False negatives (FN):** 701 cases were inaccurately predicted as 'Denied' (6.90%).
- **True Positives (TP):** 4,376 instances were accurately identified as 'Certified' (43.10%).

Conclusions:

- The XGBoost model demonstrated high performance in classifying 'Denied' instances (True Negatives).
- The model's ability to correctly identify 'Certified' instances is high, suggesting it is effective in detecting potential security vulnerabilities.
- Overall, the model's performance is strong, with a high True Positive Rate and a low False Positive Rate.

Observations for the Validation Confusion Matrix of XGBoost (original validation data):

- **True Negatives (TN): 1,155** instances were accurately classified as 'Denied' (22.66%).
- **False Positives (FP): 538** instances were falsely classified as 'Certified' (10.56%).
- **False Negatives (FN): 1,040** cases were mistakenly classified as 'Denied' (20.41%).
- **True Positives (TP): 2,363** instances were accurately identified as 'Certified' (46.37%).

Conclusions:

- On the validation set, the model performs moderately, with a substantial number of true positives and false negatives.
- The decrease in performance as compared to the training set suggests that the model is not generalising well to new data, confirming indicators of overfitting.
- The mismatch in measurements shows that further tuning is required to improve robustness and generalisation.

8 HYPERPARAMETER TUNING

We chose AdaBoost, Gradient Boosting, XGBoost, and Random Forest for hyperparameter tuning. Gradient Boosting had the **highest validation F1 score**, demonstrating outstanding baseline performance. **AdaBoost's initial F1 was somewhat higher than XGBoost's** so we chose that as well. We also chose **XGBoost since it has more tuning options** and typically performs better following hyperparameter optimisation. Random Forest was chosen because of its **consistent and reliable performance**.

We did not select models trained on oversampled or undersampled data because they had lower validation scores. Sampling probably didn't help much because th

Hyperparameter Tuning Strategy

- To improve model performance and directly address the class imbalance, we efficiently tuned critical parameters using **RandomizedSearchCV**.
- Objective: Reduce overfitting and improve minority class recognition ("Denied").

Common Key Parameters:

- ***n_estimators***: Increased the number of trees to make the model more resilient.
- ***max_depth***: Restricts tree complexity to avoid overfitting.
- To address imbalance, misclassifying "Denied" situations was penalised directly using ***class_weight (RF)* and *scale_pos_weight (XGBoost)***.
- ***min_samples_leaf* & *min_samples_split***: Ensures that splits were based on appropriate sample sizes.
- ***RandomizedSearchCV*** was used to identify the optimal combination of these parameters across a large search space.

```
class_weight = compute_class_weight('balanced', classes=np.unique(y_train), y_train)
class_weight_dict = dict(zip(np.unique(y_train), class_weight))

verbose=1,
random_state=42,
n_jobs=-1
)

# Perform tuning
print("Performing hyperparameter tuning for Gradient Boosting:")
gb_random.fit(X_train, y_train)

# Best parameters and score
print("Best Parameters for Gradient Boosting:")
print(gb_random.best_params_)
print(f"Best Cross-Validation F1 Score: {gb_random.best_score_:.4f}")

# Evaluate tuned model
gb_tuned = gb_random.best_estimator_

Performing hyperparameter tuning for Gradient Boosting:
Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best Parameters for Gradient Boosting:
{'subsample': 1.0, 'n_estimators': 300, 'min_samples_split': 20, 'min_samples_leaf': 4, 'max_features': 'log2', 'max_depth': 5, 'loss': 'exponential', 'lear
```

Observations:

- After tuning, the best Gradient Boosting model achieved an average F1 score of 0.8261, somewhat higher than the original score of 0.8237. While the gain is minor, it indicates that hyperparameter adjustment improved the model's generalisation.
- The tuned Gradient Boosting model achieved an average F1 score of 0.8237. While the gain is minor, it indicates that hyperparameter adjustment improved the model's generalisation.

Checking Performance of Tuned Gradient Boosting Model

[140] print("Tuned Gradient Boosting - Training Performance:")

somewhat higher than the original score of 0.8237. While the gain is minor, it indicates that hyperparameter adjustment improved the model's generalisation.

Performance Evaluation

- The tuned Gradient Boosting model achieved an average F1 score of 0.8261, somewhat higher than the original score of 0.8237. While the gain is minor, it indicates that hyperparameter adjustment improved the model's generalisation.

Checking Performance of Tuned Gradient Boosting Model

```
[148] print("Tuned Gradient Boosting - Training Performance:")
gb_train_perf_tuned = model_performance_classification_sklearn(gb_tuned, X_train, y_train)
gb_train_perf_tuned
```

Tuned Gradient Boosting - Training Performance:				
	Accuracy	Recall	Precision	F1
0	0.758438	0.89335	0.777929	0.831654

Observations on the Training Data Performance of Tuned Gradient Boosting:

- **Accuracy (0.758438):** The training accuracy is at 75.84%, indicating that the model fits the data well.

Observations on the Training Data Performance of Tuned Gradient Boosting:

- **Accuracy (0.758438):** The training accuracy is at 75.84%, indicating that the model fits the data well.
- **Recall (0.891566):** Recall remains high, at approximately 89.16%, indicating that the model continues to recognise the majority of positive cases on previously unseen data.
- **Precision (0.76713):** A precision of 76.71% implies that a significant proportion of positive predictions are true positives based on validation data.
- **F1 Score (0.824681):** The F1 score of 82.47% demonstrates an excellent mix of precision and recall, which is comparable to training performance.

Conclusions:

- The tuned Gradient Boosting model performs well on the training set, with good recall and balanced precision, suggesting effective improvement.
- The metrics scores indicate that the model accurately reflects the underlying patterns while avoiding obvious underfitting.

The screenshot displays a Google Colab notebook interface. The top part shows the training performance metrics for a Gradient Boosting model. Below this, a code cell is executed, printing the validation performance metrics. The output is a table with the following data:

	Accuracy	Recall	Precision	F1
0	0.74686	0.891566	0.76713	0.824681

Below the table, the notebook provides observations for the validation data performance, followed by a conclusion section.

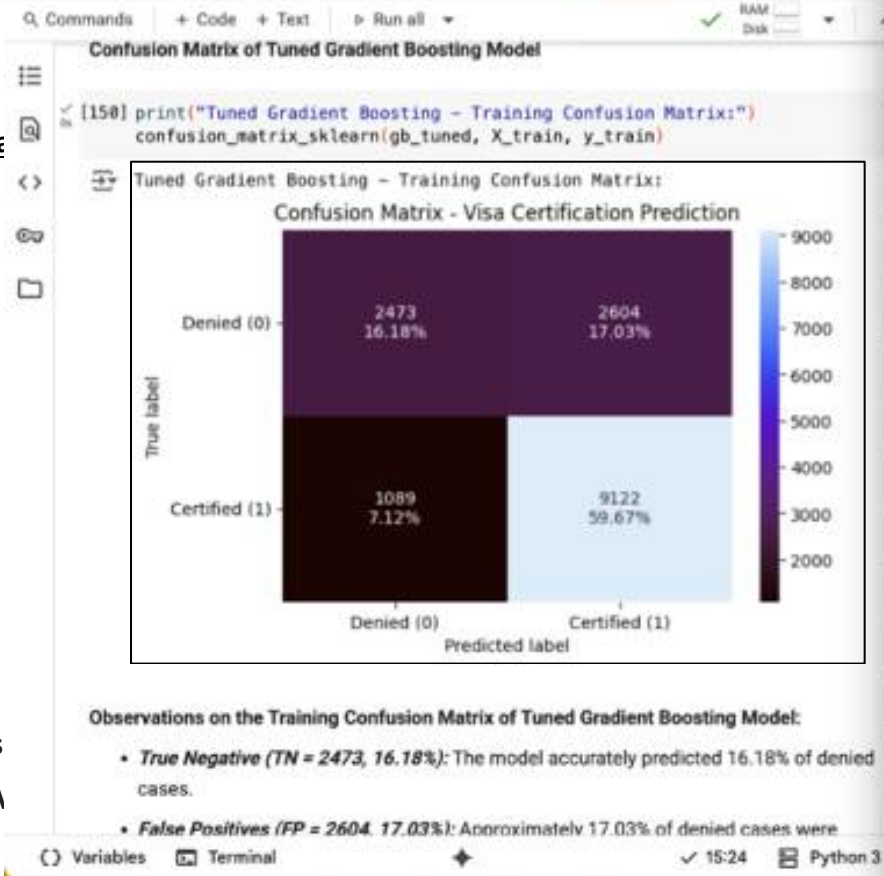
Observations for Validation Data Performance of Tuned Gradient Boosting:

- **Accuracy (0.74686):** The validation accuracy is roughly 74.69%, which is little lower than the training accuracy but still high.
- **Recall (0.891566):** Recall remains high, at approximately 89.16%, indicating that the model continues to recognise the majority of positive cases on previously unseen data.
- **Precision (0.76713):** A precision of 76.71% implies that a significant proportion of positive predictions are true positives based on validation data.
- **F1 Score (0.824681):** The F1 score of 82.47% demonstrates an excellent mix of precision and recall, which is comparable to training performance.

Conclusions:

- The model's validation measures are similar to training results, showing high generalisation and minimal overfitting.
- The minor decline in accuracy and precision is to be expected, and it demonstrates that the model's prediction capacity on new data remains strong.

Confusion Matrix

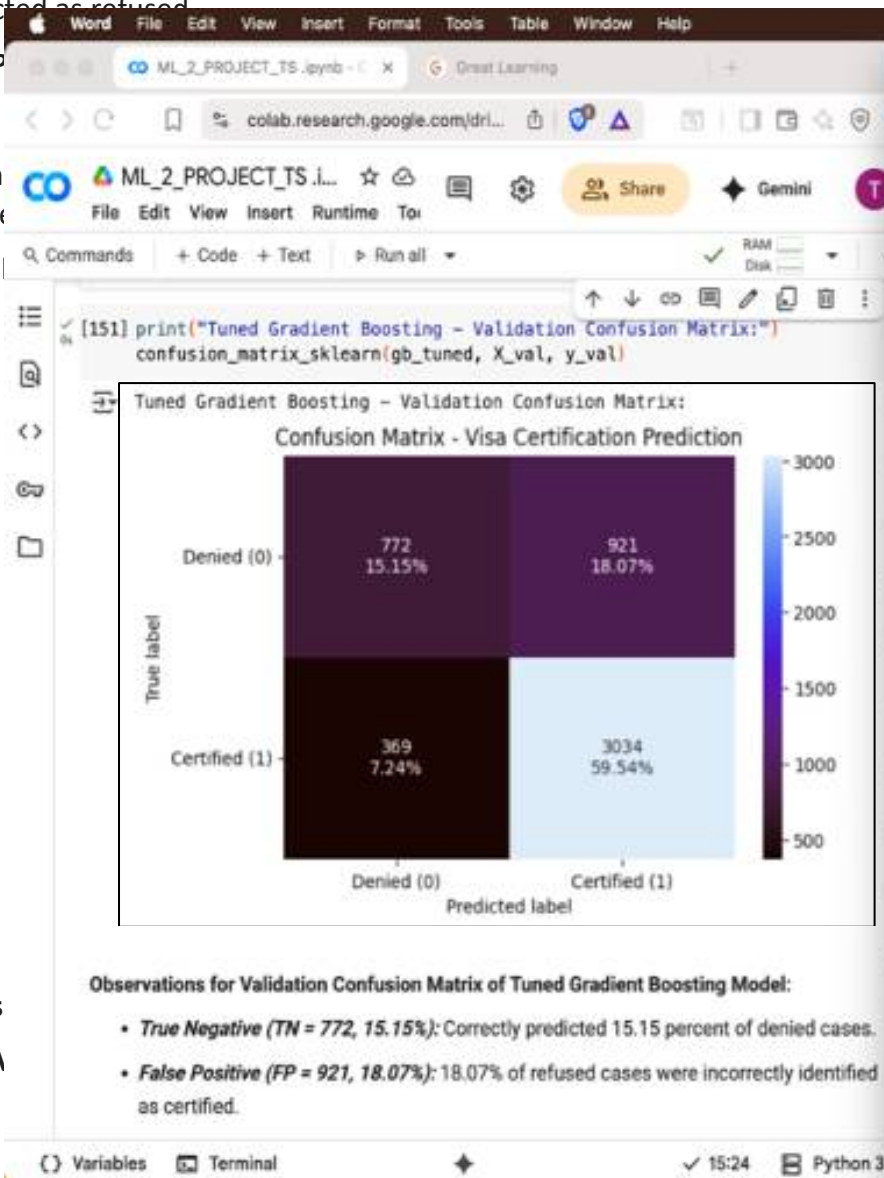


Observations

- **True Negatives (TN = 2473, 16.18%):** The model accurately predicted 16.18% of denied cases.
- **False Positives (FP = 2604, 17.03%):** Approximately 17.03% of denied cases were predicted as certified.
- **False Negatives (FN = 1089, 7.12%):** Approximately 7.12% of certified cases were falsely predicted as refused.
- **True Positives (TP = 9122, 59.67%):** The model correctly predicted 59.67% of certified cases.

Conclusions:

- The model's validation accuracy is high, suggesting minimal overfitting.
- The minor decline in precision for denied cases suggests the model's prediction capability for denied cases is slightly lower than for certified cases.



Observations

- **True Negatives (TN = 772, 15.15%):** Correctly predicted 15.15 percent of denied cases.
- **False Positive (FP = 921, 18.07%):** 18.07% of refused cases were incorrectly identified as certified.

- **training accuracy but**
- **Recall (0.891566):** Re continues to recognise
- **Precision (0.76713):** A predictions are true p
- **F1 Score (0.824681):** recall, which is compa

Conclusions:

- The model's validation accuracy is high, suggesting minimal overfitting.
- The minor decline in precision for denied cases suggests the model's prediction capability for denied cases is slightly lower than for certified cases.

Confusion Matrix

Observations on the Training

- **True Negative (TN = 2473, 16.18%):** The model accurately predicted 16.18% of denied cases.
- **False Positives (FP = 2604, 17.03%):** Approximately 17.03% of denied cases were predicted as certified.
- **False Negatives (FN = 1089, 7.12%):** Approximately 7.12% of certified cases were falsely predicted as refused.
- **True Positive (TP = 9122, 59.67%):** The model correctly predicted 59.67% of certified cases.

Conclusions:

- The model has an excellent performance, with a high accuracy and a low false positive rate, suggesting that the model is well-suited for this task.
- Overall, the training data shows that the model is performing well, with a high accuracy and a low false positive rate.

- **False Positive (FP = 921, 18.07%):** 18.07% of refused cases were incorrectly identified as certified

```

# Hyperparameter tuning
print("Performing hyperparameter tuning for XGBoost:")
xgb_random.fit(X_train, y_train)

# Best parameters and score
print("Best Parameters for XGBoost:")
print(xgb_random.best_params_)
print(f"Best Cross-Validation F1 Score: {xgb_random.best_score_:.4f}")

# Evaluate tuned model
xgb_tuned = xgb_random.best_estimator_
  
```

```

Performing hyperparameter tuning for XGBoost:
Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best Parameters for XGBoost:
{'subsample': 0.8, 'scale_pos_weight': np.float64(0.7458133385564587), 'reg_lambda': 1, 'reg_alpha': 0.1, 'n_estimators': 100, 'min_child_weight': 4, 'max_d
Best Cross-Validation F1 Score: 0.8262
  
```

Observations:

- 5-fold CV was performed with 50 parameter candidates (250 fits).
- The optimal parameters chosen were `reg_lambda=1` and `balance`
- The best cross-validation result was achieved with a score of 0.8262.
- XGBoost performance is relatively good because our baseline model was already performing well.

Checking Performance of XGBoost

```

[153] print("Tuned XGBoost - Training Performance:")
xgb_train_perf_tuned = model_performance_classification_sklearn(xgb_tuned,
  
```

Performance Evaluation

improvement by tuning from the base line model. The model was already performing good on the validation set, hence there was only a little room for improvement.

Checking Performance of XGBoost Tuned Model

```

[153] print("Tuned XGBoost - Training Performance:")
xgb_train_perf_tuned = model_performance_classification_sklearn(xgb_tuned,
xgb_train_perf_tuned
  
```

	Accuracy	Recall	Precision	F1
0	0.758307	0.887866	0.780475	0.830714

Observations on

Observations on Tuned XGBoost - Training Performance:

- **Accuracy (0.7583):** The model successfully predicts approximately 75.83% of cases based on training data.
- **Recall (0.8879):** A high recall indicates accurate detection of positive cases.
- **Precision (0.7805):** indicates that 78.05% of positive predictions are correct.

- **Accuracy** training c
- **Recall (0.**
- **Precision**
- **F1 Score**

Conclusions:

- The mod effective

- **F1 Score (0.8307):** indicates a balanced performance between precision and recall.

Conclusions:

- The model performs well on training data, with high recall and balanced precision, indicating effective learning without overfitting.

```
[154] print("Tuned XGBoost - Validation Performance:")
xgb_val_perf_tuned = model_performance_classification_sklearn(xgb_tuned, X
xgb_val_perf_tuned
```

Tuned XGBoost - Validation Performance:

	Accuracy	Recall	Precision	F1
0	0.74686	0.884808	0.770274	0.823578

Observations on Tuned XGBoost - Validation Performance:

- **Accuracy (0.7469):** is slightly lower (74.69%) on validation data.
- **Recall (0.8848):** remains high, indicating consistent positive case detection.
- **Precision (0.7703):** Slightly lower but still acceptable, suggesting positive predictions are true.

Observations on

- **Accuracy**
- **Recall (0.**
- **Precision**
- **The F1** general

Conclusions:

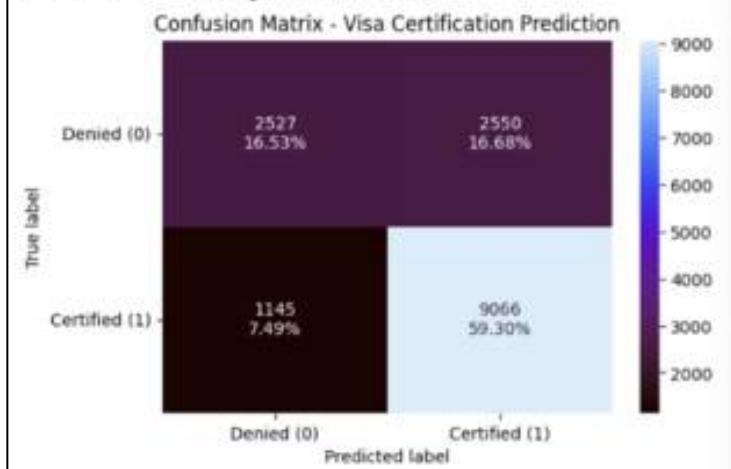
- The m minim

Confusion Ma

Confusion Matrix of XGBoost Tuned Model

```
[155] print("Tuned XGBoost - Training Confusion Matrix:")
confusion_matrix_sklearn(xgb_tuned, X_train, y_train)
```

Tuned XGBoost - Training Confusion Matrix:



Observations on Tuned XGBoost - Training Confusion Matrix:

- **True Negative (TN = 2527, 16.53%):** Correctly predicted 16.53% of denied cases.

Observations

- **True N**

- **Accuracy (0.7469):**
- **Recall (0.8848):** re
- **Precision (0.7703):**
- **The F1 score (0.82** generalisation.

Conclusions:

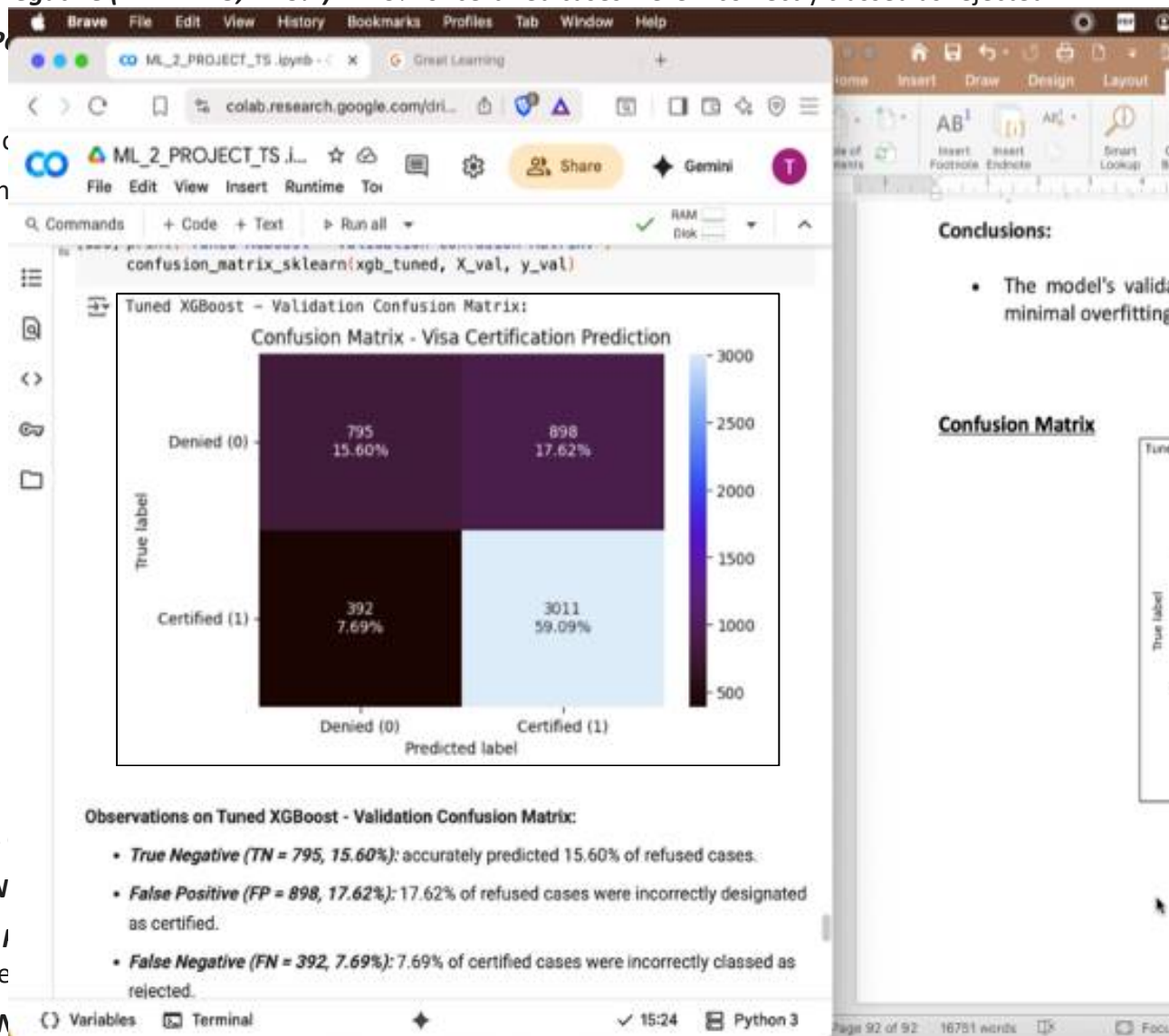
- The model's valid minimal overfitting

Confusion Matrix

- **False Positive (FP = 2550, 16.68%):** 16.68% of rejected cases were incorrectly labelled as certified.
- **False Negative (FN = 1145, 7.49%):** 7.49% of certified cases were incorrectly classed as rejected.
- **True Positive (TP = 3011, 59.09%):** Recognised 59.09% of certified cases.

Conclusions:

- The model's validation findings closely mirror training performance, with comparable rates of true and false positives.
- This consistency suggests good generalisation with no major overfitting.



Observations

- **True Negative (TN = 795, 15.60%):** accurately predicted 15.60% of refused cases.
- **False Positive (FP = 898, 17.62%):** 17.62% of refused cases were incorrectly designated as certified.
- **False Negative (FN = 392, 7.69%):** 7.69% of certified cases were incorrectly classed as rejected.
- **True Positive (TP = 3011, 59.09%):** Recognised 59.09% of certified cases.

Conclusions:

- Validation findings closely mirror training performance, with comparable rates of true and false positives.
- This consistency suggests good generalisation with no major overfitting.

8.3 Random Forest (RF) Model Tuning

Model Tuning

```
# Evaluate tuned model
rf_tuned = rf_random.best_estimator_
```

Performing hyperparameter tuning for Random Forest:
 Fitting 5 folds for each of 30 candidates, totalling 150 fits
 Best Parameters for Random Forest:
 {'n_estimators': 100, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'log2', 'max_depth': 10, 'class_weight': None}
 Best Cross-Validation F1 Score: 0.8260

+ Code

+ Text

Observations:

- Performed a 5-fold CV
- The optimal parameter
- The best cross-validated
- Random Forest performed

Checking Performance of T

```
[158] print("Tuned Random Forest Performance:")
rf_train_perf_tuned = model_performance_classification_sklearn(rf_tuned, X_train, y_train)
rf_val_perf_tuned = model_performance_classification_sklearn(rf_tuned, X_val, y_val)
```

Tuned Random Forest

{ } Variables { } Terminal

Observations on T

- Accuracy (0.7682):** Higher training accuracy of 76.82%, indicating good model fit.
- Recall (0.8979):** Very high recall indicates great positive case identification.
- Precision (0.7857):** Precision is moderately high at 78.57%, suggesting that a significant proportion of positive predictions are true.
- F1 Score (0.8380):** Balanced with slightly better overall performance than XGBoost.

Conclusions:

- Random Forest exhibits excellent training performance, with good recall and precision, indicating effective learning.

- Accuracy (0.7682):** Higher training accuracy of 76.82%, indicating good model fit.
- Recall (0.8979):** Very high recall indicates great positive case identification.
- Precision (0.7857):** Precision is moderately high at 78.57%, suggesting that a significant proportion of positive predictions are true.
- F1 Score (0.8380):** Balanced with slightly better overall performance than XGBoost.

Conclusions:

- Random Forest exhibits excellent training performance, with good recall and precision, indicating effective learning.

```
[159] print("Tuned Random Forest - Validation Performance:")
rf_val_perf_tuned = model_performance_classification_sklearn(rf_tuned, X_val, y_val)
rf_val_perf_tuned
```

Tuned Random Forest - Validation Performance:

	Accuracy	Recall	Precision	F1
0	0.747841	0.887746	0.769878	0.824621

Observations on Tuned Random Forest - Validation Performance:

- Accuracy (0.7478):** The validation accuracy is 74.78%, which is close to training.
- Recall (0.8877):** Recall remains high, in line with training, showing that the model is exceptionally good at detecting positive cases.
- Precision (0.7699):** Slightly lower precision but still solid, indicating that positive predictions are correct.

Observations on T

- Accuracy (0.7478):** The validation accuracy is 74.78%, which is close to training.
- Recall (0.8877):** Recall remains high, in line with training, showing that the model is exceptionally good at detecting positive cases.
- Precision (0.7699):** Slightly lower precision but still solid, indicating that positive predictions are correct.

{ } Variables { } Terminal



✓ 15:24

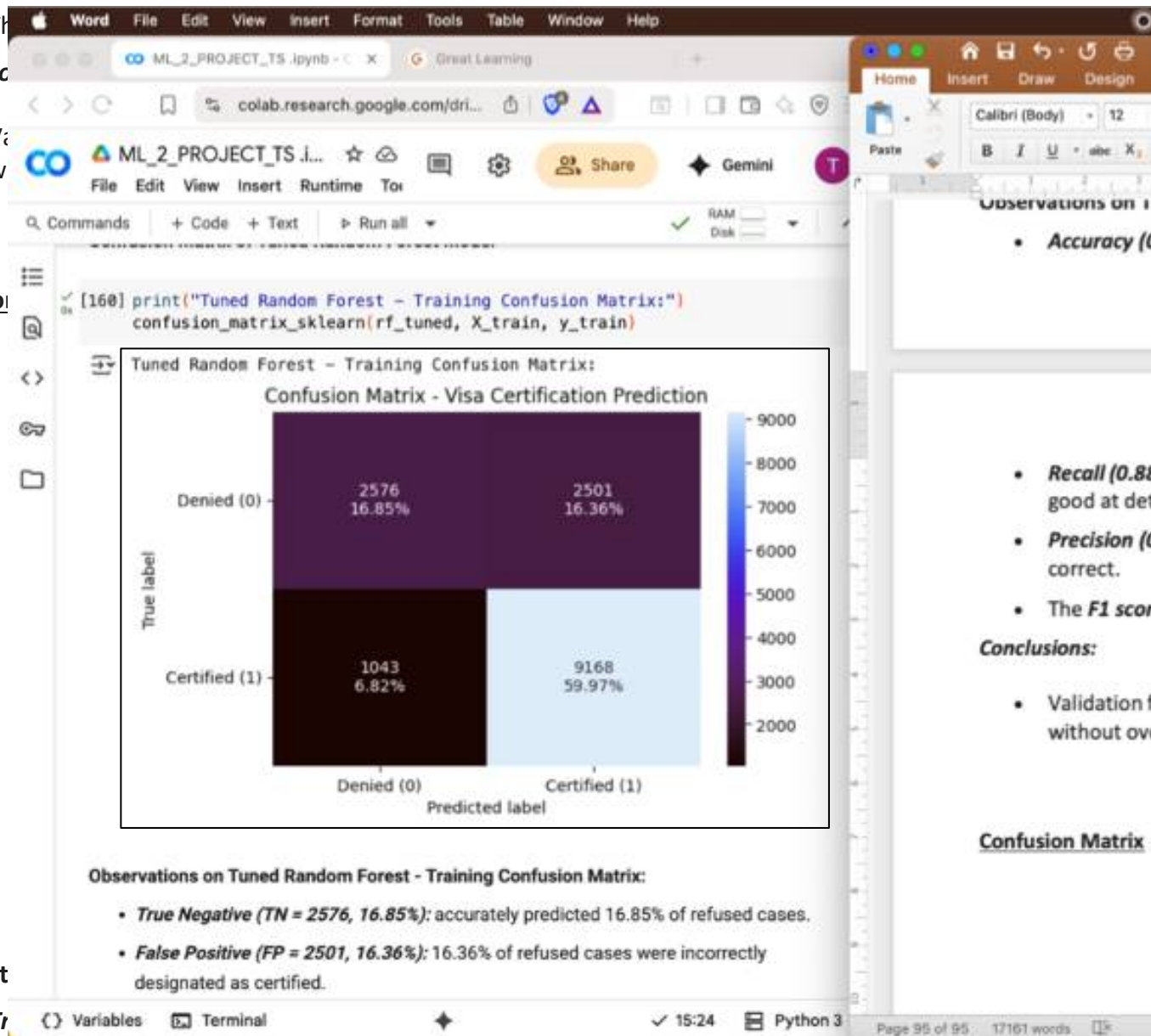
Python 3

- The

Conclusion

- Validation
- Without

Confusion



Observat

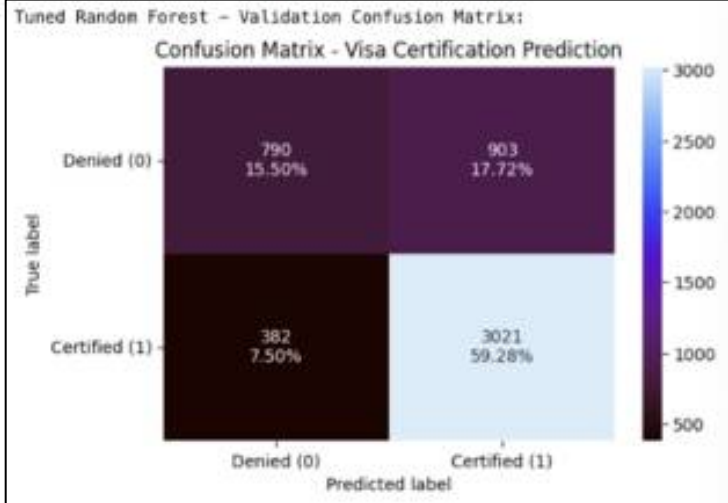
- **True Negative (TN = 2576, 16.85%):** accurately predicted 16.85% of refused cases.
- **False Positive (FP = 2501, 16.36%):** 16.36% of refused cases were incorrectly designated as certified.
- **False Negative (FN = 1043, 6.82%):** 6.82% of certified cases were incorrectly classed as rejected.
- **True Positives (TP = 9168, 59.97%):** Correctly detected 59.97% of certified cases.

Conclusions:

- On training data, the tuned Random Forest model beats XGBoost marginally due to higher true negative and true positive rates, as well as lower false negatives and false positives, showing robust learning.

positives, showing robust learning.

```
[161] print("Tuned Random Forest - Validation Confusion Matrix:")
confusion_matrix_sklearn(rf_tuned, X_val, y_val)
```



Observations on Tuned Random Forest - Validation Confusion Matrix:

- **True Negative (TN = 790, 15.50%):** Correctly predicted 15.50% of denied cases.
- **False Positive (FP = 903, 17.72%):** 17.72% of denied cases were incorrectly designated as certified.

Observations

- **True Negative (TN = 790, 15.50%):** Correctly predicted 15.50% of denied cases.
- **False Positive (FP = 903, 17.72%):** 17.72% of denied cases were incorrectly designated as certified.
- **False Negative (FN = 382, 7.50%):** 7.50% of certified cases were incorrectly classed as rejected.
- **True Positive (TP = 3021, 59.28%):** Accurately identified 59.28% of certified cases.

Conclu

-

8.4

Model

```
print(ada_random.best_params_)
print(f"Best Cross-Validation F1 Score: {ada_random.best_score_:.4f}")
```

```
# Evaluate tuned model
ada_tuned = ada_random.best_estimator_
```

Performing hyperparameter tuning for AdaBoost on original data:
Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best Parameters for AdaBoost:
{'n_estimators': 200, 'learning_rate': 0.05, 'estimator': DecisionTreeClassifier(max_depth=3, random_state=1)}
Best Cross-Validation F1 Score: 0.8234

Observations:

- A 5-fold CV was performed with 30 parameter candidates (150 fits).
- Best parameters: `n_estimators = 200`, `learning_rate = 0.05`, and `estimator = DecisionTreeClassifier(max_depth = 3)`.
- The best cross-validation result was `F1 = 0.8234`.
- AdaBoost performance is equivalent to Random Forest (0.8260) and XGBoost (0.8262), with just a little improvement over the baseline (0.8198).

Checking Performance of Tuned AdaBoost Model

```
[163] print("Tuned AdaBoost - Training Performance:")
ada_train_perf_tuned = model_performance_classification_sklearn(ada_tuned, X_train, y_train)
ada_train_perf_tuned
```

- **False Positive (FP = 903, 17.72%):** 17.72% of denied cases were incorrectly designated as certified.
- **False Negative (FN = 382, 7.50%):** 7.50% of certified cases were incorrectly classed as rejected.
- **True Positives (TP = 3021, 59.28%):** Accurately identified 59.28% of certified cases.

Conclusions:

- On training data, the negative and true p

Obsen

-
-
-

- AdaBoost p
just a little

Performance Evalu

- AdaBoost performance is equivalent to Random Forest (0.8260) and XGBoost (0.8262), with just a little improvement over the baseline (0.8198).

Checking Performance of Tuned AdaBoost Model

```
[163] print("Tuned AdaBoost - Training Performance:")
ada_train_perf_tuned = model_performance_classification_sklearn(ada_tuned,
ada_train_perf_tuned
```

Tuned AdaBoost - Training Performance:				
	Accuracy	Recall	Precision	F1
0	0.751177	0.87631	0.778832	0.8247

Observations on T

- **Accuracy** (0.751177)
- **Recall** (0.87631)
- **Precision** (0.778832)
- **F1 Score** (0.8247)

Conclusions:

- AdaBoost ou
learning.

- **Precision (0.7788)**: indicates that most positive predictions are right.
- **F1 Score (0.8247)**: indicates a balanced performance in terms of precision and recall.

Conclusions:

- AdaBoost outperforms on training data, with high recall and precision, indicating effective learning.

```
[164] print("Tuned AdaBoost - Validation Performance:")
ada_val_perf_tuned = model_performance_classification_sklearn(ada_tuned, X
ada_val_perf_tuned
```

Tuned AdaBoost - Validation Performance:				
	Accuracy	Recall	Precision	F1
0	0.743721	0.876579	0.771	0.820407

Observations on Tuned AdaBoost - Validation Performance:

- **Accuracy (0.7437)**: Validation data showed a slightly lower accuracy (74.37%).
- **Recall (0.8766)**: Recall is steady, indicating stable positive case detection.
- **Precision (0.7710)**: Slightly lower but still acceptable, suggesting model is able to predict positive case correctly.
- **F1 Score (0.8204)**: This score is near to training and indicates strong generalisation.

Conclusions:

- AdaBoost's validation measures are similar to training, indicating strong generalisation without overfitting.

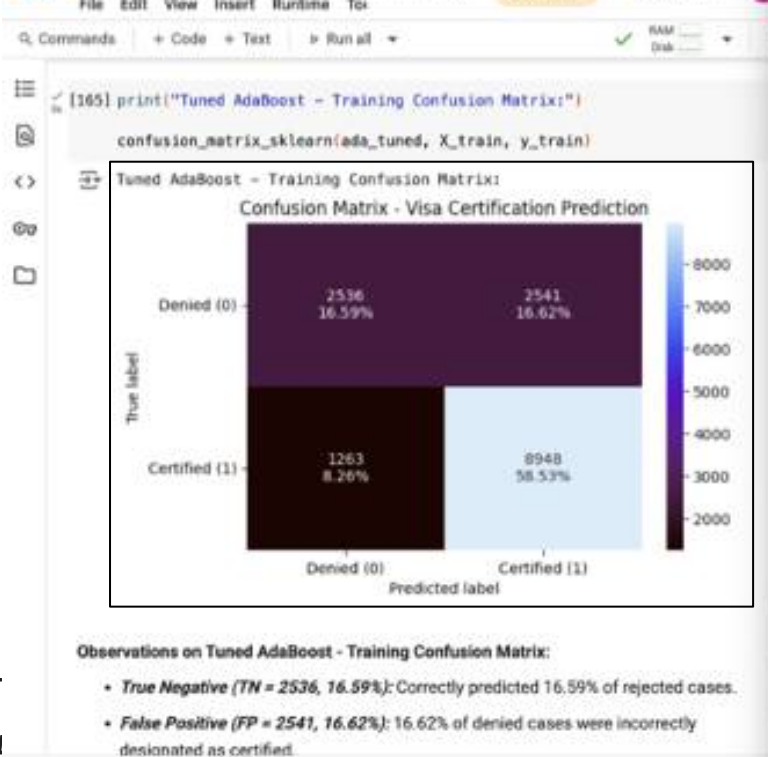
Observations on Tur

- **Accuracy** (0.743721)
- **Recall** (0.876579)
- **Precision** (0.771)
- **F1 Score** (0.820407)

Conclusions:

- AdaBoost's v
overfitting.

Confusion Matrix

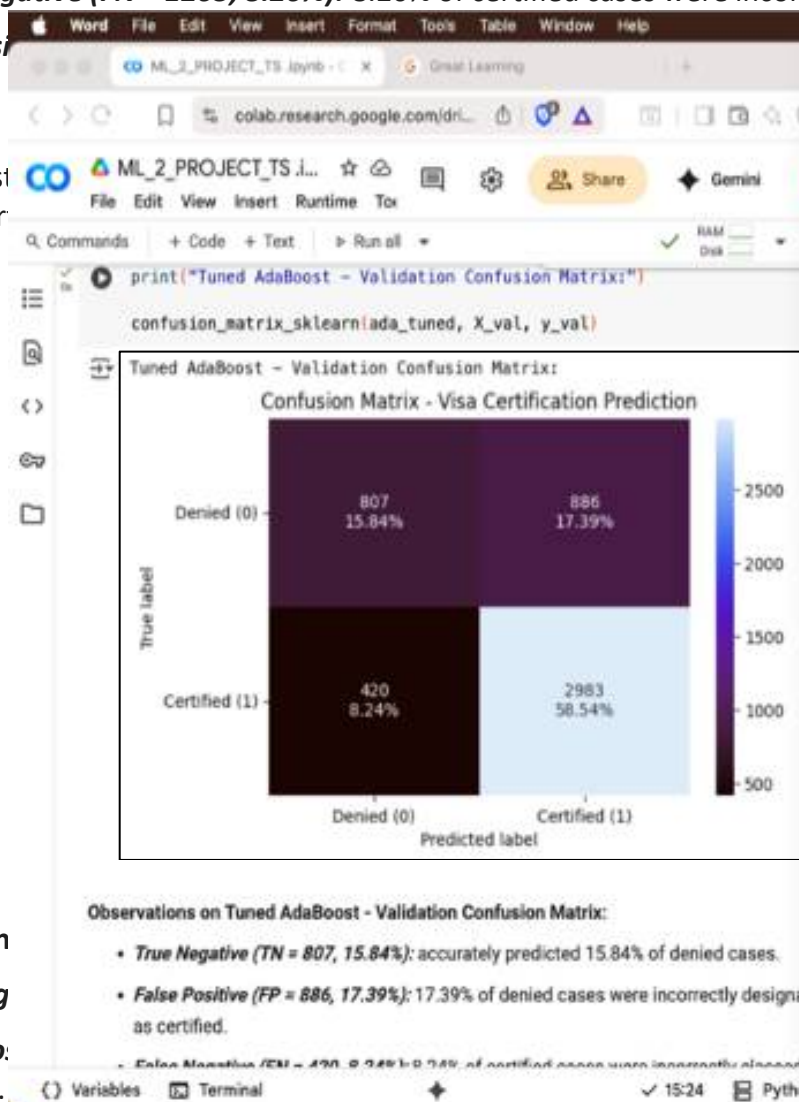


Observations on T

- **True Negative (TN = 2536, 16.59%):** Correctly predicted 16.59% of rejected cases.
- **False Positive (FP = 2541, 16.62%):** 16.62% of denied cases were incorrectly designated as certified.
- **False Negative (FN = 1263, 8.26%):** 8.26% of certified cases were incorrectly classed as rejected.
- **True Positive (TP = 8948, 58.53%):** 58.53% of certified cases were correctly designated as certified.

Conclusions:

- AdaBoost performs well in more certified cases are more



Observations on

- **True Negative (TN = 807, 15.84%):** accurately predicted 15.84% of denied cases.
- **False Positive (FP = 886, 17.39%):** 17.39% of denied cases were incorrectly designated as certified.
- **False Negative (FN = 420, 8.24%):** 8.24% of certified cases were incorrectly classed as rejected.
- **True Positive (TP = 2983, 58.54%):** 58.54% of certified cases were correctly designated as certified.

Observations on Tuned AdaBoost - V

- **Accuracy (0.7437):** Validation
- **Recall (0.8766):** Recall is stea
- **Precision (0.7710):** Slightly low case correctly.
- **F1 Score (0.8204):** This score i

Conclusions:

- AdaBoost's validation measur overfitting.

Confusion Matrix

certified.

- **False Negative (FN = 1263, 8.26%):** 8.26% of certified cases were incorrectly classed as rejected.
- **True Positive (TP = 8948, 58.53%):** 58.53% of certified cases were correctly designated as certified.

Conclusions:

- AdaBoost performs well in more certified cases are m

Concl

9

Ti

Ti

Si

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

```
xgb_train_perf_tuned.T, # Tuned XGBoost training performance
rf_train_perf_tuned.T, # Tuned Random Forest training performance
ada_train_perf_tuned.T # Tuned AdaBoost training performance

],
axis=1,

)
```

Word File Edit View Insert Format Tools Table Window Help

ML_2_PROJECT_TS.ipynb - x Great Learning

colab.research.google.com/dri...

ML_2_PROJECT_TS.i... ☆ ☁

File Edit View Insert Runtime To

Q Commands + Code + Text ▶ Run all ▼ RAM Disk

```
gb_val_perf_tuned.T, # Tuned Gradient Boosting validation perform
xgb_val_perf_tuned.T, # Tuned XGBoost validation performance
rf_val_perf_tuned.T, # Tuned Random Forest validation performance
ada_val_perf_tuned.T # Tuned AdaBoost validation performance

],
axis=1,

)

models_val_comp_df.columns = [
    "Gradient Boosting (Tuned)",
    "XGBoost (Tuned)",
    "Random Forest (Tuned)",
    "AdaBoost (Tuned)"
]

print("Validation performance comparison (Tuned Models):")
models_val_comp_df
```

Validation performance comparison (Tuned Models):

	Gradient Boosting (Tuned)	XGBoost (Tuned)	Random Forest (Tuned)	AdaBoost (Tuned)
Accuracy	0.746860	0.746860	0.747841	0.743721
Recall	0.891566	0.884808	0.887746	0.876579
Precision	0.767130	0.770274	0.769878	0.771000
F1	0.824681	0.823578	0.824621	0.820407

Next steps: Generate code with models_val_comp_df View recommended plots New intera

Summary on Tuned Model Comparison and Final Selection:

{} Variables Terminal 15:24 Python 3 Page 99 of 99 17774 w

Rejected models:

- Gradient Boosting** works well with excellent recall and balanced precision, but *its F1 score on training data is somewhat lower than Random Forest*, indicating a little less ideal balance of false positives and false negatives.
- XGBoost** is similar to Gradient Boosting, but with somewhat *lower F1, recall, and precision*. Given the crucial need to reduce errors in visa approvals, even minor discrepancies matter.

- **AdaBoost** has the **lowest F1 score and recall of any model**, increasing the likelihood of rejecting competent candidates, which is not aligned with the primary objective.

Selected Model- Tuned

- Random Forest model achieved the best performance with **validation (0.825)**.
- It also has the best precision as well as high recall.
- The **model generation** option for generating the model.

10 FINAL MODEL

10.1 Performance

Checking Performance

Observations:

- **Accuracy (0.7402)**: The model accurately predicts around 74.02% of the test set, showing good overall performance.
- **Recall (0.8825)**: A high recall indicates that the model accurately detects 88.25% of certified cases, lowering false negatives and reducing the risk of dismissing qualified candidates.
- **Precision (0.7648)**: A precision of 76.48% indicates that the majority of positive predictions (certified cases) are correct, which limits false positives.
- **F1 Score (0.8194)**: The F1 score, which is the harmonic mean of precision and recall, is 0.8194, indicating a good balance between precision and recall.

Conclusion:

- The **Tuned Random Forest model performs well on the test set**, with only a minor drop in performance compared to training and validation. This consistency provides **high generalisation** ability and reliability for real-world visa certification predictions.

Final Model- Tuned Random Forest

Checking Tuned Random Forest - Final Model's Performance on Test Set

```
[169] # Evaluate Random Forest performance on test set
rf_test_perf = model_performance_classification_sklearn(rf_tuned, X_test,
print("Tuned Random Forest - Test Performance:")
rf_test_perf
```

Tuned Random Forest - Test Performance:				
	Accuracy	Recall	Precision	F1
0	0.740188	0.882491	0.764766	0.819422

Observations:

- **Accuracy (0.7402)**: The model accurately predicts around 74.02% of the test set, showing good overall performance.
- **Recall (0.8825)**: A high recall indicates that the model accurately detects 88.25% of certified cases, lowering false negatives and reducing the risk of dismissing qualified candidates.
- **Precision (0.7648)**: A precision of 76.48% indicates that the majority of positive predictions (certified cases) are correct, which limits false positives.

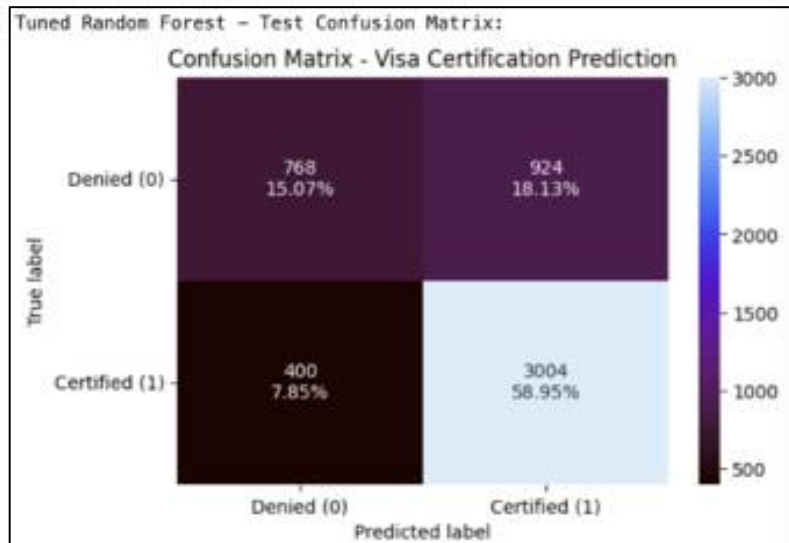
Variables Terminal 15:24 Python 3 Page 100 of 100

10.2 C

Tuned RF C

Confusion Matrix-Tuned Random Forest Model

```
[170] # create confusion matrix
print("Tuned Random Forest - Test Confusion Matrix:")
confusion_matrix_sklearn(rf_tuned, X_test, y_test)
```



Observations on Tuned Random Forest - Test Confusion Matrix:

- **True Negative (TN) = 768 (15.07%):** The model accurately predicted 15.07% of denied

Observations

- True
- False
- False
- True

Conclusion:

- The
- The

- The final model **performs well** at detecting certified applicants (high TP), but a significant number of denied cases are misclassified (high FN), indicating moderate sensitivity for class 0.
- There are **no signs of overfitting** because performance measurements are consistent across test data.

Predictions on Test Set & Classification Report

```
# Generate classification report
from sklearn.metrics import classification_report
y_test_pred = rf_tuned.predict(X_test)

print("Detailed Classification Report:")
print(classification_report(y_test, y_test_pred))
```

Detailed Classification Report:

	precision	recall	f1-score	support
0	0.66	0.45	0.54	1692
1	0.76	0.88	0.82	3484
accuracy			0.74	5096
macro avg	0.71	0.67	0.68	5096
weighted avg	0.73	0.74	0.73	5096

Observations:

Class 0 (Denied cases)

10.3 Pr

Test Predict

Observations:

Class 0 (Denied cases)

- **Precision (0.66)**, meaning when the model predicts denial cases, it is correct 66% of the time.
- **Recall is relatively low at 0.45**, meaning that the model only **detects 45% of actual denied cases**, leaving out more than half of them.
- The **F1-score of 0.54** reflects this imbalance, indicating moderate performance in detecting rejected cases.

Class 1 (Certified Cases)

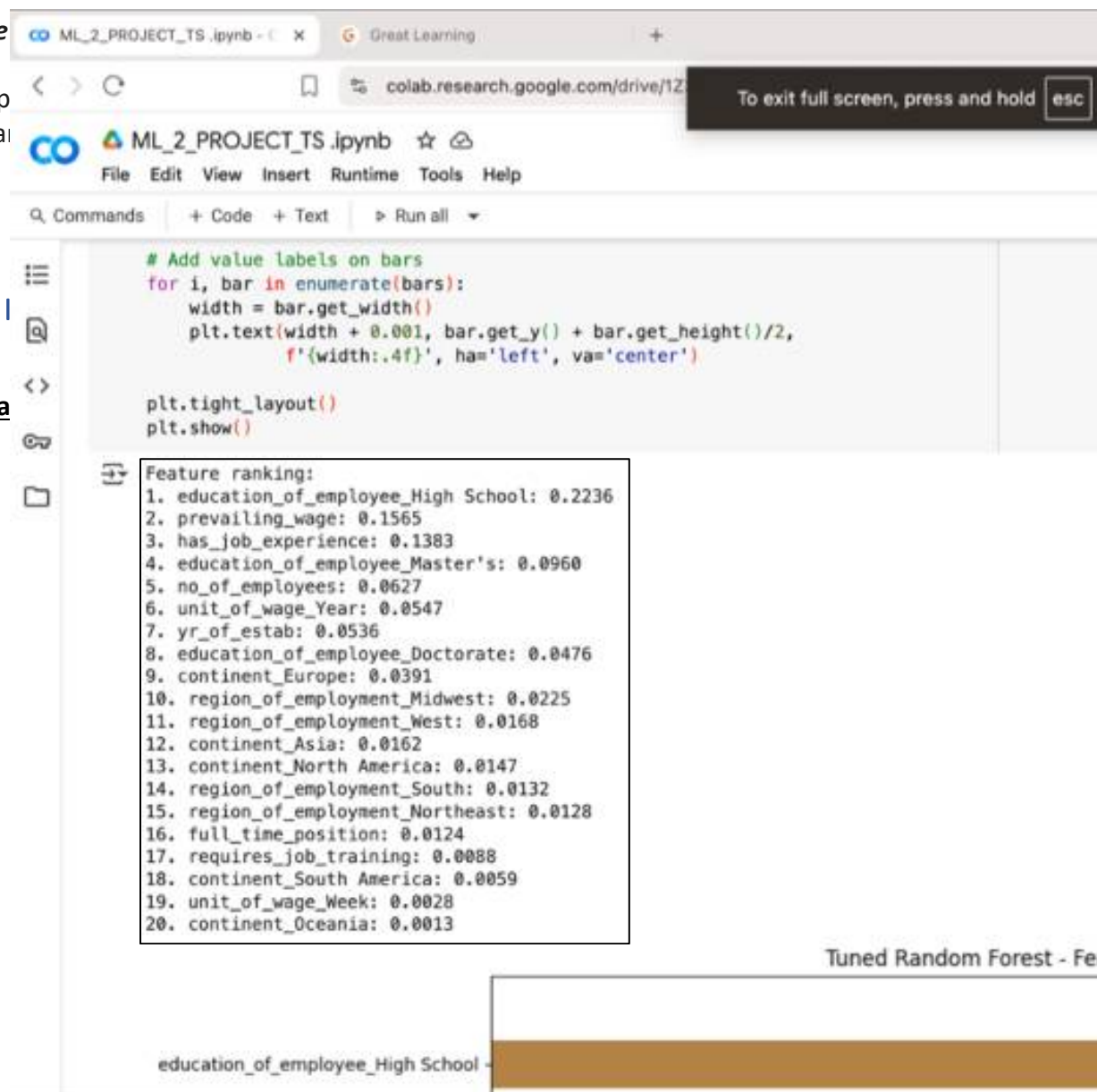
- **Precision is 0.76**, which means 76% of predicted certificates are correct.
- Recall is strong, at 0.88, indicating that the model correctly detects **88% of actual certified cases** while minimising false negatives.
- An **F1-score of 0.82** indicates excellent efficiency in recognising certified applications. Overall accuracy:
- The model has **74% accuracy**, correctly predicting around three-quarters of all cases.

Macro Averages

- Precision (**0.71**), recall (**0.67**), and F1-score (**0.68**) indicate that the model performs better on the majority class (certified) than the minority class (denied).

Weighted Average

- Weighted p
overall bal



The screenshot shows a Google Colab notebook interface. The top bar includes the file name 'ML_2_PROJECT_TS.ipynb' and the Google Learning logo. The notebook's menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all' are visible. The code cell contains a loop that adds value labels to bars in a plot. The output cell displays a 'Feature ranking:' list of 20 features with their corresponding scores. The top features are 'education_of_employee_High School' (0.2236) and 'prevailing_wage' (0.1565). The bottom of the notebook shows the start of a 'Tuned Random Forest' model.

```
# Add value labels on bars
for i, bar in enumerate(bars):
    width = bar.get_width()
    plt.text(width + 0.001, bar.get_y() + bar.get_height()/2,
            f'{width:.4f}', ha='left', va='center')

plt.tight_layout()
plt.show()
```

Feature ranking:

1. education_of_employee_High School: 0.2236
2. prevailing_wage: 0.1565
3. has_job_experience: 0.1383
4. education_of_employee_Master's: 0.0960
5. no_of_employees: 0.0627
6. unit_of_wage_Year: 0.0547
7. yr_of_estab: 0.0536
8. education_of_employee_Doctorate: 0.0476
9. continent_Europe: 0.0391
10. region_of_employment_Midwest: 0.0225
11. region_of_employment_West: 0.0168
12. continent_Asia: 0.0162
13. continent_North America: 0.0147
14. region_of_employment_South: 0.0132
15. region_of_employment_Northeast: 0.0128
16. full_time_position: 0.0124
17. requires_job_training: 0.0088
18. continent_South America: 0.0059
19. unit_of_wage_Week: 0.0028
20. continent_Oceania: 0.0013

Tuned Random Forest - Fe

education_of_employee_High School -

10.4 Feature I

Checking Tuned Ra

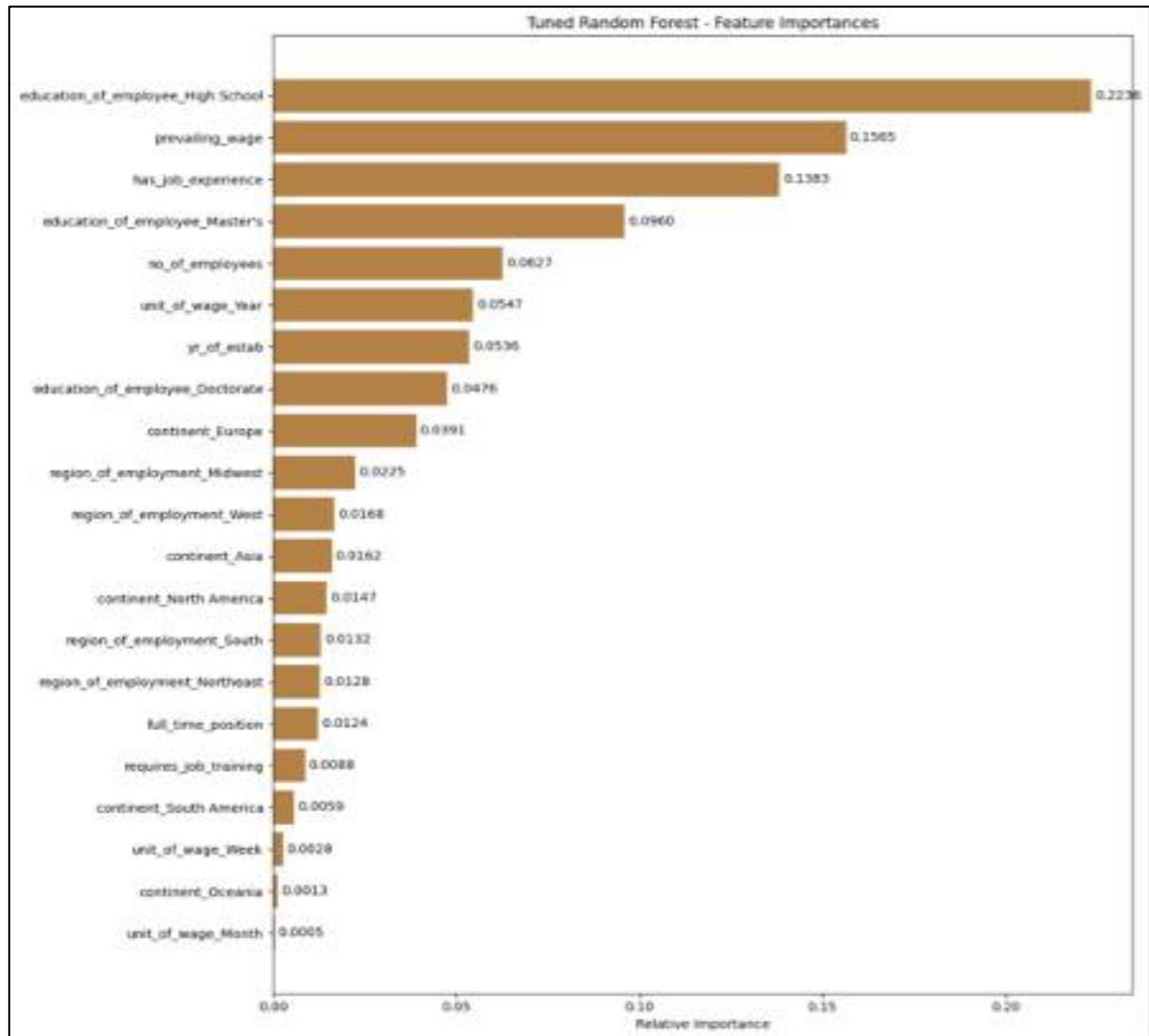


Figure 87. Feature Importance Plot

Feature Importance Observations:

- **education_of_employee_High School** has the most importance (**0.2236**) because applicants with only a high school education are more likely to be denied, making this attribute a strong predictor of rejection.
- Although **Master's (0.0960)** and **Doctorate (0.0476)** degrees are associated with faster acceptance, their **lesser relevance** shows that these categories are less influential individually or in the data.
- **Prevailing_wage (0.1565)** and **has_job_experience (0.1383)** are **significant predictors**, indicating that higher earnings and relevant experience improve the likelihood of certification.
- Organisational characteristics such as **no_of_employees (0.0627)** and **yr_of_estab (0.0536)** **have a moderate influence** on the model, demonstrating that company size and establishment year affect results.
- Geographical factors such as **region of employment & continent** have a **lesser** but cumulative impact, with location playing a little contribution.

Conclusion:

- Overall, the model focuses on factors that help distinguish refused applicants (e.g., high school education), which is critical for dealing with class imbalance and enhancing predictive accuracy.

11 ACTIONABLE INSIGHTS

Education plays a critical role

- Applicants with **Master's, Bachelor's, and Doctorate degrees** have a much higher approval rate compared to High School graduates.
- A “High School” education was found to be the most significant negative factor in visa denials.

Work experience strongly influences approval

- Applicants with prior **job experience** are significantly more likely to get approved than those without.

Wage level matters, but only marginally

- Higher **prevailing wages** slightly increase the chances of approval, but it is not a deciding factor. Extremely low wages are more likely to lead to denials.

Company size adds credibility

- Applications from **larger employers (higher employee counts)** tend to have slightly higher approval rates, likely due to greater financial stability and credibility.

Company age (year of establishment) is neutral

- The data shows **no significant difference** in visa outcomes between older, legacy companies and newer ones.

Employment type is a secondary factor

- **Full-time roles** have higher approval rates compared to part-time positions, but the effect is small compared to education and experience.

Geographic & demographic factors matter less

- The region of employment and continent of origin show only minor influence compared to education, experience, and wage.

Model performance trade-off

- The **Tuned Random Forest model** achieved about 74% accuracy with high recall (88% for certified cases).
- However, about 18% of denied cases were incorrectly classified as certified, indicating that the model favours approvals more than denials.

12 BUSINESS RECOMMENDATIONS

Prioritize applicants with higher education and experience

- Visa authorities and EasyVisa should **flag applicants with Master's/Doctorate degrees and prior work experience** as strong candidates for certification.

Enhance applicant profiles with training/experience

- For applicants with only High School education or no work experience, EasyVisa can **recommend additional certifications, internships, or training programs** to strengthen their profiles.

Monitor wage benchmarks

- Employers should be encouraged to **offer at least median-level prevailing wages (about \$70k)** to improve certification chances and avoid denials based on low wage offers.

Build trust with smaller companies

- Since smaller employers have lower approval odds, EasyVisa could **help them document financial stability, compliance history, and legitimacy** to reduce perceived risks.

Improve decision support for authorities

- Use the **Tuned Random Forest model as a screening tool** to automatically shortlist high-probability applicants.
- Visa officers can then focus their manual reviews on borderline or high-risk applications, saving time and effort.

Use insights for advisory services

- EasyVisa can create **advisory reports for employers and applicants**, showing how factors like education, wage, and experience impact approval odds.
- This positions EasyVisa not just as a service provider but also as a **strategic consultant** in visa processing.

Continuous monitoring & retraining

- The model should be **retrained annually** as visa approval trends, wage standards, and labour market conditions change.